In-Situ Interactions for Direct Manipulation of Parameterized Scenes

by

Joonho Jonathan Kim

A thesis submitted in conformity with the requirements
for the degree of Doctor of Philosophy

Graduate Department of Computer Science
University of Toronto

© Copyright 2026 by Joonho Jonathan Kim

In-Situ Interactions for Direct Manipulation of Parameterized Scenes

Joonho Jonathan Kim
Doctor of Philosophy

Graduate Department of Computer Science
University of Toronto
2026

Abstract

3D graphical scenes are often the result of a construction history of modeling and rendering operations controlled by a multitude of user-editable parameters. Creators can often describe their desired visual changes spatially, but the parameter controls used to produce these changes are typically buried deep in traditional stand-alone graphical user interface with lists of numerical knobs, drop-downs, and graph interfaces. This thesis explores how visual scene geometry and perception can be leveraged to design in-situ and direct manipulation interactions that provide an efficient and expressive entry point into controlling the hidden, high-dimensional parameter space and construction history of objects in 3D scenes. Design principles of in-situ and direct manipulation are used to propose techniques to spatially query and browse the construction history of objects, manipulate object attributes interactively using squidgets (sketch widgets), and to automatically compute optimal layouts of in-situ attribute controls on 3D scene objects. The techniques show that in-situ and direct manipulation of 3D scene objects can be both engaging and effective.

Soli Deo Gloria

Acknowledgements

I thank my advisor Dr. Karan Singh for supporting me throughout this degree. I came in with little to no research experience, hoping to solve ambitious problems in hand-drawn animation. Quickly after arriving, I struggled with motivation and productivity due to covid, paper rejections, disinterest in the topic, and failed experiments using neural methods. It wasn't until 4 years in that I decided to pursue teaching over research and found a deeper interest in the field of interface design and HCI. You supported my new direction by showing me unlimited patience and grace during my slumps and aggressively pushing me towards conference deadlines.

Your hands-off style of advising (something I initially struggled with) gave me freedom to explore new ideas and room to grow in my own maturity in research, critiquing, writing, and thought. Your ability to always spin ideas in multiple different directions taught me there's an even grander vision of the problem than what I currently see. Your reminder to pursue novel modes of interaction rooted in human experience reminds me that breakthrough can come from observing everyday people. These lessons over the years have manifested in my work outside of research, reflections on everyday events, and interactions with people and their ideas. I attribute my growth and confidence as a capable academic worker to you.

I thank my committee member and co-author Dr. Fanny Chevalier for teaching me about running user studies well and the integrity of fair assessments. Walking me through the thoughtful design of the user study taught me the importance of reducing bias and grew my deeper interest in user experience and careful design.

I thank my committee member Dr. Alec Jacobson for asking me a thoughtful question in one committee meeting. During covid while I was struggling with my progress, you asked me if I was doing okay. That question left a deep impression on me; in an environment where students and professors have a relationship of technicalities and ideas, asking about a student's well-being is a fresh reminder that everyone goes through things in life outside the lab.

I thank my "master's advisor" Dr. Greg Turk for introducing me to the field of computer graphics and academic research. You met with me weekly as I learned computational geometry through 3D printing and encouraged me to volunteer at Siggraph 2017. Your support helped propel me to continue researching in Toronto and showed me a new career direction in academia.

I thank Prof. Jennifer Campbell, Prof. David Liu, and the other teaching faculty in DCS for giving me the opportunity to teach courses. During these classes, I improved as a speaker, manager, and educator and formed my teaching philosophy to create story-driven content.

I thank my labmates Karran, Zhecheng, Evan, and Bingjian for being my bros in the lab.

I thank my family, Sung Kim, Dr. Kyung-Mi Kim, and Dr. Joonsung Kim, for their love and support during my entire life. You've always believed in my capabilities and encouraged me to pursue greater education and experiences in life.

I thank Jessie Kim for loving me and supporting me through the incredibly difficult last mile of my thesis. You suppressed your needs to help me focus and continually provided for me in small, thoughtful ways. I am so lucky to be with you.

I thank my church New Hope Fellowship Downtown for being my community and family: my close ELT friends Gigi, Seong, Brendan, Joyce, Christie, Maisie, and Pastor John for walking together to grow as leaders, disciples, planners, and friends; Life Group Ministry members Gigi, Toluwa, Constance, and Becky for creating an important experience for other church members; Rachel,

Cherlene, Jenny, Grace for being amazing friends. You all were my community away from home and taught me to be a more thoughtful and caring friend.

I thank Isaac + Rosalynn, Patrick + Hannah, Jaewon + Helen, Soohan + Emrys, Miki, Juheon, and Dr. Mary Hudachek-Buswell for supporting me from a distance.

And, to my Lord Jesus Christ who loved me so much that he died so that I may truly live a beautiful life.

Contents

1 Introduction **4**

1.1 Graphical Parameterized Scenes . . . 4
1.2 Detached Interface Elements and Translation Cost . . . 5
1.3 In-situ Interactions . . . 7
1.4 Direct Manipulation . . . 8
1.5 A Combined Interaction . . . 10
1.6 Contributions . . . 11
1.7 Thesis Organization . . . 12

2 Related Work **13**

2.1 Editable Graphics and Hidden Control Structures . . . 13
2.2 Detached Control Representations and Translation Costs . . . 14
2.3 Direct and In-Situ Interaction as Situated Control . . . 14
2.4 Visual Handles, Proxies, and Scene-Referential Input . . . 15
2.5 Mapping Visible Evidence to Hidden Structure . . . 16
2.6 Positioning This Thesis . . . 17

3 In-situ Visual Querying and Navigation of Dependency Graph Structures **18**

3.1 Introduction . . . 19
3.2 Background . . . 21
3.3 Design and Overview . . . 24
3.3.1 Graph-interface Workflow . . . 24
3.3.2 Design Principles . . . 24
3.3.3 Overview . . . 25
3.4 Method . . . 26
3.4.1 Visual Querying . . . 27
3.4.2 Clustering . . . 29
3.4.3 In-Situ Slider Creation . . . 30
3.5 Examples . . . 33
3.5.1 Structural Creation . . . 33
3.5.2 Composition . . . 33
3.5.3 Textural and Mixed Change . . . 37
3.6 Discussion and Direction . . . 39
3.6.1 Query Ambiguity . . . 39

3.6.2 Representative Selection and Summary . . . 40
3.6.3 View-Dependent Sliders . . . 42
3.6.4 Future Direction . . . 43
3.7 Conclusion . . . 43

4 Squidgets: Sketch-based Widget Design for Scene Manipulation 45
4.1 Introduction . . . 46
4.2 Related Work . . . 48
4.3 The Squidgets Framework . . . 50
4.3.1 Design guidelines . . . 51
4.3.2 Inferring scene curves and attributes . . . 51
4.3.3 Stroke to select a squidget curve . . . 52
4.3.4 Stroke as a squidget manipulation handle . . . 52
4.3.5 Rig and Bookmark squidget curves . . . 53
4.4 Implementation . . . 54
4.4.1 Perceptual curve segment matching . . . 54
4.4.2 Squidget curve creation . . . 56
4.4.3 Squidget curve selection . . . 57
4.4.4 Squidget manipulation . . . 57
4.5 Applications . . . 58
4.6 Evaluation . . . 59
4.6.1 Controlled experiment: manipulate objects . . . 60
4.6.2 Impressions from visual artists . . . 63
4.7 Discussion . . . 65
4.7.1 Limitations . . . 66
4.7.2 Future work . . . 66
4.8 Conclusion . . . 67

5 Optimizing UI Layouts for Deformable Face-Rig Manipulation 68
5.1 Introduction . . . 69
5.2 Related Work . . . 71
5.3 Design Principles . . . 72
5.3.1 Feeling of Direct Manipulation . . . 73
5.3.2 UI Elements . . . 74
5.3.3 UI Layout Design Objectives . . . 75
5.4 Layout Optimization Algorithm . . . 76
5.4.1 Integer Quadratic Programming . . . 77
5.4.2 Optimization Pre-processing . . . 79
5.4.3 Implementation Details . . . 82
5.5 User Study . . . 82
5.5.1 Experiment Setup and Protocol . . . 83
5.5.2 Analysis . . . 84
5.6 Results and Discussion . . . 87
5.7 Conclusion . . . 90

6 Toward Adaptive In-Situ Workflows **92**
6.1 Adaptive In-Situ Workflows 92
6.1.1 Context Disambiguation 93
6.2 In-Situ AI Workflows 94
6.2.1 Prompt-Based Workflows and Their Limits 94
6.2.2 From Prompting to In-Situ Direct Manipulation 95
6.2.3 Parameterized Scenes and Latent Representations 96
6.2.4 AI Support for Adaptive In-Situ Workflows 97

7 Conclusion **99**

Bibliography **101**

Chapter 1

Introduction

Creating in graphics begins from a visible scene. An artist sees or imagines a face, a shape, a curve, a light, a material, or a modeled object, and describes desired changes in visual terms: this region should move, this expression should soften, this curve should bend, or this part of the result should be inspected more closely. The software representation behind that scene, however, is usually not organized in the same visual terms. It is organized through digital representations such as low-level parameters, rig controls, scene attributes, procedural operations, construction histories, and dependency graphs. This separation between visible intent and underlying control structure is the starting point for this thesis.

1.1 Graphical Parameterized Scenes

Imagine a dense jungle of lush vegetation: hazy rays of light threading through the gaps of leaves differently shaped and differently shaded, hanging from tall, crooked trees bowing over an earthy dirt pathway lined with stubby shrubs. How do we create a digital version of this imaginary scene? There are many questions we need to ask: How hazy is the fog in the air? What shade of green are the leaves? How tall are these trees and how short are the shrubs? How bumpy is the dirt pathway? Hundreds of visual questions can be asked about the scene, and each one must have a defined answer in order for the digital scene to be fully rendered. Each of these visual questions and defined answers constitutes a parameter of the scene, and the collection of these parameters forms the *parameterized scene*.

Parameters are property values that determine characteristics of entities in a graphical scene. For example, the jungle scene may include the following parameters: the intensity of light rays and diffuseness of the fog body, the material color of the leaves, the height and diameter of the tree trunks, the roughness of the dirt pathway, and the density of shrub leaves. Some of these values are low-dimensional: the intensity of light can be represented as a continuous numerical value between 0 and 1, and the color of the leaves can be represented as a combination of color channels such as RGB or HSV. But many are high-dimensional and characterize non-trivial relationships between other parameters: the geometry of a tree, the construction history of a procedural model, or the blendshape parameters of a face rig. Eventually, these parameters are processed through a rendering pipeline to produce the visual output of the scene.

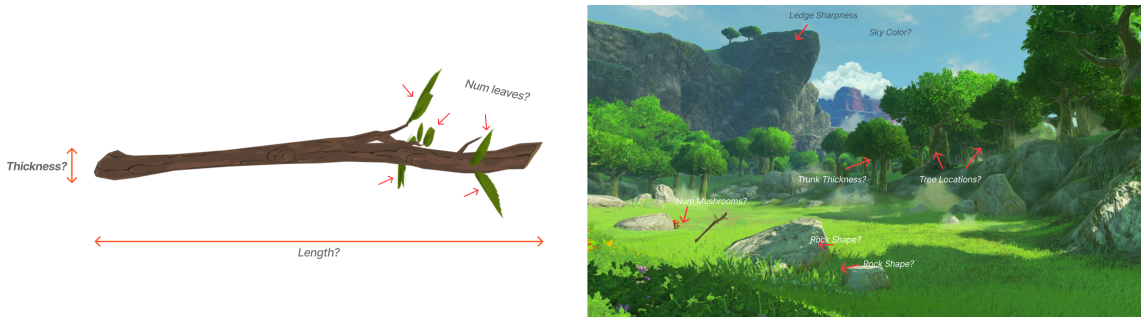


Figure 1.1: Parameters describe characteristics of scene entities like the branch's shape and leaves. A large number of parameters describe the visual characteristics of a parameterized graphical scene. Assets from *The Legend of Zelda: Breath of the Wild*.

Specifying these parameters is a fundamental part of designing a graphical scene. Many are available automatically during creation by the software from formal definitions of the scene entities, mostly mathematical definitions. These pre-defined parameters are general-purpose, simple, and reusable across different scenes with pre-defined relationships to other parameters. Relationships can be linear/non-linear functions, causal operator chains, hierarchical groupings, or semantic and custom relationships. If pre-defined parameters do not capture desirable concepts for the scene, artists can explicitly define custom parameters and relationships to better organize entities, utilize novel representations, or write custom algorithms. Our jungle scene may have geometry to define the shape of simple 3D primitives such as spheres, cubes, and cylinders, but an artist may create trees from these primitives by defining a custom parameter that controls the shape of a tree, which is mapped to the underlying geometry parameters of the primitives.

1.2 Detached Interface Elements and Translation Cost

With the parameters of a scene defined, how does a user update the data and parameters of a graphical scene to fit their visual intent? Modern graphical interfaces address this problem by creating a layout with the scene viewer in the center of the screen and surrounding detached interface elements that read and update the underlying parameters. Simple elements such as sliders, check boxes, dropdown menus, and text boxes make scene state visible and editable without requiring users to manipulate raw data directly. These interface layouts follow the traditional WIMP (windows, icons, menus, pointer) paradigm of graphical user interfaces [75, 138]. Through these interactions, artists can edit these values through simple gestures, such as dragging a slider to increase the intensity of a light or selecting which tree trunk texture to use. With the vast number of parameters in a scene, these interface elements are essential for precision, inspection, and repeatability.

They work well for low-dimensional parameter spaces but become harder to use as graphical representations become more high-dimensional with complex parameters. High-dimensional parameter spaces can include parameters of mixed types bound together through complex relationships and mappings that, when edited, affect multiple aspects of the scene in non-trivial ways. To control these parameters, simple interface elements are either mapped directly to individual low-dimensional parameters, a setup that requires granular control over the underlying data, or mapped to higher-level parameters that reduce many underlying values into a smaller set of meaningful actions. As the

dimensionality of the parameter space increases, the former becomes cumbersome because it requires users to understand and manipulate too many low-level parameters (with possibly little visual feedback). The latter becomes hard to design because mapping many low-level parameters into a smaller set of meaningful actions can be difficult to create and keep visually separate, especially when the mapping is complex or non-linear. However, when mapping is successful and elements are designed well, these controls can be powerful for artists manipulating complex scenes.

Even with clearly defined mappings, another problem arises with detached interface elements: Translation cost. *Translation Cost* is defined as the cognitive cost of shifting attention between the visual subject and the controls. There are many factors that contribute to translation cost, but this thesis will highlight three of them:

— **Spatial distance**: the rendered distance between the visual subject and the controls.

— **Indirect control mapping**: the disconnect between an editing gesture and the visual change.

— **Search effort**: the energy spent finding the relevant parameter and control to manipulate the visual subject.

In other literature, this can be called the split-attention effect or spatial contiguity principle [28, 101].

In our jungle scene, say the artist wants to change the shape of the tree using sliders that update height and radius. In a detached interface, the sliders would appear in a separate panel adjacent to the visual scene containing the tree. Interaction would consist of shifting attention between the visual changes in the scene and interaction gestures with the sliders. Updating the height of the tree would require a horizontal dragging gesture that changes a vertical component of the tree. Then, to update another part of the scene, such as the dirt path, the artist would need to locate the detached controls that drive its roughness, which may be hidden behind a different panel or menu.

This seems like a trivial case of translation cost, but when spread across many parameters, management of the detached interface elements quickly grows into a significant cognitive load. Updating hundreds of parameters multiple times during a workflow can require a large amount of attention to be spent on the interface itself, rather than on the visual subject. Common interface elements are meant to cover a wide range of parameters across many different scenes, but the visual mapping between the interaction gesture and the visual effect may not be clear. Then, with large parameter spaces, the relevant controls may be hidden behind tabs, shelves, or menus, and may require searching, reading labels, or remembering tool-specific conventions to find the right control.

This is where the problem becomes less about parameter count alone and more about the mental management of the interaction itself. An interface may expose many parameters whose labels or locations do not immediately reveal which part of the rendered scene they affect. In these cases, the artist can often imagine what should change, but still needs to explore and locate the parameters that can change it.

The practical cost is a shift of attention. Instead of asking only what should change in the scene, the user must also ask where the relevant control is, what it is called, how it affects the scene, and whether the desired control exists at all. This is similar to working in an unfamiliar workshop: the tools may be available, but the work slows when the worker must continually search through drawers whose organization changes from project to project. In graphical systems, this problem appears across many custom tools and asset-specific control spaces. As graphical scenes become more

complex and customized, and as graphical software becomes more bloated with tools and features, the translation cost of visible intent into detached interface controls becomes more important.

1.3 In-situ Interactions

One promising response to this interface problem is to bring detached controls into the scene itself by anchoring the elements onto or near the scene entities they affect. For example, when the artist wants to adjust the size of the tree trunks, selecting the trunk would render the visual sliders locally on or near the trunk itself, rather than in a detached panel. This simple idea of placing interface elements within the scene is called ***in-situ***, where the interface elements are placed in the same location as the scene objects they control.

In-situ interaction is more than a spatial translation of interface elements into the viewport; merely moving the detached interface elements into the viewport would not be considered in-situ. An interface element that appears in the viewport but is statically placed on the 2D screen space, such as a heads-up display (HUD), would be considered in-scene, not in-situ. Thus, an important principle for in-situ interaction is the *visual anchoring* of the controls to the scene entities: any camera changes, entity transformations, or scene updates that affect the location of the scene entities must also update the location and rendering of the interface elements to maintain their visual connection. This continuous tracking evokes an implied connection between the two, suggesting that the in-situ element affects the object it is anchored to. A toggle menu anchored to the clouds in the sky through camera changes suggests the options will change a parameter related to the clouds. We see this principle applied in the real world: door knobs are attached to the doors they open, light switches in a room illuminate the lights in that room, and an on-off switch controls a laptop. This follows Beaudouin-Lafon's instrumental interaction model, where users act on domain objects through instruments that provide action and feedback [13]. In this thesis, an in-situ handle can be understood as such an instrument with a low degree of indirection from low spatial offset.

Anchoring also implies that the effect invoked by the in-situ interaction is localized to the region of the scene entity where the in-situ element is anchored. If the same toggle menu on the cloud produced a visual change around a shrub on the ground, this would weaken the in-situ connection and even recreate the same translation cost problem as a detached interface element. Local visual feedback reinforces the interaction and visual changes and makes it easier to understand the purpose of the control.

Global changes loosely benefit from in-situ interactions. A global visual change across the entire viewport may not provide a visually clear anchor location for the in-situ element because no local region may characterize the entirety of the change. Instead, the details of the visual change may be abstracted away as just an overall change. Conversely, anchoring an in-situ element that produces a scene-wide effect may still be an acceptable interaction by reducing spatial distance, but still weaker than an interaction that would produce a local change. The interface element could be set aside as an in-scene interaction or moved back as a detached interface element to save the visual real estate for an in-situ interaction that does produce a local change.

This visual real estate (or visual clutter) of in-situ elements becomes an important factor to consider when determining what interactions should be in-situ. In-situ elements appear in the viewport and will consequently occlude the rendered scene. If too many in-situ elements appear at

once, the rendered scene becomes visually cluttered and reduces the user's visual understanding of the scene. Contextualization from both a software and interaction standpoint can help reduce the number of in-situ elements that appear at once. Modern graphics software provides buttons to activate tool contexts that determine which controls are shown. Interactions that specify which entities to contextualize (selecting) limit in-situ elements to appearing only on the specified entities. The methods in this thesis do not explore methods of contextualization, but instead focus on the design of in-situ interactions themselves.

In-situ interactions address two factors of Translation Cost: the spatial distance problem by reducing the physical distance between the visual subject and the controls that affect it, and the search effort problem by making the controls appear in or near the rendered scene parameter they affect rather than in a hidden detached panel or menu. But in-situ interactions do not address the control mapping directly; artists may recognize there is a mapping that exists, but in-situ does not state which interaction gesture to drive the visual change. Instead, we can introduce another principle.

1.4 Direct Manipulation

Placing interface elements in the scene opens up an opportunity for powerful visual interaction: direct manipulation. ***Direct Manipulation*** is a term coined by Ben Shneiderman in 1982 [135] to describe user interface interactions that follow 4 principles:

— Continuous representation of objects of interest.

— Physical actions or labeled button presses instead of complex syntax.

— Rapid, incremental, reversible operations with immediate visible feedback.

— Learning that permits usage with minimal knowledge.

In graphical scenes, the principles characterize real-time interactions where gestures and consequential visual effects share a physically direct relationship with one another: dragging an object with a mouse, moving a character left and right using a joystick, or sketching over a curve to bend it.

The benefits of direct manipulation are the reduction of cognitive effort when performing an action. Rather than performing a sequence of keyboard commands, button presses, or menu selections to manipulate a parameter, the artist performs the command through a physical gesture that resembles the effect they want to see. Say an artist wants to change the roundness of a shrub's silhouette. Instead of interacting with a roundness parameter through a slider or text box, the artist can perform direct manipulation by grabbing the edge of the shrub and pulling to make it rounder, or pushing to make it more jagged. Instead of translating the shrub to a different location by inputting location parameters, the artist can directly drag the shrub in space and place it where they want. The parameter manipulation is embedded in the physical gesture, and the rendered change follows from that gesture, thus the user forms a physical connection to the visual change. In our parameterized scene, this is particularly powerful because the parameterized scene renders visual outputs, the work itself is visual, and thus the notion of editing the parameters of a visual scene through physical gestures that directly manipulate the visual output is a natural fit for graphical interfaces.

On the technical side, direct manipulation adds a new layer to interaction: physical gesture drives parameter manipulation, which produces a visual change. Assuming the mapping between parameter change and visual change is established, direct manipulation asks how to map a physical gesture to a parameter update that produces an output resembling physically direct control. A physical gesture that moves a mouse left but directs an object to transform right is not direct manipulation. A physical gesture that moves a mouse left but directs an object to transform left at a different displacement rate from the gesture *is* considered direct manipulation, but for this thesis we further require the feeling of direct control - a 1-to-1 mapping of manipulation.

One popular method for deriving this mapping is to calculate an inverse function of the parameter-to-visual mapping, which takes a visual change and calculates the parameter change that would produce it, as in inverse kinematics for posing articulated figures or direct manipulation of blendshape models [93, 156]. Then the physical gesture can be mapped to the visual change, and the visual change can be mapped back to the parameter change through the inverse function. For simple mappings such as translating an object in space, the inverse function can be solved analytically and direct manipulation can be achieved by mapping the physical gesture to the visual change with great success. However, for complex mappings such as changing the shape of a tree or posing a character, the inverse function can be difficult to calculate and may need to be approximated using numerical methods, machine learning, or optimization techniques.

The reward for finding this mapping is a powerful interaction that can feel efficient, physically natural, and fun to use. It is applied to a majority of graphical interactions from transforming objects in space, posing characters, sculpting shapes, adjusting material properties, and even navigating the viewport. Outside of graphics, direct manipulation techniques are used in robotics, XR, and data visualization, and have been shown to be effective for a wide range of users and tasks. Tasks that require real-time, iterative refinement are particularly well-suited for direct manipulation, such as updating shapes through deformation or manually arranging furniture in a room. Fortunately, many artistic tasks in graphics fall into this category, where artists often make expressive changes to the scene benefiting from the physical connection it provides.

However, the benefits of direct manipulation can be a double-edged sword. Direct manipulation makes it difficult to perform exact parameter changes, not because the mapping is flawed, but from the limitations of physical input itself. The physical precision of direct manipulation depends on the physical capabilities of the artist. Skillful artists may perform tasks more easily and quickly than novice artists. Additionally, to perform an exact parameter change, artists may be better off inputting exact values through a text box. Constraints added to direct manipulation can support more precise edits such as snapping to guidelines or restricting degrees of freedom.

Additionally, this style of interaction may be an expressive method of editing a scene, but not the most efficient. The physical demands of direct manipulation can be tiring to perform for long periods of time. If an artist has mapped out a sequence of exact editing operations, recording a macro or writing a script can be more efficient than repeatedly performing the same gestures. When the tasks are less established and expressive creation is the goal, direct manipulation still stands out.

Direct manipulation addresses the indirect control mapping problem by stating that parameter control should follow direct physical gestures. The mapping function is constrained to measuring not only the magnitude of gestural displacement but the motion trajectory as well. However, direct manipulation does not specify how the interaction is activated. The principle indirectly implies that

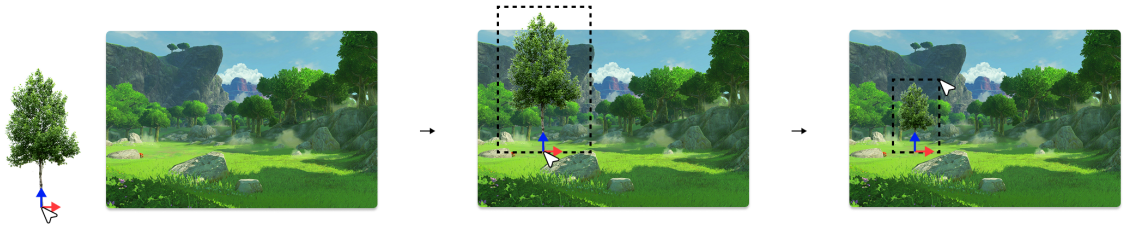


Figure 1.2: With in-situ interaction and direct manipulation, a translation widget is minimally rendered on top of the object. The artist can select a widget handle and drag the tree directly to a location in the scene. Another in-situ scaling widget is rendered on-top the tree to provide an interactive handle the artist can drag to scale down the tree.

successful direct control requires an interface that provides a direct experience.

1.5 A Combined Interaction

When combined, in-situ interaction and direct manipulation describe an interface style where users act on controls anchored to the visible scene entity they intend to change, using interactions that follow the visual effects being displayed, with feedback tied to the visual effect of that action. A good example of in-situ interaction and direct manipulation is a translation widget for a graphical object. The widget is displayed and anchored to the center of the object with 3 thin arrows pointed along the xyz axes. The user can drag an arrow element to translate the object with the mouse, and the visual change of the object follows the direction and magnitude of the dragging gesture. The benefit of this combined style is a visually compact interaction that reduces the translation cost of detached interface elements, while also providing a physical gesture to change the parameter. This interaction style is already prevalent within graphical software. The in-situ translate, rotate, and scale widgets with direct manipulation are standard for all transformation tasks in graphical software. Deformation handles that allow users to directly manipulate the shape of an object or rendering techniques to paint textures directly on the surface of an object are also examples of in-situ direct manipulation.

For simple scene objects, the mapping between an in-situ handle and the underlying parameter can be straightforward. For complex graphical representations, the mapping itself becomes the challenge. The system may need to decide where a control should appear, which parameter or attribute a gesture should manipulate, how the interface updates through interaction, which visible feature should become a handle, or which parameter groups are relevant to a selected output region. Though each decision can be determined separately, our goal is to explore interaction techniques that consider these decisions together as a design problem of the interaction itself.

One source of inspiration for designing these interactions is to use *visual evidence* from the rendered scene to help design of the interaction itself. Visual evidence, such as deformation trajectories, silhouette shape changes, or geometric features, can be utilized to anchor interaction handles, drive interaction gestures, and infer relevant parameters to manipulate. This evidence pairs well with in-situ interactions and direct manipulation for parameterized scenes because the rendered output is the basis for our visual questions, intentions, and interactions. Thus, inspecting visual evidence can extract contextual information and features as constraints to guide the design of the interaction itself.

The goal of my thesis expands on the following statement:

High-dimensional representations of graphical content (such as FACS blendshapes, vertex geometry for shapes, or construction history parameters of a scene), while conceptually well understood by humans, lack efficient interactive controls for object and scene manipulation. Humans, however, can comfortably specify desired changes visually by directly manipulating scene objects in-situ using physical gestures such as dragging or drawing. Providing visual techniques where users express desired scene changes directly and in-situ through inferred interaction handles can result in compelling user interfaces for exploring complex graphical scenes.

1.6 Contributions

This thesis makes the following contributions:

— **Conceptual contribution:** This thesis presents the idea that editing existing graphical scenes can be accomplished through in-situ direct manipulation interactions designed by the analysis of visible evidence from the scene itself. Specifically, visible evidence, such as rendered outputs, perceived curves, visual differences, and deformation trajectories, provide a rich source of visual information for identifying un-annotated yet relevant hidden scene parameters for manipulation, connecting visible features to editable attributes, and guiding the design (appearance, usage, mapping) of the in-situ direct manipulation interactions that control these found parameters.

— **Technical contributions:** This thesis contributes three interaction techniques:

  — Chapter 3: "In-situ Visual Querying and Navigation of Dependency Graph Structures" 2026. The system lets users highlight a visible region of a rendered procedural result, then generates a compact in-situ slider for navigating relevant construction-history states. This work studies how questioning visible output can become an entry point for exploring hidden dependency graph structures.

  — Chapter 4 [79]: "Squidgets: Sketch-Based Widget Design for Scene Manipulation." *UIST*, 2025. Squidgets use visible or perceived scene curves as interaction handles, allowing users to draw over the scene to select and manipulate associated scene parameters. This chapter studies how visible and perceived scene curves can become handles for hidden parameter control.

  — Chapter 5 [80]: "Optimizing UI Layouts for Deformable Face-Rig Manipulation." *ACM Transactions on Graphics*, 2021. Given a set of face-rig parameters and their deformation effects, the method creates an optimized layout that places interface elements near the facial regions they affect. This chapter studies how known high-dimensional parameters and their visual consequences can automate interface generation.

— **Empirical contributions:** This thesis contributes evidence about how these techniques behave in practice: a demonstration of visual querying across several types of dependency graphs; a controlled Squidgets study on the effectiveness to performing simple manipulation tasks, a

qualitative feedback study from visual artists, and proposed design guidelines for sketch-based widget design; and a face-rig UI study comparing performance with generated in-situ layouts against slider lists and an artist-created layout.

1.7 Thesis Organization

Chapter 2 reviews the broader interaction lineage behind this thesis, including direct manipulation, in-situ interfaces, visual handles, and methods that connect visual user input to hidden parameters. The three technical chapters then describe a progression from broad scene exploration to increasingly focused attribute control. Chapter 3 begins with an existing procedural scene whose dependency graph contains many objects, states, and attributes, and uses visual evidence from a selected output region to narrow the search space and find desired state and attributes. Chapter 4 then assumes the user is focused on relevant scene attributes and studies how scene rendering yields visible and perceived curves that act as direct interaction handles to manipulate underlying parameters. Chapter 5 goes further by using sampled visual trajectories from face rigs to generate persistent in-situ controls near the regions they affect. Chapter 6 discusses future directions for adaptive in-situ workflows, including the limits of in-situ interaction and the role of AI-assisted control. Chapter 7 concludes with a summary of the thesis contributions and directions for future research.

Chapter 2

Related Work

This thesis is organized around a recurring mismatch in graphical authoring. Artists judge and describe their work through visible results: a facial expression, a curve silhouette, a modeled part, or a region of a procedural asset. The systems that produce those results, however, are often controlled through structures that are not visible in the same way: rig parameters, scene attributes, procedural rules, construction histories, dependency graphs, and design spaces. I use *visible evidence* to describe the parts of the rendered artifact that motivate an action or question, such as highlighted regions, deformation trajectories, or perceived curves. I use *hidden control structure* to describe the underlying editable representations that produce those visible results. They are not hidden because they are inaccessible; many are exposed directly through channel boxes, node editors, parameter panels, and layer stacks. They are hidden in the sense that their relationship to the rendered result feels indirect.

This chapter reviews the broader lineage behind the thesis. It first describes why graphical artifacts are often represented as editable computational structures, then discusses the translation cost created by detached control representations. It then reviews direct manipulation, in-situ interaction, visual handles, and mapping techniques as ways to connect visible evidence back to hidden control.

2.1 Editable Graphics and Hidden Control Structures

Interactive graphics systems have long treated visual artifacts as editable computational structures. Sketchpad showed that drawings could be constructed and manipulated through drawing gestures rather than through text input alone [146], while early graphical procedure systems showed how computation itself could be represented as connected visual structures [147]. In graphics authoring, this idea appears in many forms: object transforms, animation rigs, material and shader networks, deformation models, procedural rules, simulation parameters, and graph-based construction histories. These structures make digital graphics powerful because they preserve editability, reuse, and non-destructive control. They also create a layer of abstraction between the visible artifact and the mechanisms that produce it.

Procedural graphics makes this distance especially clear. Rule-based and algorithmic models can generate plants, cities, buildings, surfaces, and shaders from compact editable rules [33, 122, 123]. Node-based authoring systems expose this editable structure through networks of operators and dependencies [56]. These representations are valuable because they make complex scenes reproducible

and adjustable, but they also require artists to understand the relationship between a visible result and a separate computational representation. An artist may see a deformation, material boundary, missing detail, or unwanted procedural artifact, yet the edit may live in a parameter, rule, subgraph, or earlier construction state.

The hidden structures considered in this thesis differ in form, but share a common interaction problem. They include known parameters and rig controls, scene attributes associated with visible geometry, procedural and dependency structure, construction histories, learned or sampled design spaces, and authored semantic controls. Some are explicit and named, while others are inferred or only visible through their effects. Some are local to a visible feature, while others act globally or through long chains of dependency. Across these cases, the user's intent often begins in the visible realm, while the software asks the user to act through another representation.

2.2 Detached Control Representations and Translation Costs

Conventional graphical interfaces make hidden control structures available through detached control representations. Interfaces such as sliders, text fields, channel boxes, graph editors, and asset browsers are effective because they expose precise editable state directly. They support labeling, inspection, debugging, organization, duplication, and precise control. When users know which parameters or elements to adjust, manipulation is straightforward.

The limitation appears when the user's intent begins from visible evidence rather than from a known interface element. In that case, the interaction task shifts from making an edit to finding where the edit can be made. Graph visualization helps users inspect graph-structured workflows, navigate histories, and compare computational states [61]. Visual programming research similarly shows that visual notations can improve closeness of match while still introducing costs such as viscosity, hidden dependencies, and secondary notation issues [53]. These approaches make complex structures more manageable, but they still often ask users to begin from the representation itself: a graph topology, parameter list, history view, or visual program.

High-dimensional parameter spaces create translation costs because large numbers of parameters require reduction techniques to manage. Design Galleries is the canonical example: it treated parameter setting as browsing a collection of *representative* visual alternatives rather than manually manipulating each dimension [99]. More recent design-space systems extend this idea through collaborative sampling and interactive optimization of large procedural spaces [24, 152]. These examples show how detached representations can be softened by making the space visible through examples. However, this strategy also shows the central tension of the thesis: a visual result can be easy to judge while the editable structure that produced it remains difficult to locate or control.

2.3 Direct and In-Situ Interaction as Situated Control

Direct manipulation provides a foundational interaction idea for reducing translation distance. Shneiderman characterized direct manipulation interfaces through visible objects of interest, incremental and reversible actions, and rapid feedback [135]. Hutchins, Hollan, and Norman further argued that directness is not just a property of input devices, but a relationship between user intentions, interface representations, and system state [65]. This framing is important for graphical authoring

because artists often know what they want in terms of a visible result, even when they do not know the parameter or graph operation that will produce it. Direct manipulation reduces the expressive distance between intention, action, and feedback when the object of interest can be acted on directly.

In-situ interaction adds a spatial and attentional dimension to this idea. Controls, feedback, or guidance are placed in the workspace where the user is already looking, rather than separated into fixed panels or command areas. Toolglass and Magic Lenses demonstrated how tools can be layered over the workspace [19], while augmented reality and tangible interaction research broadened this direction by asking how interfaces can draw on users' knowledge of spatial action and graspable objects [71, 72]. These examples show why spatially situated controls can reduce translation effort when the object of attention, the action, and the feedback remain in the same context.

Graphics and design systems inherit this principle when they keep editing actions, feedback, and guidance in the working view. Sketch-based and in-context design systems such as Teddy, ShadowDraw, and Insitu keep construction or guidance in the same visual context as the artifact being authored [67, 91, 119]. The common benefit is preservation of perceptual context: the user can express an action where the visual evidence for that action is already present.

This does not make detached interfaces obsolete. Panel-based controls, timelines, graph interfaces, and scripts remain better for naming, precise values, global operations, and explicit authoring. The value of direct and in-situ interaction is strongest when the user's intent is local, visual, and tied to a visible part of the artifact. This distinction becomes important for parameterized scenes, where several hidden controls may produce similar visible effects together and where a scene-embedded control can represent the correct underlying structure. Instrumental interaction is useful here because it describes interaction in terms of domain objects and instruments that mediate action on those objects [13]. The design problem is therefore not only to place controls near graphical objects, but to choose instruments whose behavior matches the user's visible intent and the structure of the underlying representation.

2.4 Visual Handles, Proxies, and Scene-Referential Input

Visual handles are one practical bridge between visible evidence and hidden control. A *visual handle* is a visible object, proxy, curve, marker, widget, or overlay that a user can act on and that the system maps to an underlying graphical structure. They are scene-anchored instruments that make a hidden control structure visible, local, and actionable.

Prior work has explored handles for editing and manipulation. Traditional manipulators and 3D widgets represent transformations or constrained spatial operations [17, 140]. Deformation proxies such as cages, wires, lattices, and bounded biharmonic handles expose lower-dimensional control structures for complex geometry [73, 130, 137]. Sketch-based widgets, sketch sliders, and gesture recognizers reinterpret controls for pen and stroke input [126, 128, 157]. These examples share a common goal: create a visible object that can stand in for a more abstract or higher-dimensional control.

Handles also support control layout. Rig interfaces and on-screen animation controls provide visual access to animation parameters, often through artist-authored curves or pickers that are easier to use than raw parameter lists. Face Poser and region-based face models show a related idea: controls can be organized around facial regions and deformation behavior rather than exposed only

as abstract parameters [88, 153]. In such systems, the handle is not only a way to issue a command; it is also a visual explanation of which part of the scene a parameter affects. This idea is important for high-dimensional graphical controls because a useful handle must disambiguate a collection of parameters as an explainable local effect.

Handles can also support navigation and explanation rather than only editing. Shape-collection summaries, learned variability models, and suggestion systems use visual abstractions as navigational proxies for large sets of alternatives [29, 118, 168]. Dynamic illustration systems use sketched motion controls and causal relationships as visible explanations of hidden animation behavior [77, 78]. Visual grammar editors similarly expose procedural rules through editable generated structures, making rule-level dependencies inspectable through the artifact itself [95]. A handle can therefore be a control layout element, a comparison object, a representative state, or a local summary of a larger hidden space.

This broad definition matters because the visible evidence used by artists is not always an existing widget. It may be a drawn stroke over a perceived scene curve, a deformation trajectory on a face, or a highlighted region of a rendered result. The paper chapters treat the domain-specific details of these cases. At the thesis level, the shared point is that scene-referential input becomes useful when it can be turned into a handle for an underlying editable structure.

2.5 Mapping Visible Evidence to Hidden Structure

A visual handle is only useful if interaction with that handle can be mapped back to the underlying structure. Sometimes this mapping is explicit and authored. A transform widget can update an object's matrix, and a rig picker can set a known parameter. In richer graphical scenes, however, the system may need to infer which hidden structure the user intends to control, inspect, or modify. This inference may involve selection, inverse mapping, parameter fitting, graph search, summarization, or layout optimization.

Inverse control problems appear across computer graphics: spatial goals, visible deformations, and drawn strokes must be mapped back to lower-level parameters or structures. Teddy is the canonical sketch-based modeling example, interpreting a user's strokes as 3D freeform geometry [67]. Demonstrational interfaces and programming-by-example research study how concrete user actions can define more abstract procedures [111]. Output-directed programming makes the connection even more explicit: systems such as Sketch-n-Sketch let users directly edit a visible output while the system infers changes to the hidden program that generated it [31].

The same pattern appears in visual search, provenance, and design-space exploration. Dynamic Queries showed how visual search constraints can produce immediate feedback [4]. VisTrails is a provenance example, allowing users to inspect histories of computation and revisit earlier states [25]. AverageExplorer is a more recent visual-collection example: it summarizes large image spaces into representative averages and modes so users can compare meaningful variation rather than inspect every state individually [168]. In this section, these systems are relevant as mapping and search strategies: the user's task is often not to edit a known parameter directly. It is to use visible evidence to discover which part of a larger space, program, graph, or history matters.

The central challenge is that visible evidence and hidden structure often have a many-to-many relationship. One stroke can correspond to several scene curves or gestures. One facial deformation

can be caused by several rig parameters. One rendered region can be affected by several graph states or construction operations. For this reason, scene-centered systems need more than inference alone. They also need ways for users to understand and correct the inferred mapping through previews, ranked alternatives, local refinement, confirmation, or explicit override. This combination of inference and correctability is what lets visible evidence become a practical entry point into hidden graphical control.

2.6 Positioning This Thesis

The thesis chapters that follow provide detailed related work for their individual domains while this chapter positions them within a broader design space for scene-centered interaction with hidden graphical structures. The shared question is how an interface can let users express intent in the visible scene while the system manages the underlying control structure.

Work	**Visible evidence**	**Hidden structure**	**Mapping and ambiguity**	**System response**
Visual querying	Highlighted output region	Dependency graph and construction history	Infer causally relevant states and reduce the history sequence	In-situ history slider
Squidgets	Stroke over perceived scene curve	Scene attributes associated with curves	Match stroke to the intended curve and attribute update	Sketch-based handle
Face-rig UI layouts	Deformation regions and trajectories	Rig parameters and blendshape controls	Map known parameter effects to placed, grouped in-situ controls	Generated rig-control layout

Table 2.1: The three main thesis contributions as instances of scene-centered interaction with hidden graphical structures.

The contributions lie in methods that make hidden graphical structure available through visible scene context. The visual-query work turns a highlighted output region into a navigable summary of construction history. The Squidgets work turns perceived scene curves into sketch-based manipulation handles [79]. The face-rig UI work maps known rig-parameter effects to in-situ control placement and grouping [80]. Together, these works show how in-situ interaction can support not only scene editing, but also graphical structure exploration and control layout. The same framing also motivates the later discussion of adaptive and AI-assisted workflows, where systems may infer controls from prompts, scene context, and interaction history, but still need legible, correctable, and situated handles.

Chapter 3

In-situ Visual Querying and Navigation of Dependency Graph Structures

This chapter begins the technical progression of the thesis with the broadest version of the editing problem: an existing procedural scene whose dependency graph contains many possible objects, operations, intermediate states, and editable parameters. When a user wants to edit a visible result, the first difficulty is not yet how to manipulate an attribute, but how to find which hidden state or attribute is relevant to the region they care about.

Visual querying uses rendered output as the entry point into this hidden structure. By selecting a local region of interest, the user can use visual evidence to navigate the dependency graph and narrow a large construction history to the objects, operations, intermediate states, and editable parameters most relevant to the desired edit. This chapter therefore establishes the first role of visual evidence in the thesis: finding what matters in an already-created scene.

3.1 Introduction

Dependency graphs are a central authoring paradigm in modern graphics software. In these systems, outputs are produced by evaluating a directed acyclic graph of operations that progressively derive a final visual output. The underlying dependency graph is typically exposed through a graph interface in which operations are represented as nodes and data dependencies are connected through directed edges. This graph-based model supports non-destructive editing, parameterization, and reuse, while graph-interface editing offers a visual programming interface that avoids some of the overhead of text-based programming [43, 56, 112]. The graph interface has become popular in industry tools such as Houdini, Maya, Blender, and Grasshopper across domains ranging from film VFX to game asset creation to architectural design.

Nonetheless, graph interfaces become harder to work with once dependency graphs grow. Production dependency graphs often contain hundreds of operators, nested subgraphs, and long chains of computation whose effect on the visual output may only be apparent later in the graph. This becomes a practical problem when artists want to perform a visual edit by finding the relevant operations or parameters in the graph to edit. Specifically, artists need to understand which operations in the dependency graph are responsible for the visible changes they observe in the final output. Yet, without prior knowledge of how the graph is organized, or which terms were used to name nodes and parameters, navigating to the right part of the dependency structure to explore the construction or edit attributes for the final outcome becomes slow.

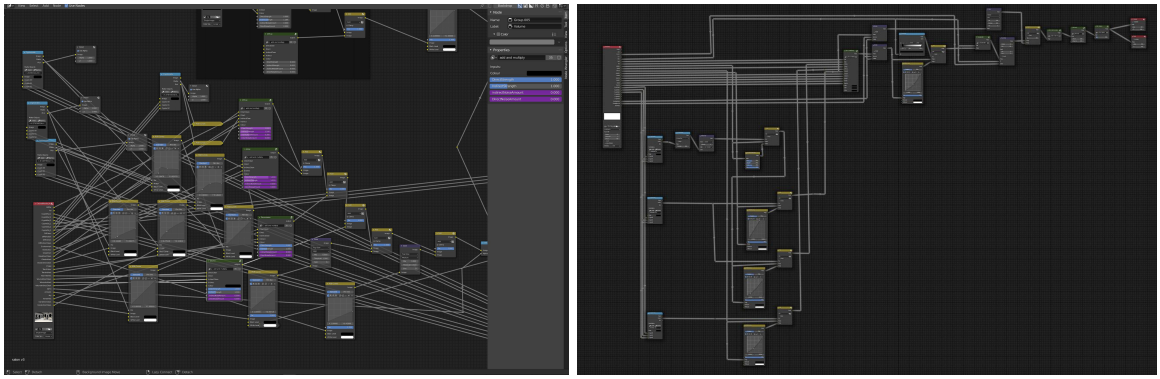


Figure 3.1: Left: An example of a graph posted on a Blender community forum providing user feedback on the unreadability of large graphs. Right: An organized graph by a forum responder who suggested that the artist learn Blender's network organization features. The original artist complained it was laborious.

A simple response to graph complexity is better graph organization. Modularization can break large graphs into subnetworks with narrower responsibilities, while naming conventions, layout discipline, and documentation can improve readability and navigation, especially for experienced authors working with well-maintained graphs (Figure 3.1). However, the effectiveness of these practices depends heavily on user expertise and sustained upkeep. They are manual to create, costly to refactor, and can become brittle as graphs evolve. More importantly, even a well-structured graph still requires the artist to translate a visible effect in the output into a structural search through the graph.

Prior work on graph visualization and visual programming helps explain this gap. Graph interfaces

are effective for exposing connectivity and supporting graph-level inspection, but their readability and navigability degrade as graph size and density increase [61]. Research on graph-visualization tasks and large-graph interfaces has therefore emphasized navigation, topology inspection, and focus+context techniques for managing structural complexity [32, 90].

On the surface, the same principles of graph-visualization research can be applied to graph interfaces for dependency graphs, and are widely adopted in software. However, a crucial difference between dependency graphs and other graph visualizations is that understanding the graph structure is usually not the artist's end goal. Creating and editing graphical outputs is the goal, and the graph interface is the typical way of controlling the underlying dependency graph. Because the creative process is focused on visible changes to the output, artists are often trying to recover which parts of the dependency graph are responsible for a particular visual effect: a silhouette change, a Boolean cut, a missing component, or a material transition. This disagreement is therefore not only one of scale, but of workflow: the user's question begins from visible consequence in the rendered result, while the interface requires a structural search through the dependency graph. In many cases that visible symptom is first noticed as a bounded phenomenon in screen space even when the responsible cause in the dependency graph is broader, more distributed, or global in its effect.

I use this chapter to ask: can dependency-graph structure be explored without beginning from the graph visualization itself? In-situ interaction can offer a useful alternative framing for this problem. Across graphics and HCI, in-situ systems place interaction and feedback in the same spatial context as the artifact being interpreted or edited, reducing mode switching and preserving perceptual continuity [67, 91, 119]. For dependency-graph exploration, that principle suggests that query specification should begin in the viewport where the user first notices a visual effect, rather than in a separate graph editor where that effect must be translated into structural querying; and navigation should be supported in the viewport on the output itself, rather than through a separate graph interface.

This chapter presents an output-centric visual-query workflow for in-situ dependency-graph exploration. Rather than beginning with the graph interface itself, the workflow begins with a highlighted region of interest in the rendered result and uses that visual query to retrieve and reduce relevant intermediate node outputs into a smaller ordered representative summary that can be traversed in the viewport. The method is therefore scoped to visual questions that can be expressed as a bounded highlighted region in the current view and for which masked differences within that region remain informative across intermediate states. In this way, this work introduces an alternative interaction model for dependency-graph exploration: users begin from visible consequence, and only later return to the graph interface once the relevant part of the history has already been reduced.

This workflow offers several benefits: users do not need to begin from detailed graph knowledge before beginning a search, the query is expressed through visible consequence rather than keywords, and the interaction remains in-situ without needing to inspect external panels. The result is a way to retrieve relevant graph states from the same visual evidence that motivated the search in the first place.

The main contributions are:

1. an output-centric visual-query formulation for dependency-graph exploration that begins from highlighted visible consequence rather than graph-interface search;

2. a query-driven reduction strategy that summarizes relevant intermediate node outputs into an ordered representative sequence for visual exploration;

3. an in-situ traversal interface that lets users explore that reduced sequence directly in the scene rather than through the graph interface.

This is not a complete graph-less workflow. My goal in this chapter is to explore an alternative method for dependency-graph exploration and understanding; this work does not address in-situ editing or the broader problem of provenance interfaces. More broadly, this work questions current dependency-graph workflows by emphasizing visible consequence as the first entry point into exploration rather than relying only on manual traversal of authored structure.

I begin with related work (Section 3.2) and then discuss the design of the interaction (Section 3.3). Section 3.4 formalizes the highlighter, clustering, and in-situ slider construction pipeline. The method is then illustrated across several dependency-graph scenarios, followed by a discussion of its scope, limitations, and future directions.

3.2 Background

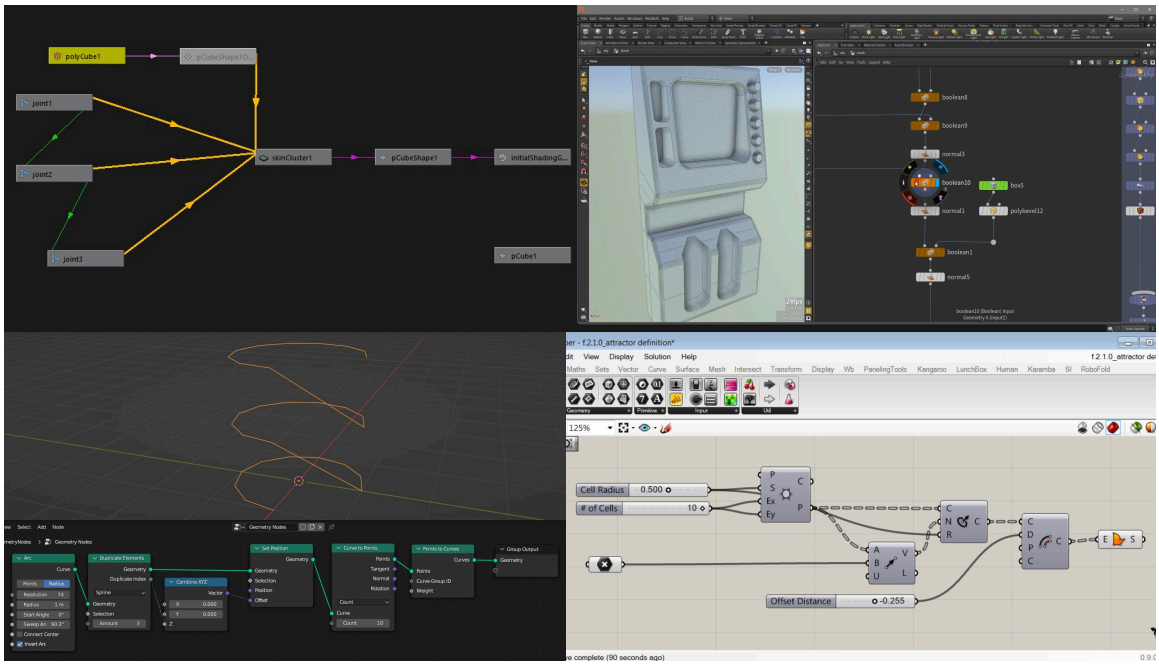


Figure 3.2: Graph interfaces in commercial software. Top-Left to Bottom Right: Maya, Houdini, Blender, Grasshopper.

Dependency graphs and graph-interface workflows. Dependency-graph workflows emerge from a broader history of dataflow and visual programming systems, in which computation is represented as a directed graph of dependent operations [35, 43, 56, 112, 147]. In graphics authoring, this model underlies procedural shape construction, shading, and scene assembly [33, 123, 141]. Such workflows are now central to tools including Maya's Construction History, Houdini's operator networks, Blender's dependency graph (Depsgraph), and Grasshopper's visual dataflow graph (Figure 3.2). In these systems, the dependency graph is the computational structure that governs evaluation, while the graph is the visual interface through which that structure is authored and inspected. This

representation is powerful for procedural authoring, but it does not by itself solve the problem of locating visually relevant intermediate states once graphs grow large.

Graph and provenance interfaces. Alongside dependency-graph representations, a parallel literature explored how users inspect, query, and revisit graph-structured information. Graph-visualization work showed that graph representations expose relationships clearly, but readability and navigability degrade as graphs grow in size and density [61]. Research on large-graph interfaces therefore emphasized overview+detail, zooming, and focus+context techniques for managing structural complexity [32], while dynamic-query interfaces showed how query and filter controls could support more fluid exploration [4]. Ware and Bobrow also showed that visual highlighting can support targeted search in graph representations by making specific relational structure easier to perceive [159]. In workflow systems, *VisTrails* treated workflow histories as provenance objects that can be versioned, revisited, and compared over time [25].

These systems are important because they support graph inspection, graph querying, and workflow provenance. However, they still assume that the user's exploration proceeds through the graph representation itself. They do not address output-centric query and traversal of intermediate states inside a dependency graph, where the user's question begins in the rendered result rather than in the graph interface.

Design-space reduction and clustering. Another relevant area is how large design spaces are presented to users for interaction and exploration. Marks et al. made this question explicit with *Design Galleries*, which treated a parameter space as a collection of visually browsable alternatives chosen to reveal meaningful variation [99]. This work highlighted the importance of reducing large design spaces into manageable sets of representative and comparable alternatives. Later work extended this idea from parameter studies to collections of visual artifacts. Kim et al. framed shape exploration as identifying and relating similar regions across a collection, using soft correspondences to reveal shared structure, variation, and ambiguity between 3D models while supporting interactive comparison [82]. Zhu et al.'s *AverageExplorer* frames the exploration of large image collections as navigating distributions rather than individual instances, using averages of clustered images to expose common structure, variation, and modes within the data while supporting interactive refinement of these groupings [168]. Other systems used retrieval, suggestion, or learned variability models to make large shape spaces navigable without asking users to inspect every instance. Chaudhuri and Koltun generated data-driven component suggestions for 3D modeling from a partially built shape, supporting exploratory design through retrieved examples that are relevant to the current artifact [29]. Ovsjanikov et al. learned continuous variability in shape collections and exposed that variability through a compact deformation-based navigation interface [118]. Pandey et al.'s *Juxtaform* is a closer presentation precedent for the slider construction used in this chapter, since it uses sparse juxtaposed abstractions from clustered shape collections to support in-context comparison [120].

These works are important because they show how large visual state spaces can be reduced into representative, comparable alternatives without destroying the distinctions a user cares about. This setting differs in that the states being summarized are not external artifacts or unrelated samples, but intermediate outputs generated within one dependency graph. The missing capability is therefore not only browsing many alternatives, but reducing a query-specific subset of intermediate graph

states into a compact sequence that can later be traversed in situ.

Visual query by region, example, and pattern. Content-based retrieval systems provide a more direct precedent for the output-centric query formulation. QBIC showed how users could retrieve image and video content through visual properties such as color, texture, shape, and example imagery rather than metadata alone [46]. VisualSEEk and Blobworld made the query more spatial and local: users could specify arrangements of color regions or select segmented regions to retrieve images with similar local visual structure [26, 139]. In visualization, TimeSearcher and ShapeSearch similarly let users specify desired visual patterns directly over displayed data or trendline shapes, turning a drawn region or sketched pattern into a query over many candidate outputs [64, 136]. These systems establish visual querying as a way to retrieve or filter results from visible examples, regions, or patterns. This chapter uses the same output-centric entry point, but the retrieved results are intermediate states inside one dependency graph rather than independent database items.

In-situ and direct interaction. Shneiderman's formulation of direct manipulation argued that interfaces become more understandable when users act on visible objects of interest rather than through detached symbolic commands [135]. Bier et al. extended that logic with *Toolglass* and Magic Lenses, showing how interface controls could be layered directly over the workspace instead of being isolated in separate panels [19]. In graphics and design applications, this idea evolved into a broader in-situ paradigm in which interaction, guidance, and feedback are embedded in the same visual context as the piece being worked on.

From this idea, many use cases have been explored in graphics and design applications. Paczkowski et al. used in-context sketching to support architectural design directly over the site and composition being considered [119]. Lee et al.'s *ShadowDraw* embedded guidance into the drawing space itself, treating the workspace as the place where assistance should appear rather than an external control channel [91]. Igarashi et al.'s *Teddy* showed how sketch input in the drawing space can be interpreted as 3D form, while as-rigid-as-possible shape manipulation showed how direct edits to visible geometry can be propagated through an underlying deformation model [67, 69]. The face-rig UI chapter is a closer example for the control-layout aspect of this chapter, since it places controls near the facial regions affected by high-dimensional rig parameters [80].

Other graphics work has explored these principles specifically for DAG-based representations. Michel et al. used DAG amendment to support inverse control of parametric shapes while keeping the underlying procedural structure intact [103].

These works establish why in-situ interaction is valuable once the user already knows what visible object or control should be manipulated. This chapter's problem begins earlier: the user first needs to query and recover which intermediate states inside a dependency graph are relevant to a visible effect in the final result. This makes output-centric, in-situ querying an appropriate direction: the query can begin in the desired region, and the resulting reduced history can remain anchored in the same spatial context rather than forcing repeated transitions to a detached node editor.

3.3 Design and Overview

3.3.1 Graph-interface Workflow

Dependency-graph workflows in graphics software are typically split across a scene view, a graph editor, and a parameter editor. Artists author visual artifacts by chaining node operations together within the graph editor, adjusting individual node parameters in the parameter editor, and visualizing the current result in the scene view. Typical workflows cycle through these main panels: artists imagine a desirable visual edit, find existing or create new nodes to support this edit, and tweak parameters to achieve the desired result in an iterative process.

As dependency graphs grow, finding the relevant node becomes a bottleneck in that workflow. Large networks contain nested subnetworks and long chains of computation whose visible effects only become clear much later in evaluation. Though the graph remains useful for explicit authoring, it can become a slow search space for answering visual questions because it does not directly reveal which nodes are involved in the desired edit quickly. Some graph interfaces allow rendering small thumbnails on nodes themselves or allow display nodes and visibility toggles to view the output of specific intermediate nodes, but these features are still set within graph interfaces themselves.

This workflow forces artists to carry two mental models at once: a visual model of the graphical output in the viewport and a structural model of the dependency graph in the editor. The previous section reviewed different methods of facilitating graph navigation and inspection, but they all assume that the artist is trying to understand the graph itself as a primary task. However, an artist who notices a seam, a protrusion, or a missing component is not trying to understand the graph structure for its own sake; they are trying to recover the cause of a visible symptom in the result. As navigation becomes increasingly difficult as graphs scale, maintaining that structural model during early search becomes a burden.

The workflow therefore defers graph-interface structural details during early exploration and provides a more output-centric workflow. This applies specifically to exploration tasks; graph interfaces are currently necessary for authoring new chains of operations. The proposed tool is not meant to replace the graph interface but to complement it by providing a more visual entry point for exploration.

3.3.2 Design Principles

A visual question from the artist about the output of a construction history implies an informal in-situ interaction between the artist and the rendered artifact in the viewport. This provides an opportunity for a formal entry point into exploration and navigation. An in-situ interaction requires a visual and localized anchoring of the interaction and visual feedback that is directly related to the artifact in question, which complements the fact that the artist is already focused on the artifact. Additionally, in-situ interaction benefits from direct manipulation as an interaction style. As the artist's attention shifts into the scene, the exploration of the dependency graph should also stay within the scene as much as possible, rather than forcing the artist to switch back and forth between the scene and the graph interface. Direct manipulation, particularly scrubbing, is a universally familiar interaction style that drives inspection and navigation, especially for temporal sequences (videos). Because construction histories are ordered sequences of operations, scrubbing through a

sequence of intermediate states is an appropriate candidate method of exploring the history and understanding how the artifact is formed.

The following design principles guide an in-situ method for exploring dependency graphs while deferring graph structure during early inspection.

D1: Query through visual consequence. Artists usually begin with a visible issue or a desired visual result before knowing which node operation produced it. From this querying task, not every node in the dependency graph is relevant to that question and needs to be searched. The first principle is to query through visible consequence from the artist's view rather than direct lookup in the graph interface. Instead of asking artists to search for the right operator name or network region, the tool lets them specify a region of interest directly in the viewport and retrieves intermediate node outputs whose rendered results exhibit relevant variation within that selected region. This does not assume that the responsible node is itself spatially local in graph terms; it assumes only that the visual question can be expressed as a bounded highlighted region in the current view.

D2: Image-space analysis. The method treats image space and the artist's current view as the primary domain for estimating visual relevance across different node outputs in the dependency graph rather than analyzing the graphical representations themselves. The underlying graphical representation may be geometry, point clouds, signed distance fields, simulations, or learned outputs, which poses compatibility challenges for direct graph analysis. But all representations are visually rendered in screen space, and the screen-space domain remains the most direct representation of what appears, disappears, overlaps, or changes from the artist's point of view. This also motivates persisting the visual query in a view-aware form, so the queried region can remain meaningful under modest camera change while the search still begins from the artist's current view. The method is strongest when masked differences inside that highlighted region remain discriminative across intermediate states, even if the underlying graph dependency is more distributed.

D3: Reduced history traversal in place. Even after a visual query narrows the search, the remaining nodes are still too large to inspect one by one. The interface should therefore summarize these states to a smaller set of distinct representatives and let the artist traverse them without leaving the viewport. The workflow therefore reduces the relevant intermediate node outputs into a smaller ordered representative sequence and realizes that sequence as an in-situ query slider in the viewport. In the current implementation, that slider is not formed by directly replaying every representative state as an attached control point. Instead, the ordered sequence is converted into a smaller set of image-space segment cues, and only the winning cleaned path through those cues is realized as the final in-situ slider. This keeps navigation embedded in the scene, reduces context switching, and supports provenance inspection within the selected region without requiring continuous returns to the node editor.

3.3.3 Overview

The tool follows a three-step workflow: *query*, *reduction*, *and in-situ traversal*. Consider an artist who wants to understand how the wing of an airplane is formed in an unfamiliar dependency graph

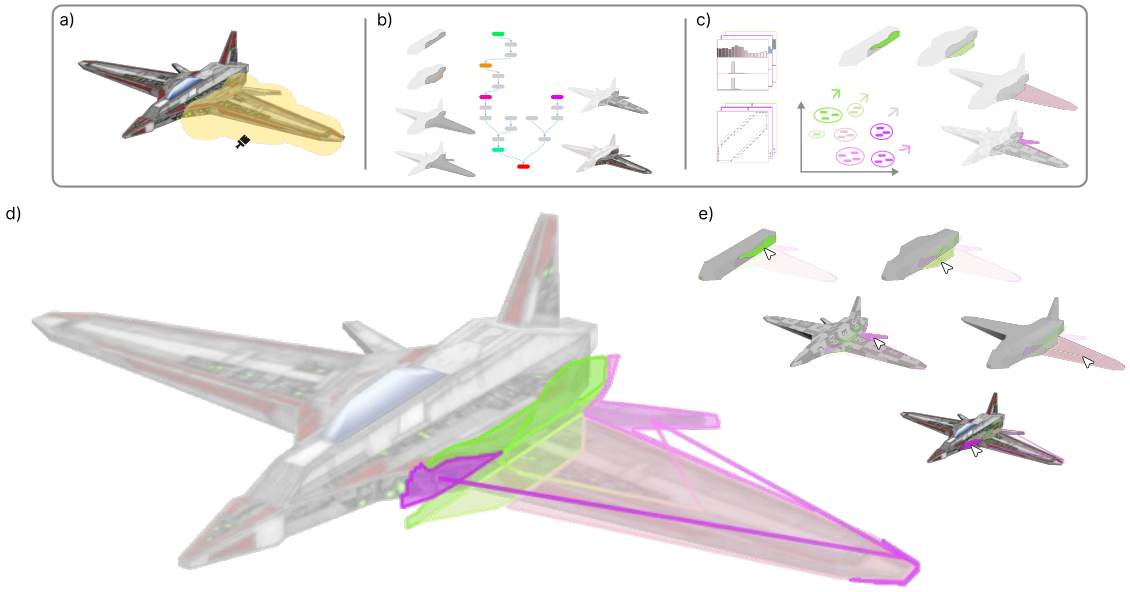


Figure 3.3: Given an airplane model, a visual query asks how the wing changes in the dependency graph over time by highlighting the wing a). The system captures masked outputs of all nodes in the dependency graph b), clusters them by visual similarity, and generates non-overlapping colored segments c). These colored segments are combined to create an in-situ visual summary of the airplane wing's dependency graph d). The summary becomes a slider interface where clicking on ticks displays a summary of the cluster output and dragging between nodes in increasing gradient colors summarizes the construction history e).

(Figure 3.3). Instead of switching to the graph editor and tracing nodes manually, the artist highlights the wing as a region of interest directly in the viewport and issues a visual query.

In the first step, the tool captures the queried portions of intermediate node outputs from the current view. It then reduces those queried results into a smaller ordered representative summary of the major visual changes within the selected region.

Finally, the tool presents this reduced sequence through an in-situ query slider embedded in the scene. The slider is built from localized segment candidates extracted from the ordered representative sequence in image space, cleaned into one non-overlapping path, and then attached back to the current scene for traversal. The artist can traverse the sequence directly in the viewport to understand a compact visual summary of how the selected region changes through the dependency graph. This slider allows the user to keep attention on the artifact itself rather than on a separate graph-interface view. In this way, the workflow begins from visible consequence and only returns to the graph interface after the search space has already been reduced for editing purposes.

3.4 Method

Given an active dependency graph, a current camera, and an artist-specified visual query, the goal is to recover a small ordered representative sequence of node outputs that are visually relevant to the

query, visually distinct from one another, and traversable in situ. The method takes as input

$$I = (G, v_0, \mathbf{C}, \mathbf{M}_q), \tag{3.1}$$

where G is the active dependency graph, v_0 is the currently displayed baseline node, $\mathbf{C}$ is the active camera, and $\mathbf{M}_q$ is the screen-space visual-query mask drawn by the artist. The output is

$$O = (\mathcal{C}, \boldsymbol{\pi}, \mathcal{P}^\star), \tag{3.2}$$

where $\mathcal{C} = \{\mathcal{C}_1, \ldots, \mathcal{C}_K\}$ is clusters of nodes in G with visually similar outputs within the masked region, $\boldsymbol{\pi} = (v_1^\star, \ldots, v_K^\star)$ is the ordered sequence of representative nodes for slider tick order, and $\mathcal{P}^\star$ is the selected cleaned path used to build the final slider. The final query slider is a derived in-situ object realized from this representative order and selected path.

The dependency graph is modeled as a directed graph $G = (\mathcal{V}, \mathcal{E})$ where each node $v \in \mathcal{V}$ is an operator with inputs and outputs and each directed edge $(u, v) \in \mathcal{E}$ feeds output from u into v. Let $\mathcal{X}(v)$ denote the scene state obtained after evaluating the dependency graph up to node v. Under the active camera $\mathbf{C}$, the rendered output of node v is an image

$$\mathbf{I}_v = \mathrm{Render}(\mathcal{X}(v); \mathbf{C}), \tag{3.3}$$

where Render denotes the rendering operator.

3.4.1 Visual Querying

Highlighter brush. The artist highlights a region of interest directly in the viewport using a highlighter brush to ask how that region changes across the dependency graph (Figure 3.4). The brush is drawn over the currently displayed output $\mathbf{I}_{v_0}$, so the query mask is anchored to the active view of node v_0. Let $\mathbf{M}_q : \Omega_{\mathrm{img}} \to [0, 1]$ denote a binary query mask over the image domain Ω_{img} where 1 includes the pixel in the image and 0 excludes the pixel from the query. Highlighting is a common method of annotation that keeps the underlying view visible, and this method is extremely cheap and expressive for artists.

Although the query is specified in screen space, the tool preserves a lightweight 3D-backed support for the painted region so the query can survive modest camera changes. In practice, the stroke is projected onto the viewed artifact and stored as support samples that can be reprojected back into the current view to recover the active query mask.

Once a query is requested, the tool traverses the dependency graph G and renders each intermediate node output $\mathbf{I}_v$. From these outputs, only the queried region is retained. The masked node output is defined

$$\mathbf{I}_v^{(q)} = \mathbf{M}_q \odot \mathbf{I}_v, \tag{3.4}$$

where $\odot$ denotes element-wise masking.

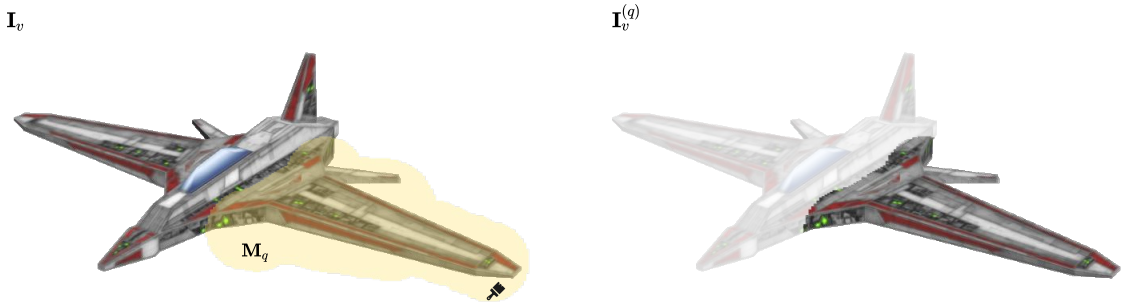


Figure 3.4: The user highlights to create a mask $\mathbf{M}_q$ over image $\mathbf{I}_v$ to create a masked image $\mathbf{I}_v^{(q)}$ that retains only the queried region. The masked out region is displayed as reference.

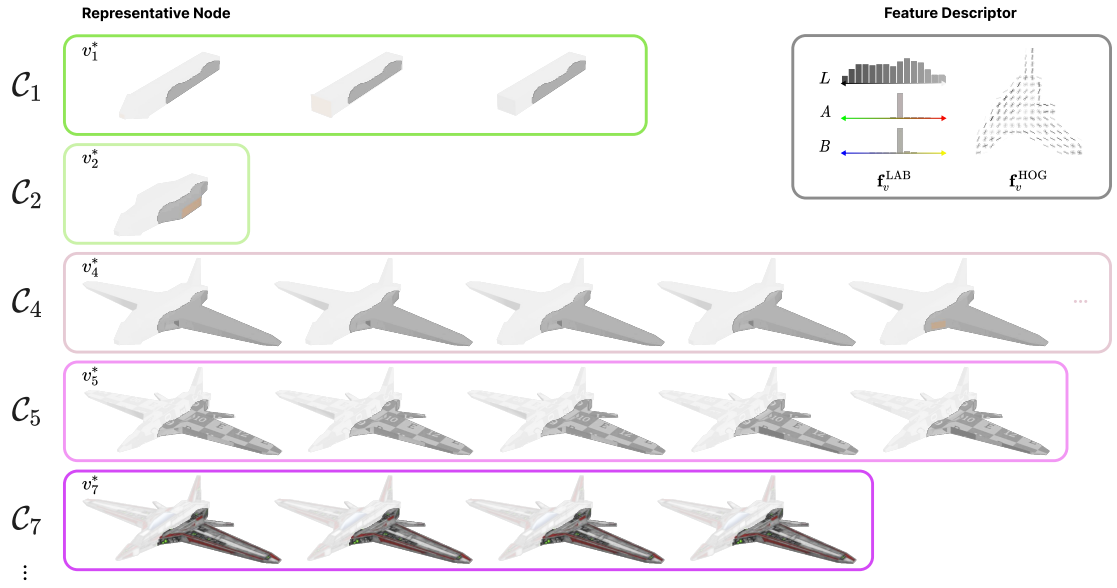


Figure 3.5: Clusters of nodes from the airplane example show visually similar outputs within the query region. The feature descriptor used for clustering captures both color and structural characteristics.

3.4.2 Clustering

With the set of masked node outputs $\mathbf{I}_v^{(q)}$, the goal is to cluster queried nodes by visual similarity within the highlighted image region and reduce them to a smaller set of distinct outputs. Figure 3.5 shows the resulting grouping on the airplane example: visually similar masked outputs are grouped into clusters, and each cluster contributes a representative node for the reduced sequence.

Combined LAB+HOG feature clustering. Let $\mathcal{V}_q \subseteq \mathcal{V}$ denote the subset of dependency-graph nodes whose queried outputs yield valid masked measurements. For each queried node output, the method computes an image descriptor that captures color and structure from the masked images. Specifically, the implementation uses a combined LAB color feature plus Histogram of Oriented Gradients (HOG) feature. The LAB component summarizes color and material variation in the queried region using color histograms in LAB space [70], while the HOG component summarizes coarse structural variation using local image gradient orientations [34]. These two feature families are written as

$$\mathbf{f}_v^{\mathrm{LAB}} = \phi_{\mathrm{LAB}}(\mathbf{I}_v^{(q)}) \in \mathbb{R}^{d_{\mathrm{LAB}}}, \tag{3.5}$$

$$\mathbf{f}_v^{\mathrm{HOG}} = \phi_{\mathrm{HOG}}(\mathbf{I}_v^{(q)}) \in \mathbb{R}^{d_{\mathrm{HOG}}}, \tag{3.6}$$

where $d_{\mathrm{LAB}} = 96$ for the concatenated 32-bin histograms over the L, A, and B channels, and $d_{\mathrm{HOG}} = 7200$ for the HOG descriptor extracted from the resized masked crop. The feature vectors are then concatenated into one clustering descriptor

$$\mathbf{f}_v = \left[\mathbf{f}_v^{\mathrm{LAB}}\,;\,\mathbf{f}_v^{\mathrm{HOG}}\right] \in \mathbb{R}^{d_{\mathrm{LAB}}+d_{\mathrm{HOG}}}. \tag{3.7}$$

Using these descriptors, the queried nodes are then clustered:

$$\mathcal{C} = \mathrm{Cluster}\big(\{\mathbf{f}_v\}_{v\in\mathcal{V}_q}\big)\,, \tag{3.8}$$

where $\mathcal{C} = \{\mathcal{C}_1, \ldots, \mathcal{C}_K\}$ is the resulting partition of queried nodes into visually similar groups in the combined LAB+HOG feature space. Other image representations could be used to segment, label, or cluster the masked regions before reduction. Classical methods such as graph cuts or superpixels could produce boundary-aware regions [2, 84], while neural methods such as STEGO and SAM could provide semantic segmentation labels or promptable masks [57, 83]. This implementation uses LAB+HOG as a lightweight alternative, prioritizing fast processing over semantic accuracy. For clustering, both k-means and agglomerative clustering were tested over these combined descriptors seeing similar results in cluster quality [98, 158]. The number of clusters is selected from a small sweep $K \in \{4, 5, 6, 7, 8\}$ whenever the queried graph is large enough, and the best silhouette-scored configuration is retained [125].

Representative node selection. For each cluster $\mathcal{C}_k$, the tool chooses the medoid of the cluster as the representative node $v_k^\star$ whose masked output is most visually central to the rest of that cluster.

This selection is written as

$$v_k^\star = \arg\min_{v \in \mathcal{C}_k} \sum_{u \in \mathcal{C}_k \setminus \{v\}} \delta\left(\mathbf{I}_v^{(q)}, \mathbf{I}_u^{(q)}\right), \tag{3.9}$$

where δ denotes pairwise visual difference between two masked outputs in the same cluster. In other words, the representative is the node whose queried image is closest, on average, to the other queried images in that cluster. These representative nodes become the characteristic states that the interface exposes.

3.4.3 In-Situ Slider Creation

To explore the representative nodes, the method constructs an in-situ slider from the ordered representative sequence $\boldsymbol{\pi}$. A slider requires ticks to apply set configurations and a path to connect those ticks into a traversable control. The implementation uses visual features of the representative outputs to form the slider ticks rather than plan circles or lines and uses $\boldsymbol{\pi}$ to define the chronology of those ticks. The tick shapes provide visual cues about the underlying node outputs, while their chronology provides a semantic ordering that reflects the underlying dependency structure.

The path is the ordered sequence of representative nodes according to the chronology induced by their dependency-graph positions. This yields the ordered representative sequence

$$\boldsymbol{\pi} = (v_1^\star, \ldots, v_K^\star), \tag{3.10}$$

where representatives further upstream appear earlier in the sequence and representatives closer to the final displayed result appear later.

Difference-guided segment candidates. Designing the shape of the slider ticks is a key challenge because the goal is to create an in-situ control that visually summarizes the underlying node outputs while remaining legible and interactive. Figure 3.6 shows the pipeline used to derive these tick candidates from adjacent representative outputs: the pipeline computes masked differences, thresholds high-signal regions, and segments them into candidate tick shapes.

Because the representative sequence $\boldsymbol{\pi}$ provides a chronological ordering of representative nodes, differences between neighboring representatives are used to identify local visual changes. For each adjacent pair in the ordered sequence, a difference image is defined

$$\mathbf{D}_{k,k+1}^\star = \Delta\left(\mathbf{I}_{v_{k+1}^\star}^{(q)}, \mathbf{I}_{v_k^\star}^{(q)}\right), \tag{3.11}$$

derived from the ordered pair $(v_k^\star, v_{k+1}^\star)$. These comparisons are evaluated in LAB image space over the queried region. The resulting difference images display high signal where neighboring representatives differ and low signal where they remain visually similar. These high signal regions can be used as slider tick shapes for either $v_k^\star$ or $v_{k+1}^\star$ because they represent visual change that occurs between those two states. The method assigns these difference image components to one of the two neighboring representatives so each representative owns a difference image.

Each difference image is normalized and segmented into connected components using thresholding followed by connected-component extraction. This produces regions r_k that can be used as slider

tick shapes.

Let

$$\mathcal{R}_k = \{r_k^1, \ldots, r_k^N\} \tag{3.12}$$

denote the ranked owned segment candidates assigned to representative $v_k^\star$. Representatives that receive no owned segment are treated as unassigned and are handled with the same fallback candidate-generation path as other missing-support cases. The implementation keeps the top $N = 2$ candidates per representative.

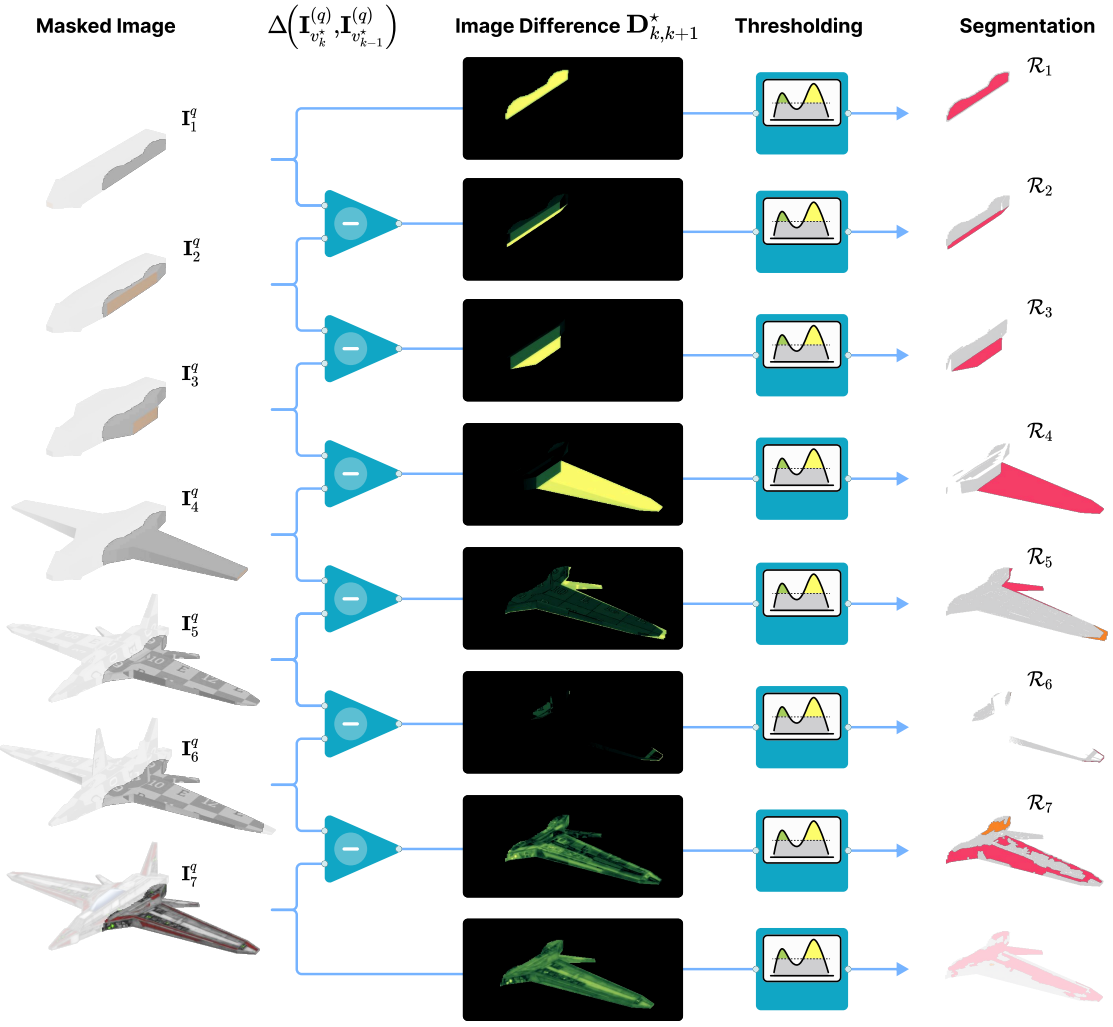


Figure 3.6: Adjacent representative outputs $(\mathbf{I}^{(q)}_{v^\star_{k+1}}, \mathbf{I}^{(q)}_{v^\star_k})$ are used to compute difference images $\mathbf{D}^\star_{k,k+1}$ in LAB space. High-signal regions that indicate local visual changes between neighboring representatives are retained through a thresholding pass. These regions are segmented into connected components $\mathcal{R}_k$ (with segments colored in red and orange), and each component is assigned to one of the neighboring representative that best explains it. The resulting owned segments become candidate tick shapes for the in-situ slider. In this figure, the last segmentation is not assigned.

Candidate-path search across ordered clusters and winning-path selection. The local image-space candidates do not yet define the final slider because each cluster can contribute several plausible segment regions. The method therefore searches over candidate paths that choose one raw marker candidate per ordered cluster:

$$\mathcal{P} = \mathcal{R}_1 \times \mathcal{R}_2 \times \cdots \times \mathcal{R}_K. \tag{3.13}$$

The result is a set of ordered candidate paths, each of which represents one possible slider interpretation through the representative sequence.

For a candidate path $\mathcal{P}_i$, its segments r_k^j may still visually overlap one another, which would produce ambiguous slider ticks. Each candidate path is cleaned by greedily resolving overlapping segment regions until the selected supports no longer overlap. The cleanup prioritizes overlaps between nearby representatives in the ordered sequence and assigns contested pixels to the stronger segment, preferring owned difference segments over fallback segments. This produces a non-overlapping set of tick supports while preserving as much usable area as possible for each representative.

Let $\widetilde{r}_k$ denote the cleaned segment assigned to ordered cluster k for a candidate path. The cleaned path is then

$$\widetilde{\mathcal{P}} = (\widetilde{r}_1, \ldots, \widetilde{r}_k). \tag{3.14}$$

Once all candidate paths have been cleaned, the method optimizes for a single winning cleaned path. This choice is driven by the usability of the resulting segment ticks. For each cleaned path step, the implementation computes a tick-size score from the surviving cleaned region. The winning path is the path with the largest minimum tick area across all steps, so the weakest tick on the path is as large as possible.

The winning path is then selected lexicographically:

$$\mathcal{P}^\star = \arg\max_{\widetilde{\mathcal{P}}} \left(\min_k area(\widetilde{r}_k) \right), \tag{3.15}$$

The slider is created by connecting pairs of cleaned segments $(\widetilde{r}_{k-1}, \widetilde{r}_k)$ with straight line segments and a slider thumb to scrub along the path. Activating the slider tick displays the corresponding representative node output and highlights $\widetilde{r}_k$ as well as paths connected to $\widetilde{r}_{k-1}$ and $\widetilde{r}_{k+1}$. Segments are colored in a gradient to indicate cluster progression with the most saturated segments marking the end ticks of the slider.

Implementation. The tool is implemented inside Houdini using Python and Houdini Viewer States, with the viewport used as the primary query surface and the dependency graph used as the computational structure being searched. The image-space stages of the pipeline use the Python libraries NumPy, SciPy, scikit-image for feature extraction and segmentation, scikit-learn for clustering, and OpenCV for other image processing steps. The implementation was run on an Apple M3 MacBook Pro.

For the 3D in-situ slider, the path $\mathcal{P}^\star$ is projected onto the Houdini geometry. Specifically, each cleaned region $\widetilde{r}_k$ is raycast back into the world and attached to the geometry of the representative node $v_k^\star$ (or the previous node $v_{k-1}^\star$ in case the visual difference between $(v_{k-1}^\star, v_k^\star)$ resulted in lost

area), so the slider remains attached to the artifact as the user moves through the scene.

3.5 Examples

The examples use several dependency graphs from the Houdini community to test the tool. These examples show how different procedural networks respond to visual queries from the viewport. Some paths are omitted in the figures to emphasize segment shapes.

3.5.1 Structural Creation

Structural creation networks progressively add or modify visible parts. These are the clearest cases for the tool because the highlighted region usually has a local construction history. The representative sequence can then be read as a compact visual buildup: early states introduce coarse supports, later states refine shape, and the slider gives the artist a spatial handle for scrubbing through those states. Figure 3.7 shows the airplane queried from multiple locations, and Figure 3.8 shows a decorated wall with layered ornament patterns.

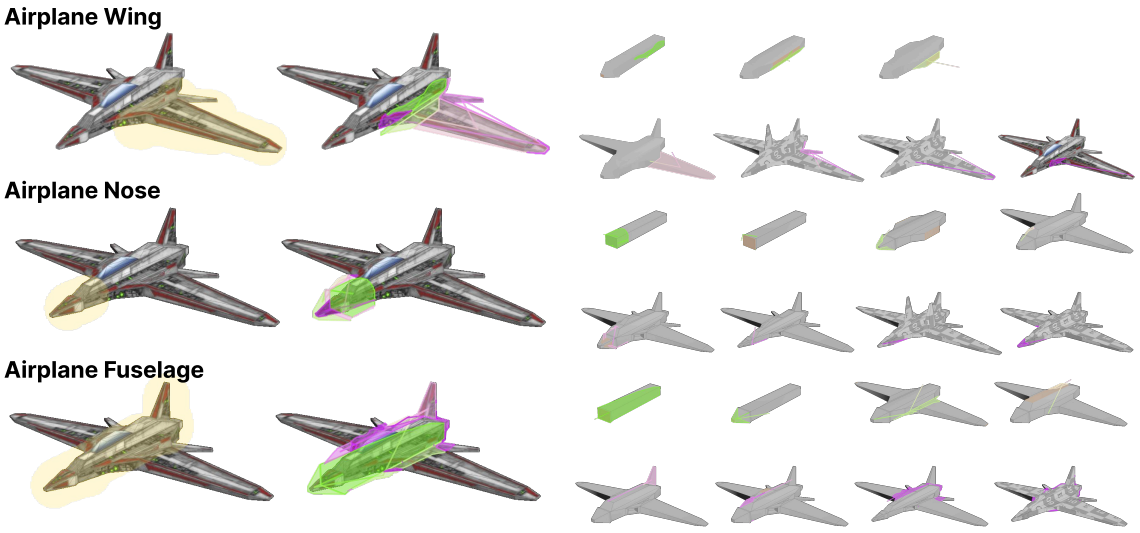


Figure 3.7: Different visual queries on the same airplane recover different local construction histories. Querying the wing emphasizes states that form the wing surface and supporting outlines, while querying the fuselage or nose shifts the representative sequence and segments toward the body and front. The same dependency graph can therefore yield several query-specific construction-history summaries depending on where the artist asks the visual question.

3.5.2 Composition

Artists often author assets by building components in local networks and then combining those components in a larger composition network. A highlighted region may therefore correspond to a local component, a final assembly, or both. Many operations in these networks are transformations that place parts together, so representative states often capture whole parts or intermediate placement guides such as curves and point locators. Figure 3.9 shows an aqueduct network that combines wall

Figure 3.8: The highlighted wall region contains both large wall surfaces and thin decorative curves, so the representatives include states for the panel, side column, top edge, and ornament strokes. In this network, the artist builds the components in one larger network rather than separating them into subnetworks, and the tool tends to select more finished components rather than only early initialization states.

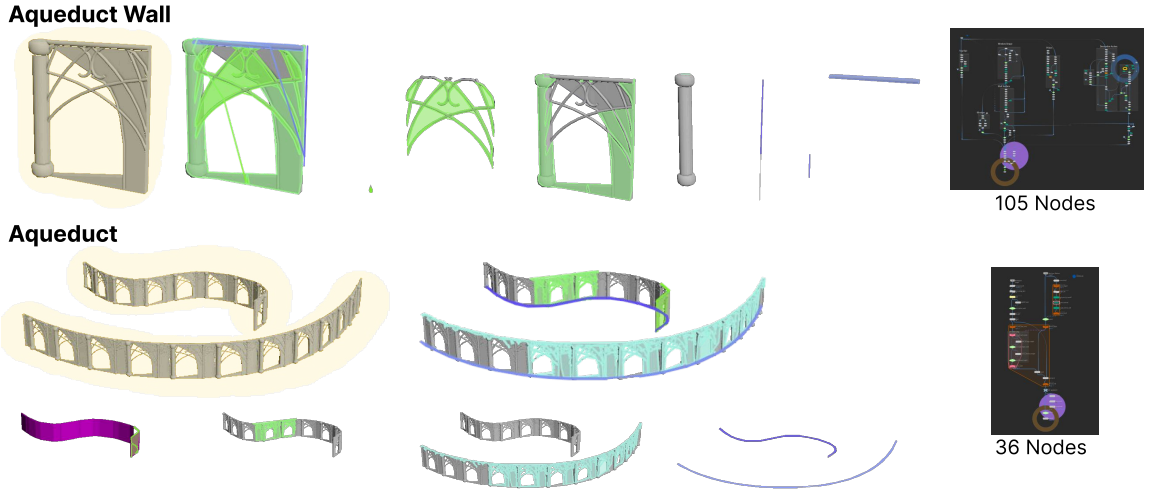


Figure 3.9: An aqueduct network places repeated wall modules with a curve-driven layout. Querying one wall module recovers the local column, arch, and brace construction, while querying the full aqueduct returns representatives that include both complete module geometry and curved placement paths. The example shows a composition workflow where the artist may care both about how a module is built and how it is arranged along a curve.

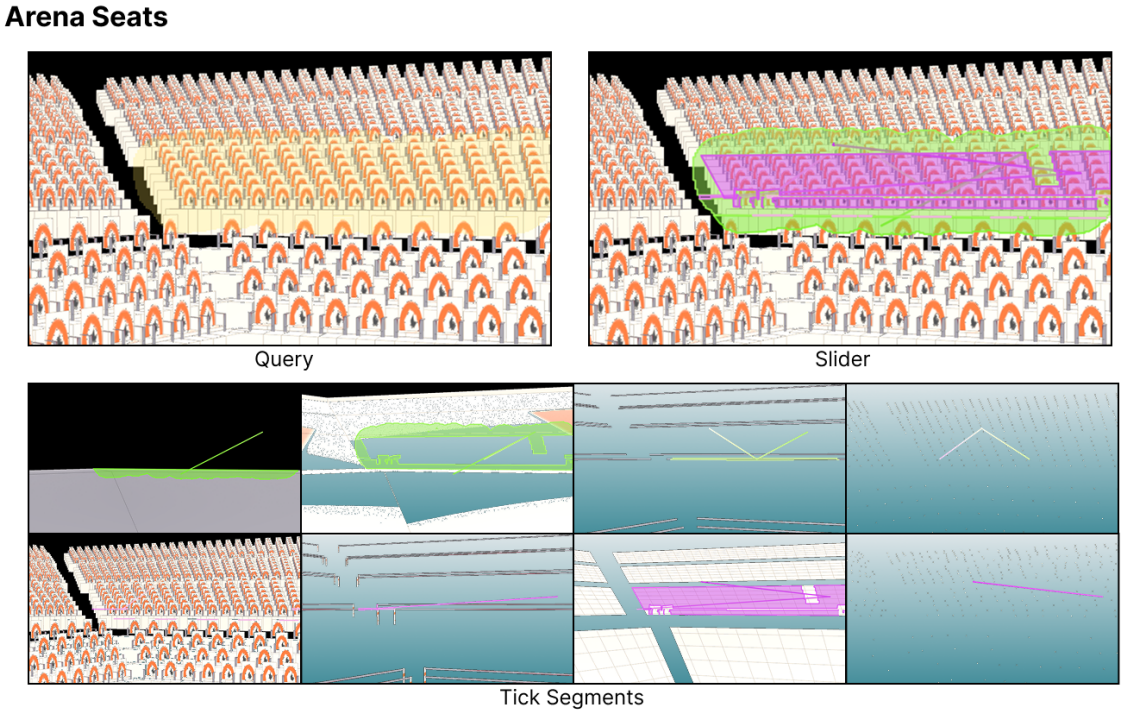


Figure 3.10: An arena-seat query shows a repeated-placement network rather than a seat-construction network. Although the artist highlights audience seats, the representatives include floor plates, layout guides, and distant or sparse geometry because the queried image region overlaps several placement-related outputs. This illustrates query ambiguity: the visible intent is “seats,” but the dependency-graph evidence also contains nodes that affect where seats are arranged.

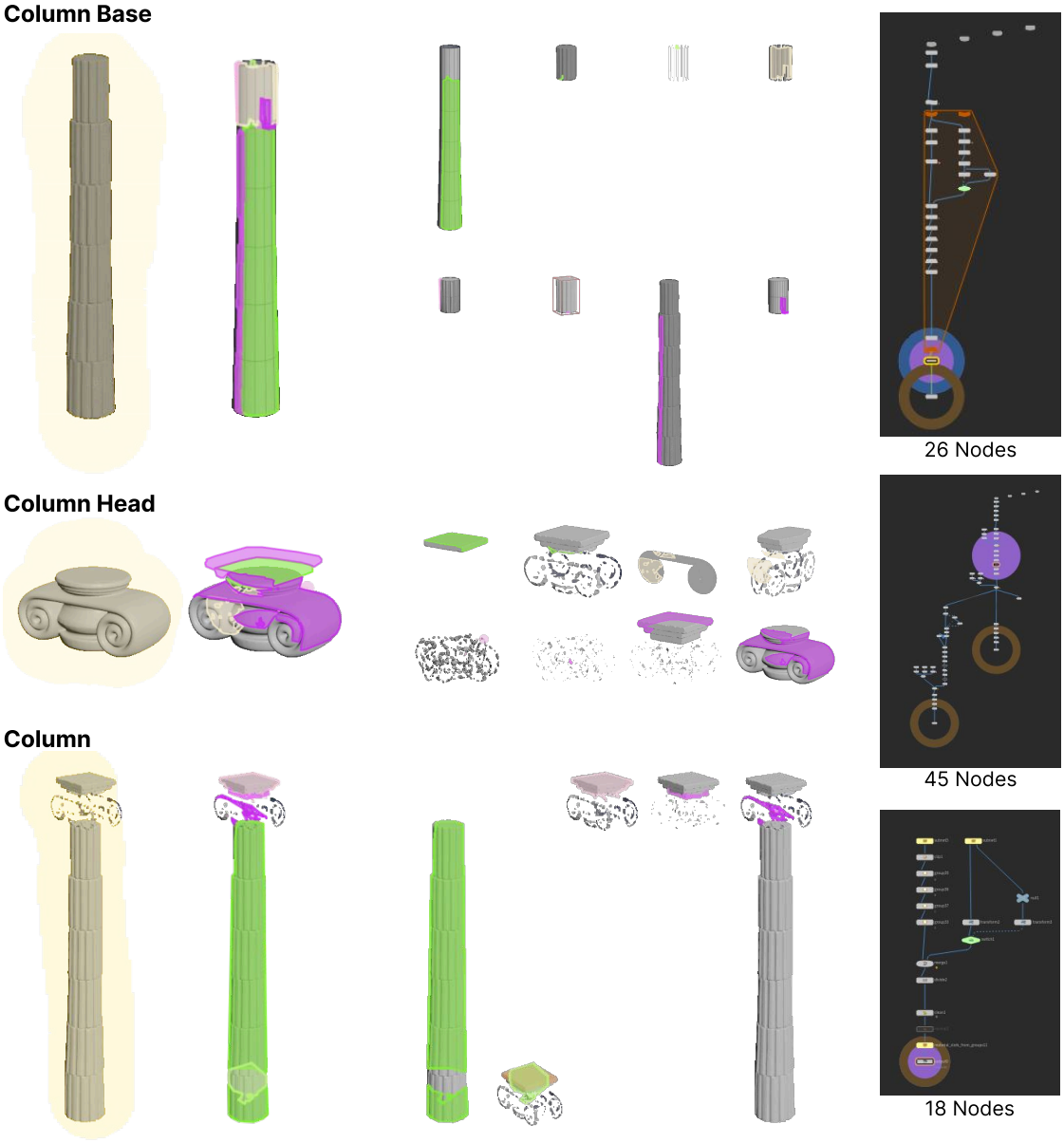


Figure 3.11: A column asset contains separate networks for the shaft, the capital, and the assembled column. Querying the base emphasizes a mostly vertical construction sequence, querying the head reveals ornamental scroll and cap geometry, and querying the whole column mixes local component creation with the final composition network. This shows how visual queries can cross from part-building networks into networks that assemble components together.

modules with a curve-driven layout, Figure 3.10 shows an arena seating area, and Figure 3.11 shows a column with separate body and head networks.

3.5.3 Textural and Mixed Change

Some dependency graphs create visible change through materials, masks, erosion, noise, or color fields. These examples test whether a visual query can summarize histories where the salient change is textural rather than part-based. Figure 3.12 shows a mountain network that creates a top-down texture map, and Figure 3.13 shows a mountain landscape that combines structural and textural change.

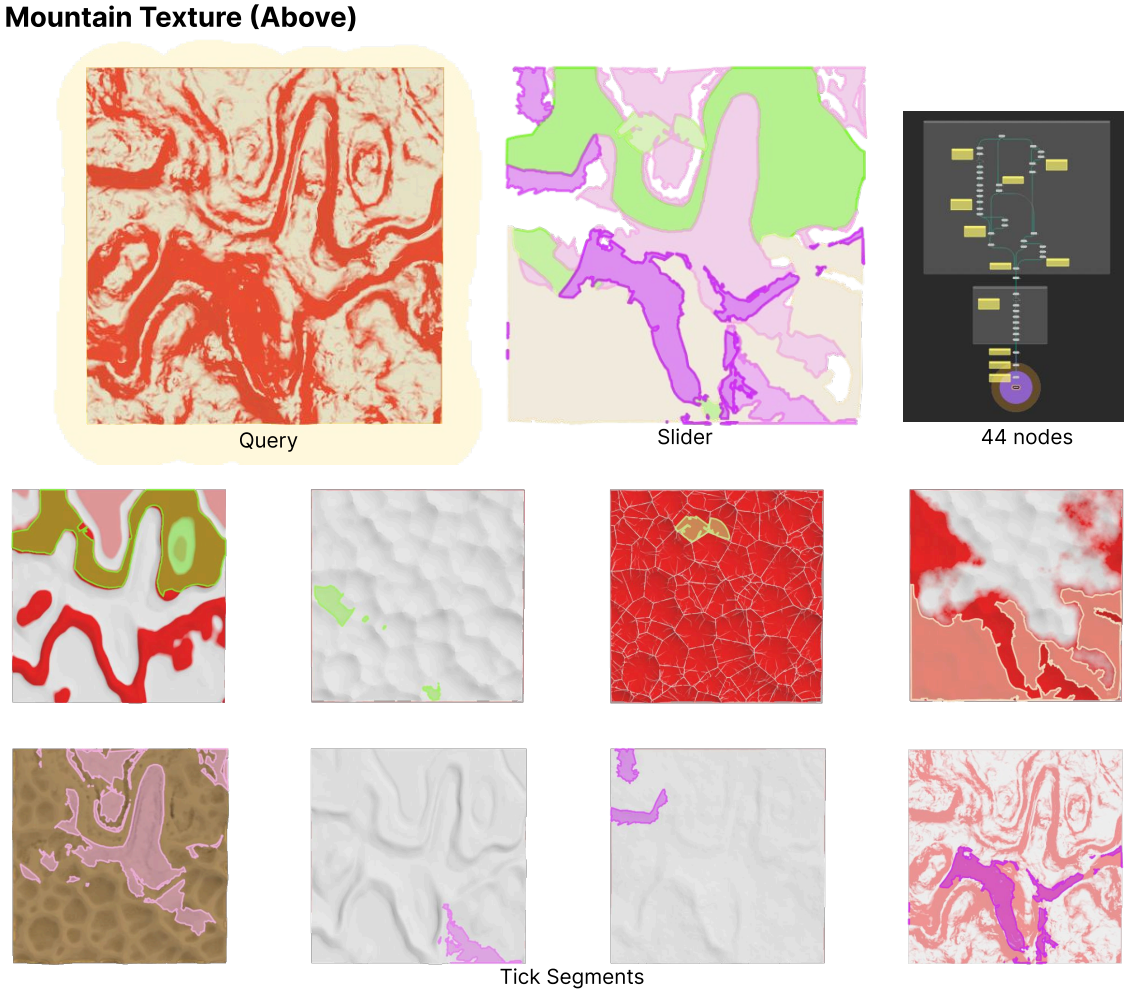


Figure 3.12: A top-down mountain-texture query emphasizes material and mask changes more than geometric structure. The representatives show height-field, color, and region-mask states that affect the queried terrain patch. The segments capture important visual features of each representative, but the sequence of segments does not show progressive textural change as clearly as the structural examples. This case shows that the same visual-query pipeline can summarize textural change, but only to a degree.

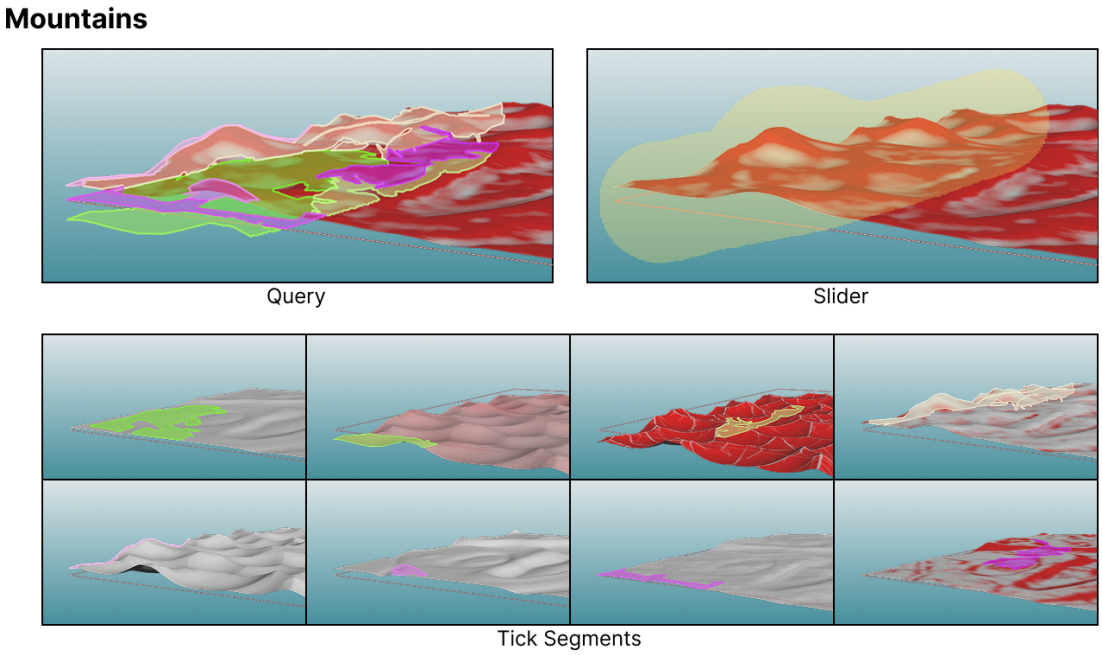


Figure 3.13: A mountain landscape combines structural and textural variation. The highlighted terrain region returns representatives for coarse terrain form, surface color, ridges, and later texture masks. Unlike the top-down texture query, this view contains both silhouette and material evidence, so the slider summarizes a mixed history of shape and appearance. When structural and textural operations are interleaved, the segment shapes can be harder to interpret. A clearer separation between structural and textural changes may produce a more readable summary.

3.6 Discussion and Direction

In-situ dependency-graph understanding can benefit from moving away from the graph interface as the only place where search happens. When artists ask a visual question by drawing directly on the image, they can express the question without first translating it into a node name, parameter name, or dependency-graph region. The in-situ summaries then show a small set of construction-history states near the artifact itself. This lets artists inspect the range of visual effects that contributed to a result without immediately reading node names or tracing nested graph structure.

The method approaches this problem by summarizing construction history through visual differences and using that summary to create in-situ handles. The image-space formulation is useful because it is mostly representation-agnostic. Construction history of image-based assets such as Photoshop files, video sequences, or image databases can be summarized with the same interaction method. Additionally with AI-assisted content creation, artists may need to understand assets whose construction process is unfamiliar, non-standard, or intentionally obfuscated.

The method is based on a simple premise: image differences can be used as a signal for summarizing construction history. In several examples, the slider handles attach to meaningful visual features such as wing segments, wall textures, and structural supports. The handles also show visible differences between stages of construction, and selecting a tick quickly reveals the corresponding representative state. This gives artists a faster way to find relevant nodes than searching through the graph interface manually, and scrubbing between ticks is more forgiving than repeatedly selecting individual nodes in the graph editor.

Despite this simple interaction, several challenges remain before the tool can be robust enough for everyday artist workflows. The examples suggest that visual difference alone is not specific enough to answer every visual question. The tool can reduce a dependency graph into a small set of visually distinct representative states, but whether those states meaningfully summarize the construction history is a separate question.

3.6.1 Query Ambiguity

An artist's visual question is often more specific than a highlighted region of pixels. They may be asking about a particular object, segment, or feature in the image. Highlighting is a quick and natural way to constrain the question, but it is also ambiguous because the highlighted region does not say which object or feature is the intended target.

This ambiguity becomes especially difficult with occlusion. Figure 3.14 shows a broad arena query where the artist's intent is meaningful but underspecified. The artist highlights the roof of the arena, but because the tool searches for visual difference inside the query region, operations behind or near the roof can also enter the result. In Figure 3.15, the artist highlights a side billboard, but interior seats, roof elements, and hidden outputs still contribute evidence in the same screen-space area. The highlight suggests a question about the roof or billboard, but the method does not yet understand that object-level intent.

Asking the artist to perfectly outline only the exact pixels that correspond to their question is not reasonable. A stronger query model would combine the mask with object identity, depth, and visibility constraints. For example, the initial query could bind to the visible surface under the brush and reject outputs that change behind that surface or on a different object segment. This would

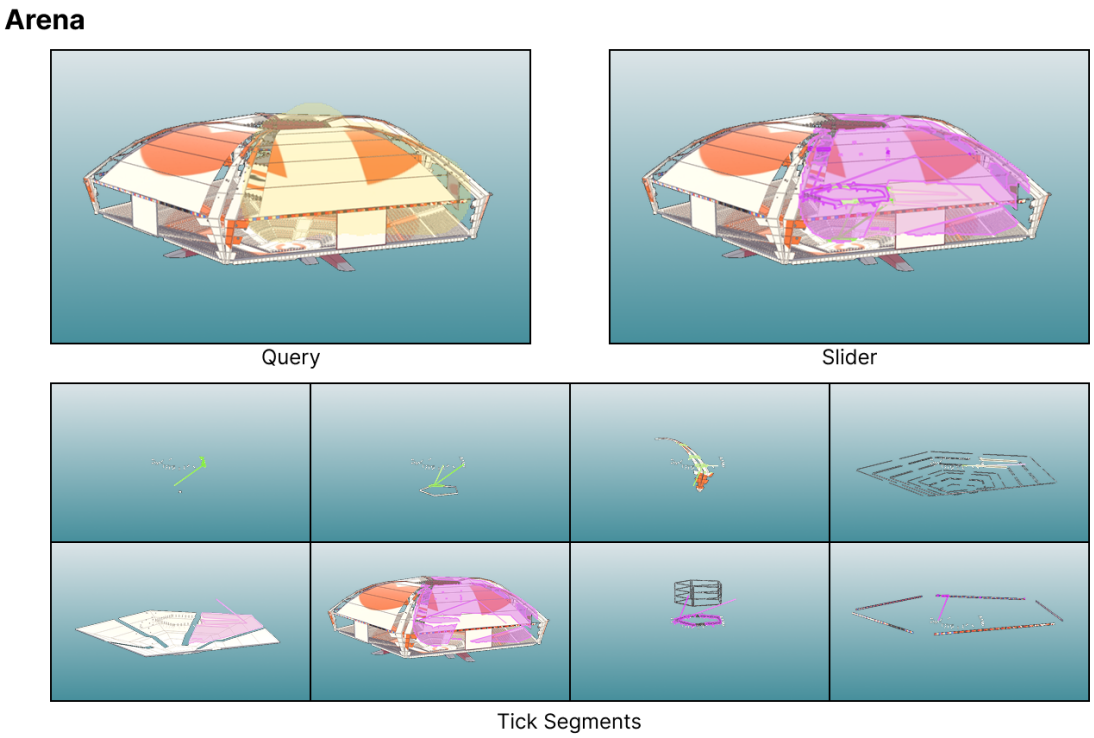


Figure 3.14: A broad arena query produces an ambiguous summary. The highlighted roof and seating region returns representatives that include roof panels, construction guides, seating-related traces, and floor or outline geometry rather than visually distinct operations applied only to the roof.

better match how artists think about queries: they often mean "this seat block" or "this wall panel," not every rendered thing under the same pixels.

Even with object identity, composition workflows make the problem harder. As shown in Section 3.5.2, artists may intentionally ask about how an object is composed rather than how one visible part is constructed. These graphs often transform, instance, and merge geometry across subnetworks, so object identity may not remain stable across the whole construction history. Instead of treating the mask as a fixed constraint for the entire query, a future system may need to interpret the initial highlight as a question about an object or segment, then follow that object or segment through the graph.

3.6.2 Representative Selection and Summary

The current tool clusters nodes by visual similarity and selects representatives from each cluster. This can produce a summary that is visually distinct but not always meaningful. Ideally, a dependency-graph summary should capture important moments in the construction history of the artifact. However, those moments are not always the most visually distinct states. Many dependency graphs are organized as long paths of operations with important checkpoints around branches, merges, and transformations. Those operational checkpoints may be better summary points than purely visual clusters.

This suggests that artists may not only want a summary of visual difference. They may want to see the key operational paths through the dependency graph. The airplane example in Figure 3.7

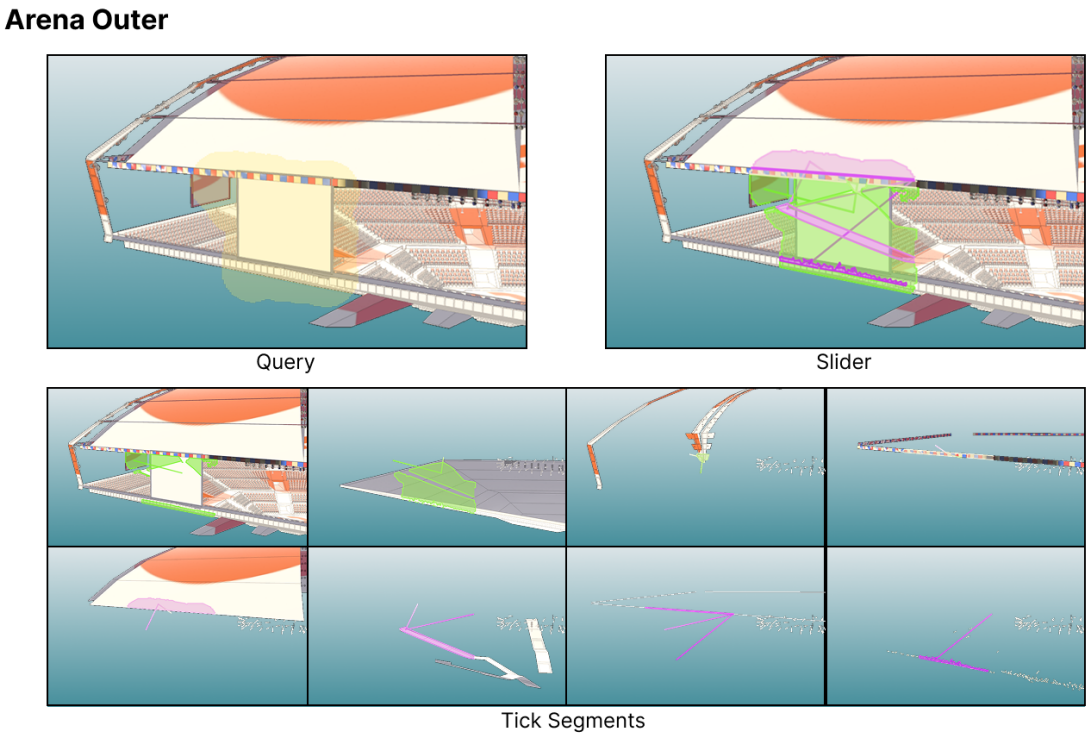


Figure 3.15: An outer-arena query shows the effect of occlusion and view ambiguity. The artist highlights a visible side region, but the masked image also overlaps interior seats, roof elements, and hidden construction outputs in the same screen-space area. Because the current method does not reason deeply about object identity or occlusion, unrelated or partially occluded nodes appear in the summary.

supports this idea: the airplane is built along a longer path with few branches, and the slider reads as a gradual buildup of the plane body. In contrast, the decorated wall in Figure 3.8 contains many branches. Its slider contains distinct segments, but the relationship between those segments is less visually continuous, which may make the construction history harder to follow. A future system could apply the slider interaction for intra-cluster navigation instead of inter-cluster selection.

3.6.3 View-Dependent Sliders

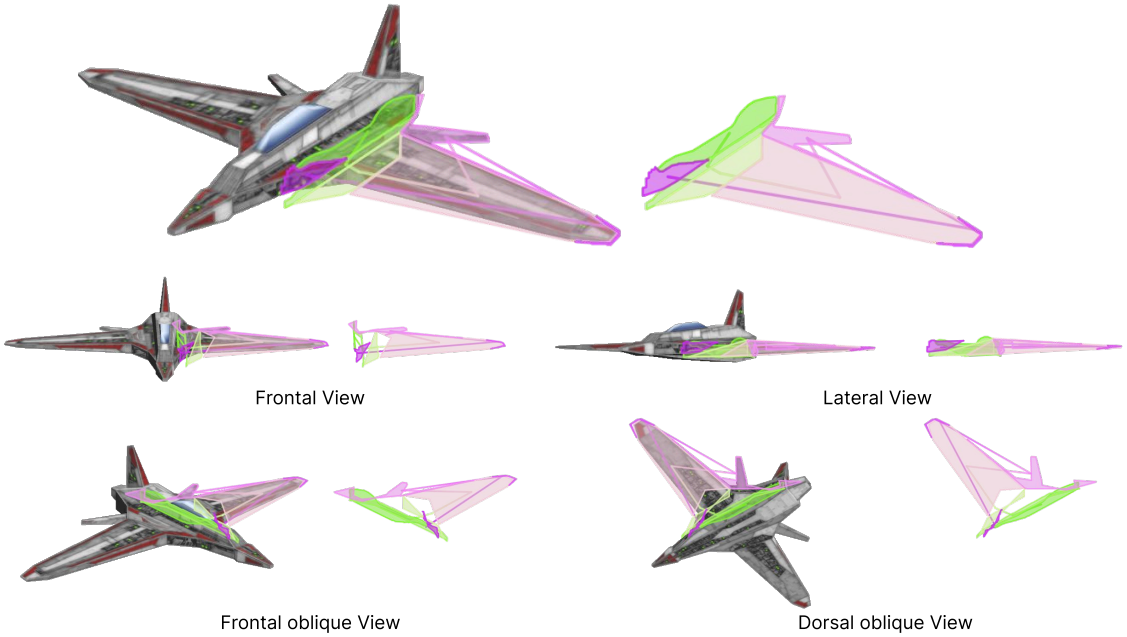


Figure 3.16: The same airplane slider viewed from several camera directions. The recovered representatives remain tied to the wing history, but the projected slider shape changes with viewpoint. Some views overlap the segment handles, while side or oblique views compress them into a thin path. This shows that scene-attached sliders suffer from view dependence.

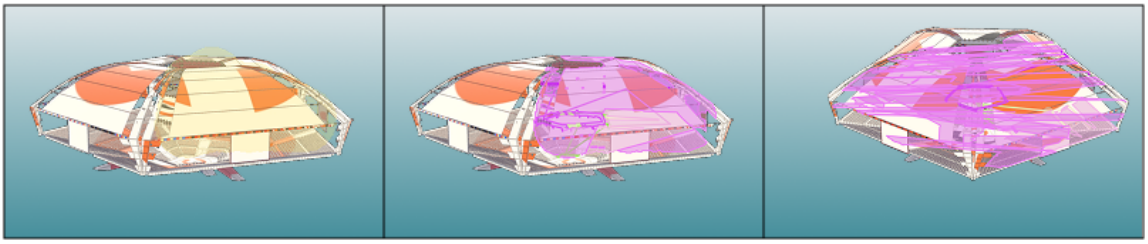


Figure 3.17: An arena slider after a view change. The segment handles become difficult to interpret when the camera moves to a view where the selected supports overlap.

For 3D environments, the in-situ slider is attached to scene geometry rather than drawn as a fixed overlay. This keeps the control connected to the artifact during camera navigation, but it also means readability depends on the camera. Because the optimization only considers the initial view, the slider can become less readable from other angles, as shown in Figures 3.16 and 3.17. This is a general issue for in-situ methods where controls are attached spatially to the scene.

The segment handles can also obstruct the artifact. Depending on the query intention, an artist may want to see more of the artifact and less of the slider. Reducing the segments to point handles would clear up the view, but it would also weaken the visual connection between a tick and the representative state it summarizes. Future work could explore more minimal handle designs that still communicate visual difference with less clutter.

3.6.4 Future Direction

The larger goal is a full in-situ dependency-graph workflow. The current system supports in-situ querying, reduction, and navigation, but it stops before in-situ editing and deeper exploration. After artists find a relevant representative state, they still need to return to the graph interface and parameter editor to make changes. A natural next step is to expose selected node parameters or subgraph controls directly near the queried region. Parameters could be mapped to in-situ controls such as sliders, dials, or toggles, but large parameter sets would require selective or summarized controls to avoid adding more visual clutter.

Finally, a formal user evaluation remains necessary. The examples support a qualitative claim that output-centric querying is a plausible entry point into understanding large dependency graphs; this claim comes from the usage by the researchers of the work and feedback from colleagues. They do not yet show when artists are faster, more accurate, or more confident than with graph-based search alone. A comparison study can be implemented that measures how artists find node parameters responsible for a visual effect using the standard Houdini network editor and the visual-query tool. However, this work proposes a new interaction paradigm focusing on the benefits of a graph-less interface for an underlying graph-structure. This contrary approach that deviates from the established comfort and standard of graph interfaces presents opportunities to better record whether graph-less workflows reveal surprising benefits that conventional methods lack from traditions. This chapter proposes such benefits informally but must be ascertained through more formal evaluation.

3.7 Conclusion

This chapter presented an in-situ visual-query workflow for exploring dependency graphs from visible consequence rather than from graph interfaces. The artist begins by highlighting a region of interest in the rendered result. The system then captures intermediate node outputs, reduces them into a small representative sequence, and turns that sequence into an in-situ slider attached to the artifact. This makes construction-history exploration begin from the visual question that motivated the search.

The examples show that this framing is useful across several kinds of procedural networks, but also that these networks behave differently. For structural creation networks, the slider can read as a local buildup of visible parts. For composition networks, the returned representatives may include whole modules, placement guides, curves, and transformations. For textural and mixed networks, the method can recover meaningful visual features.

The discussion suggests that the tool produces reasonable and simple results for certain dependency graphs by succeeding in choosing visually contrasting node outputs and in-situ UI handles that resemble what's seen. However, the tool requires more context within the visual query to correctly assume the subject being investigated. Additionally, the clustering can fail to select meaningful node

operations that summarize important steps throughout the construction history because it relies on an image-based difference assumption.

Future iterations of this work suggest understanding what visually meaningful outputs are to artists and more intelligent parsing of the visual query to match artist intention. However, the method itself provides a plausible alternative for addressing visual questions and exploration outside of traditional, panel-constrained interaction.

Chapter 4

Squidgets: Sketch-based Widget Design for Scene Manipulation

After visual querying narrows a broad scene representation toward relevant states and attributes, the next question is how those attributes can be controlled once they are the focus of interaction. This chapter studies that problem in settings where an explicit persistent interface has not already been designed for the user.

Squidgets use visible and perceived scene rendered curves as transient handles for direct manipulation. Rather than requiring users to find a named parameter in a detached interface, the system lets them draw over the visual feature they want to change and infers which editable attributes should respond. This chapter therefore moves the thesis from finding relevant hidden structure to manipulating narrowed scene attributes through visual evidence.

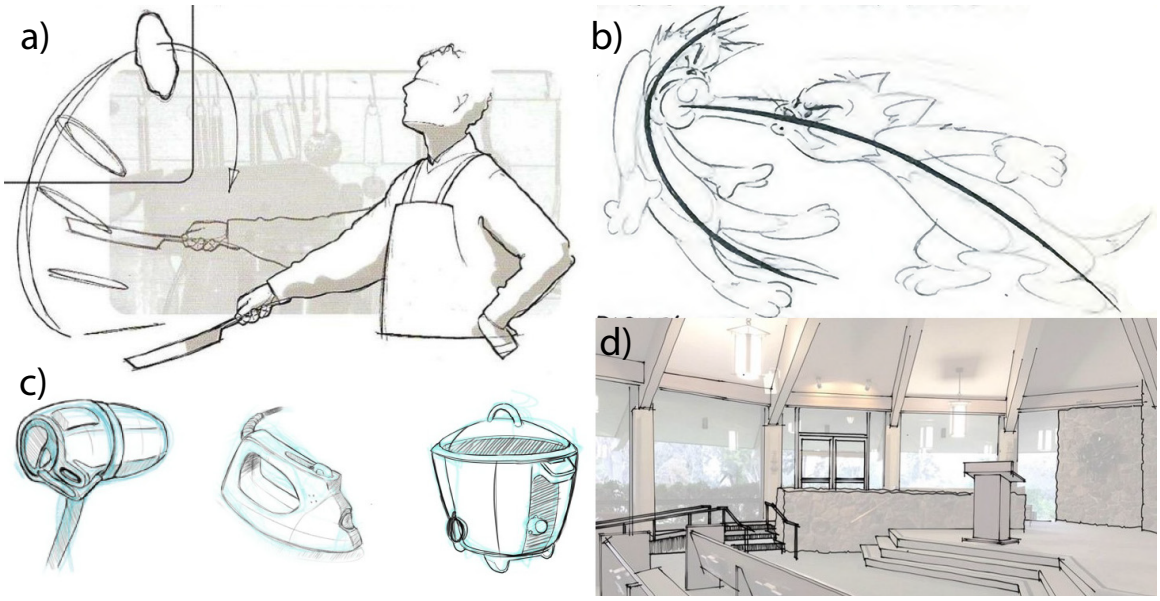


Figure 4.1: Traditional sketch stroke usage to convey aspects of animation: motion paths [40] (a), line-of-action (Tom and Jerry © Warner Bros. Entertainment) [21] (b), and design (over-sketched ideation [8] (c) and spatial overlays [104] (d)).

4.1 Introduction

Sketching is a traditionally established tool for visual communication (Figure 4.1). Digital sketching is now also ubiquitous in interactive design and graphical content creation. User control of scene objects and attributes/parameters in such applications however is typically a disparate combination of traditional UI components, pre-defined widgets, and stroke gestures [128]. While a number of seminal sketch-based systems for content creation span over half a century [10, 68, 145, 164, 166], their operational interface has largely relied on pre-defined stroke gestures and traditional UI elements.

Pen and touch stroke-based interfaces are also increasingly used to control general computing applications. A vein of research has thus adapted the interaction of UI menus/buttons [6], sliders [157], and 3D widgets [128], from a point-and-click to a stroke-friendly design. Inspired by recent research [120] using sketchy-renderings of objects for interactive visualization and exploration, this work exploits such renderings for stroke-based scene interaction.

Squidgets (*sketch-widgets*) are a novel stroke-based UI framework for direct scene manipulation (Figure 4.2). This work uses the insight that graphical scenes inherently possess implicit, in-situ handles for scene manipulation. People naturally sketch a variety of strokes (Figure 4.1) over perceived scene curves to visually indicate desired scene changes (Figure 4.2(a-e)). Practically, scene curves are either visually imagined abstraction curves (like the silhouette of the avocado in Figure 4.2(f)) or explicitly modeled scene objects (like the gaze-controller rig curve in Figure 4.2(g) or the line-of-action bookmark curves in Figure 4.2(h)). Squidgets aim to capture this natural interaction with scene curves. Conceptually, this is an inverse rendering problem: *compute a minimal change to the current scene attribute values that will result in a manipulated scene, in which some scene curve (real or imagined) visually matches the over-sketched stroke* (Figure 4.2(a-f)).

The benefits of solving such a problem are twofold. First, over-sketching is natural, direct, and

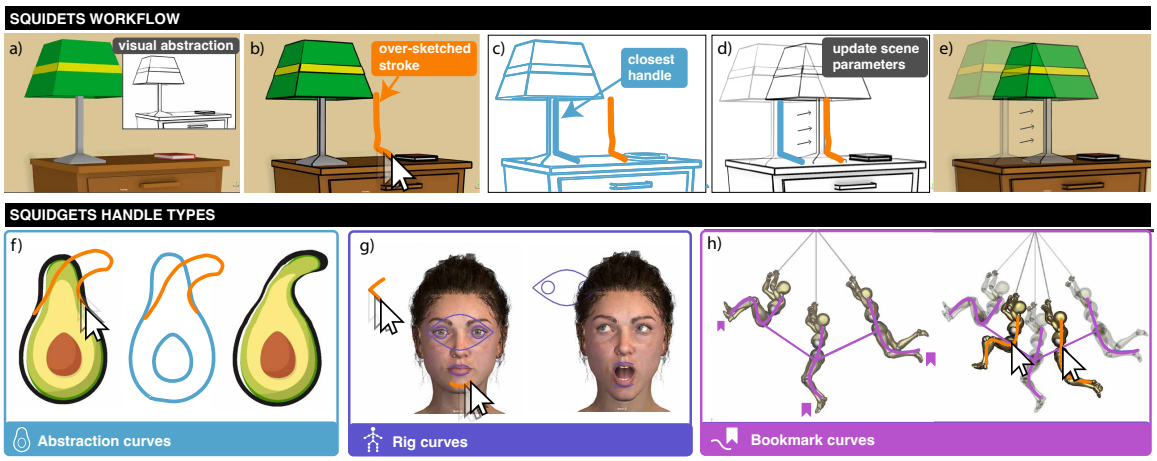


Figure 4.2: Humans naturally communicate desired scene changes by over-sketching (a,b). These sketched strokes, when aligned with visual abstraction curves of a scene (c), can effectively express user intended changes to scene attributes (d,e). Squidgets or 'sketched widgets' enable interactive manipulation of scene attributes via scene curves (f-h), such as the avocado shape attributes deformed to match its silhouette abstraction curve (f). All scene curves can homogeneously function as squidgets including pre-defined rig curves like the gaze/jaw control curves on a 3D facial rig (g), and user drawn bookmark curves that capture attribute configurations of objects, like the line-of-action curves drawn to pose an animated character (h).

in-situ for 2D (Figure 4.2(f)) or 3D (Figure 4.2(g,h)) applications. Second, no explicit gestures, widgets or UI components for the scene need to be defined or learned by the user. There are however, three non-trivial and potentially ambiguous aspects to solving this ambitious problem: (1) inferring the user-imagined abstraction curves of a graphical scene; (2) discerning what part of this curve abstraction to associate with the over-sketched stroke; and (3) computing changes to the values of a set of scene attributes that result in a manipulated scene whose associated visual abstraction best matches the over-sketched stroke.

The squidgets framework allows each of these problems to be explored, constrained, and addressed independently. (1) Visual abstraction of scenes as curves have been extensively studied in Non-Photorealistic Rendering (NPR) literature [51]. Such abstractions can be defined using occluding and suggestive contours in a scene [62] (Section 4.3), or using image-based approaches to differential rendering [92]. (2) A novel curve similarity metric (Section 4.4) matches the stroke to (partial) curves using a perceptual mix of corner, spatial, and shape proximity. A user stroke can thus be matched to a curve segment in the scene (whether explicitly modeled as a scene object, or inferred as a visual scene abstraction). (3) Scene attributes that deform the associated curve segment can then be sampled around their present values to find a resulting curve edit that best matches the user stroke.

Increasingly, artists are hand-crafting custom UI layouts using rig curves for in-situ scene manipulation [81] (Figure 4.2(g)). Such curves being explicit scene objects are homogeneously handled by the squidget framework. Inspired by artistic constructs like the line-of-action for posing animated characters [54] (Figure 4.1(b)), the framework further supports the rapid creation of bookmark curves that are associated with a given configuration of scene attributes (Figure 4.2(h)) within the squidget interaction framework. Note that abstraction/rig/bookmark curves are simply a categorization of all scene curves (real and imagined) that can serve as interaction handles in the

framework as persistent scene objects (rig curves and bookmark curves) explicitly define both a visual scene abstraction (problem 1), and the set of scene attributes associated with the curves (problem 2). Further, since these curves and their scene attributes, are related by simple invertible (often linear) functions, finding attribute values to optimally match an over-sketched stroke is simple (problem 3).

The squidgets framework thus addresses the stroke-based manipulation of scene attributes via **all visually perceived curves** in the scene (imagined abstraction curves, or explicitly modeled scene curves: rig curves and bookmark curves).

The chapter first reviews related work (Section 4.2), followed by details of the squidget framework (Section 4.3). Section 4.4 presents the approach to stroke matching, attribute inference and other implementation details of a squidget interaction prototype built within the modeling and animation system *Maya*. Section 4.5 presents a number squidget usage scenarios. The chapter discusses the outcomes of a user study (Section 4.6) followed by a discussion of overall insights, limitations, and avenues for future work on squidgets (Section 4.7).

4.2 Related Work

The squidget framework touches upon many areas of graphics and HCI research that can be roughly classified into four themes as follows.

Visual scene abstraction

The fundamental insight behind squidgets is that imagined curves in a scene provide natural manipulation handles. While such curves are often explicitly evident in 2D vector graphics (Figure 4.2(f)), they need to be algorithmically inferred in 3D scenes (Figure 4.2(a-e),Figure 4.3). Understanding and computing such a collection of curves that comprise a visual scene abstraction, is addressed by research in non-photorealistic rendering [51] and perceptual psychology [155]. The squidget framework relies on this body of work to automatically compute a set of curves that define a visual scene abstraction comprising silhouette, ridge/valley, shading contrast and border curves [14], as seen in Figure 4.3.

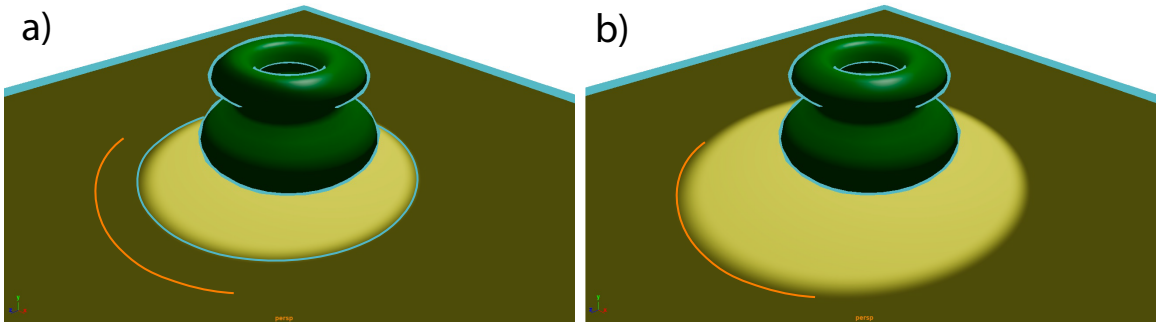


Figure 4.3: NPR and toon shading is used to compute abstraction curves automatically for a 3D scene (a). User strokes can then manipulate scene attributes associated with these curves, like the cone angle of a scene spotlight (b).

Scene object transformation and composition

A large body of research spanning 50 years [145] has specifically addressed the transformation (e.g. translation, rotation, scaling) of objects in 3D scenes. A recent survey paper [102] traces this research

from desktop to immersive devices. Techniques designed for constrained tasks like docking [18], have also been compared across various input modalities [15]. Object manipulation techniques interact with a 3D spatial context that can be physical (such as a tangible 3D proxy), or virtual (like mouse controlled 3D widgets) [23]. Squidget curves provide the spatial context, controlled by 2D (Figure 4.2) or 3D (Figure 4.9) stroke input.

Transforming 3D objects with squidgets (Figure 4.2) is but one example of the framework's ability to homogeneously manipulate arbitrary scene object attributes using squidget strokes.

Sketch-based gestures and interfaces

The recognition and use of sketch or stroke-based gestures can be roughly categorized into gestures that are hard-coded or visually matched [74]. Hard-coded gestures tend to be expressive, but context specific and difficult to customize. Visual matching, such as the simple $1 Gesture Recognizer [162], geometrically matches a given stroke to a user-given set of gesture templates. These gestures are typically scene-agnostic and can be mapped to general directives (e.g. invoking undo using a scratch gesture). Similar to visually matched gestures, squidgets match user drawn strokes to the set of squidget curves in a graphical scene.

User strokes, scribbles, and sketch gestures have been applied to interfaces in a variety of domains [16, 115] including ideation [8], modeling [10, 36, 68], illustration [77, 78, 151] and animation [54, 60, 150]. Techniques to aid the execution of sketch strokes and to aid the drawing process have also been explored [42]. While this work is focused on an application-agnostic framework for general stroke-based scene manipulation, the framework draws inspiration from this body of work for compelling examples, such as an animated line-of-action [54] in Figure 4.1(b), Figure 4.2(h), to showcase squidget interaction.

Scene proxies and custom widgets

UI widgets are visual 2D/3D elements designed to provide an in-situ interface to manipulating objects and aspects of a virtual scene [17]. Sketching such UI sliders and axes for scene manipulation have also been explored [87, 157]. Widgets are typically hand-designed to capture the form/function of scene attributes they control, and can range from simple spatial transform widgets, to custom curve controllers used to illustrate mechanical assemblies [107], or specific to the deformation parameters of complex objects such as the human face in Figure 4.2(h). Point-click-and-drag interaction of such widgets can be improved using stroke-based techniques [128]. The squidget framework is able to homogeneously interact using over-sketching with such curve-based widgets Figure 4.2(h).

Curves have also been used as visual proxies and manipulation handles for deformable objects [114, 137, 169]. Squidgets generalize this control beyond deformable objects (e.g. the avocado in Figure 4.2(f)) to any scene elements whose visual appearance can be controlled by some set of attributes (e.g. the cone-angle of a spotlight in Figure 4.3). Finally, sketched strokes have been anchored to objects in Augmented Reality scenes to provide dynamic visualizations of changing scene attributes [149], complementary to the problem addressed here where the scene attributes respond to the sketched strokes.

Direct in-situ manipulation

In-situ visualization and control of object attributes in a scene can greatly streamline a sketch-based workflow [163]. Squidgets take such a design further, allowing a user to directly sketch the visual change they expect as a result of changes in object attribute values.

Such direct visual control requires an inverse mapping from the visual abstraction of objects and other scene elements to their attribute values. Inverse computation for direct control is popular in a number of domains on Computer Graphics, such as skeletal kinematics (IK) [7, 54], facial expressions [81, 93], CAD modeling [103], and animation [30, 66]. Squidgets are inspired by these solutions from specific graphical contexts to build a general framework for manipulating sparse sets of scene attributes using sketch-strokes.

4.3 The Squidgets Framework

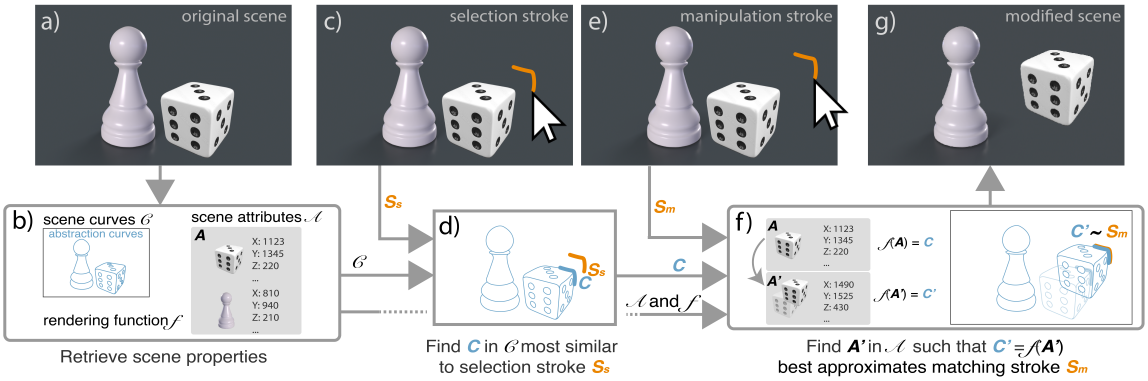


Figure 4.4: Squidgets enable manipulation of attributes that affect the visual appearance of a scene (a,b) through user selection S_s and manipulation S_m strokes (a single stroke can serve as both S_s, S_m). The system infers the scene attributes A a user wants to edit by finding a curve C from among a set of scene curves that best matches the stroke S_s (c,d: select). The system then infers the change of values from A to A' needed so the resulting curve C' best fits the stroke S_m (e,f,g: manipulate).

Squidgets define a stroke-based UI framework which develops the idea that any visible or perceived curves in a scene can serve as visual proxies and manipulation handles to scene attributes. Squidgets exploit these curves for rapid referencing and direct visual adjustment of scene attributes, illustrated in Figure 4.4: with the user workflow (top row) and underlying building blocks (bottom row). Within the squidget framework, a squidget is a scene curve C whose appearance is related to the values of an associated set of scene attributes A.

A visual scene like Figure 4.4(a) can be naturally perceived as an abstracted collection of curves $\mathscr{C}$ like Figure 4.4(b). To make a change to the scene, users first draw a selection stroke S_s in-situ to visually specify a curve $C \in \mathscr{C}$. For instance, the orange stroke in Figure 4.4c visually corresponds to the corner of the dice. Formally, given a selection stroke S_s and an ensemble of scene curves $\mathscr{C}$, the system must compute a (partial) curve $C \in \mathscr{C}$ that best matches S_s in a perceptual sense (Section 4.4.1). Addressing the added complexity of partial curve matching is necessary: motivational examples, prior art [114], and the experiments, all confirm that users naturally draw only as much of a scene curve as needed to convey a desired scene edit. For instance, only the part of the avocado silhouette that changes is drawn to convey a local deformation in Figure 4.2(f).

Once C is selected, users can draw a manipulation stroke S_m to specify how C should appear as a result of manipulating its associated scene attributes. In other words, the squidget curve C serves as an interaction handle for its associated scene attributes. Formally, let A be the set of scene

attributes and f the render function for a curve C, such that $C = f(A)$. The silhouette curve of the dice in Figure 4.4(b) for example, is a function of the dice shape, transform, and camera parameters.

Now, given a user drawn manipulation stroke S_m, the system must find changed attribute values A', such that the edited squidget curve $(C' = f(A')) \approx S_m$. For instance, modifying the translation attributes of the dice from A to A' in Figure 4.4(f) results in a curve C' where the corner of the moved dice closely approximates the user stroke S_m. A' thus represents the new set of attribute values which are applied to the scene, to result in the desired visual change to C (Figure 4.4(g)). Note, that a single user stroke can functionally serve as both S_s and S_m (Section 4.3.4).

4.3.1 Design guidelines

The following guidelines were distilled from formative, unstructured discussions with artists, observations of user scene mark-up, and in-situ manipulation workflows of scene attributes.

Curve design: All user perceived curves in a scene are potential handles for squidget interaction. These are either imagined visual abstraction curves or curves explicitly modeled and rendered as scene objects. The explicitly modeled rig curves in a scene typically capture the construction history of objects, often meticulously hand-crafted by artists to provide in-situ control over scene object attributes. These artists expressed the need for an overall interface where such rig curves could operate homogeneously with abstraction curves, but where such curves (bookmark curves) could be incrementally and efficiently created within the interface itself.

Stroke semantics: Users tend to draw simple, and short strokes whenever possible to interact with the scene. Users often draw partial strokes with just enough context to unambiguously select a scene curve (e.g. only the corner of the eye shape in Figure 4.2(g)), or to clearly convey the desired manipulation (e.g. only deformed part of the avocado silhouette in Figure 4.2(f)). For small incremental edits, interaction can be streamlined by allowing a single stroke to serve as both the selection and manipulation stroke.

Interactive manipulation: Oversketched strokes typically convey rough edits. Finer control can be enabled by repeated oversketching, or by interactively dragging the stroke spatially.

Attribute redundancy: Complex scenes often have a large number of scene object attributes that can be manipulated to satisfy a desired manipulation stroke for abstraction curves. The oversketched stroke in Figure 4.2(b) for example, could be manifested by moving the lamp on the table as in Figure 4.2(d), but also by moving the scene camera in the opposite direction, or by non-uniformly scaling the lamp to have much thicker stem. It is thus important to limit the set of attributes considered for abstraction curve manipulation, or have these associations explicitly defined with rig curves and bookmark curves.

4.3.2 Inferring scene curves and attributes

Given a 2D/3D scene, existing NPR and toon rendering techniques [14] allow efficient automatic creation of a collection of outline, silhouette, shading and feature poly-line abstraction curves of scene objects (Figures 4.2 and 4.3). These curves comprise a scene abstraction $\mathscr{C}$. The attributes A for any curve $C \in \mathscr{C}$ are typically those of their corresponding scene object. The rendering function f specifies how these attributes A define the visual appearance of C. For example, the control vertex positions of the avocado's outline curve and the spatial 2D transform attributes of the avocado object

define its shape and location in the scene in Figure 4.2(f). For shading discontinuities like the drop shadow in Figure 4.3, the attribute set expands from the objects casting the shadow to include the light attributes like the cone angle of a scene spotlight. Note that the set A associated with any curve C in the implementation is automatic and illustratively minimal.

4.3.3 Stroke to select a squidget curve

User strokes perform two functions: selecting a squidget curve (segment) based on similarity to a stroke; and changing scene attributes to make the selected squidget curve approximate a stroke.

The similarity measure between a selection stroke S_s and a target curve $C \in \mathscr{C}$ is highly dependent on scene context and application domain. For instance, in scenes where the curve shapes are distinct (e.g. rounded chess pawns vs. corners of a cubical dice in Figure 4.5(a), middle), a similarity measure based on shape alone can suffice. A shape-only metric however, performs poorly in cluttered scenes with many curve shapes being good matches for an input stroke (Figure 4.5(b,c)). For example, in a scene with multiple chess pawns, a similarity measure that favors spatial proximity between C and S_s may be necessary to disambiguate selection (Figure 4.5(a), bottom). A novel perceptual curve similarity metric based on corner, spatial and shape proximity is presented in Section 4.4.1. Note that such an algorithm can be further customized to add other matching criteria like stroke thickness and color, and additional heuristics constraining the attribute values search space.

The user's input stroke is matched for similarity against all curves in the scene, and the best matching curve is selected. Optionally, nothing can be selected if all matches are worse than a given threshold. Other mechanisms could also be leveraged for selected curve disambiguation, such as reducing the size of the curve set $\mathscr{C}$, e.g. using layer groupings of scene objects, or pre-selecting a specific scene object.

Note that single point-and-click interaction is simply a degenerate case in the squidget framework where the user selection stroke S_s is a single point. Thus, a user can select the gaze-controlling rig curve in Figure 4.2(g) with a point click (degenerate stroke) on or near any point of the eye-shaped curve, and then move the eye-shaped curve to a desired location by a subsequent click. Sketch strokes provide greater spatial and shape context for both curve selection and scene manipulation, than the spatial context provided by a single point. For example, the $<$ stroke drawn in different orientations can be used to both move and rotate the rig curve in Figure 4.2(g). Sketch strokes further enable the selection and manipulation of parts of the scene curves (Figure 4.2(f)).

4.3.4 Stroke as a squidget manipulation handle

Given a selected squidget curve, the method assumes the manipulation stroke S_m is representative of its appearance as a result of associated scene attributes being changed from their current values of A to A' (in other words $C' = \mathscr{F}(A') \approx S_m$). This can be solved as an optimization where A' is close to A in value, and $\mathscr{F}(A')$ is close to S_m. For the examples shown in the implementation, the optimization is solved as a best linear least squares minimization of transformation attributes and control point positions.

For small or incremental changes to a squidget's handle curve C, a single user stroke can specify both selection (S_s) and manipulation (S_m) (Figure 4.4). For stroke manipulation/deformation in complex scenes (e.g. Figure 4.5(b,c)), selection and manipulation steps may necessitate two strokes:

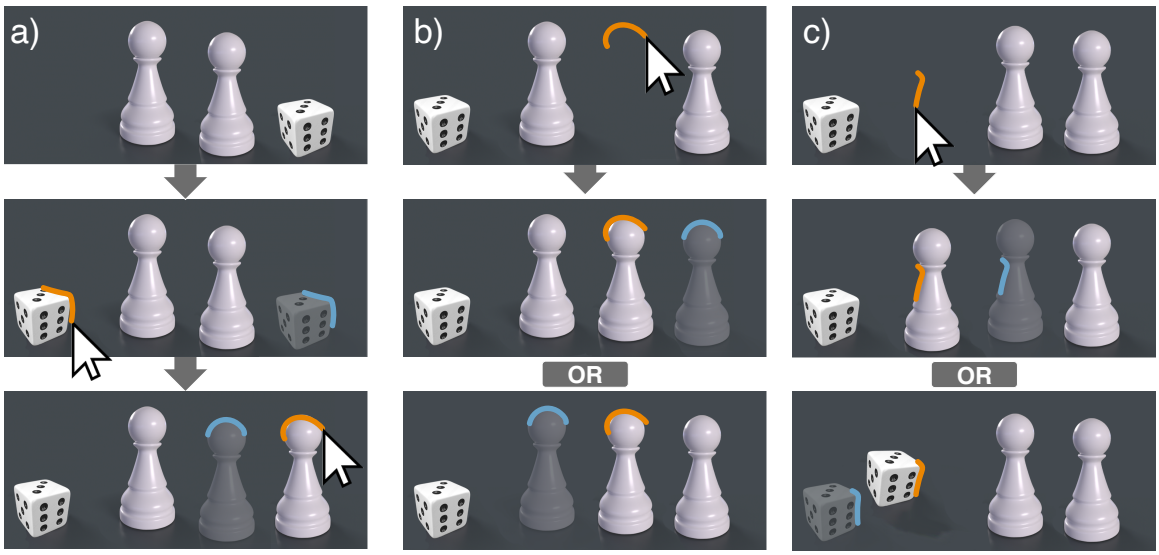


Figure 4.5: Disambiguation. Sketching unique shape features allows to unambiguously select and manipulate the dice in a one stroke interaction, even from a distance (a, middle). When multiple objects match the stroke, proximity is used for disambiguation (a, bottom). Ambiguities arise when two objects are an equally good match (b, c). In such cases, a two-stroke select-then-manipulate interaction is more suitable.

an over-traced stroke to unambiguously select a desired squidget; and a subsequent manipulation stroke.

4.3.5 Rig and Bookmark squidget curves

Artists increasingly hand-craft rig controllers: *in-situ* curve configurations pre-authored to control scene object attributes (Figure 4.2(g)). Such rig curves are selected and manipulated homogeneously using strokes within the squidget interaction framework.

Scene manipulation and setup often entails interactive exploration, where key configurations of scene object attributes are incrementally discovered and need to be bookmarked for future use. The framework supports this workflow using bookmark curves, drawn and explicitly associated with configurations of scene object attributes within the squidget framework itself (Figure 4.2(h)). Specifically a user drawn stroke can be associated with the current values of a set of scene attributes, to define a discrete bookmark curve. Selecting this curve using a squidget framework at any point sets the associated scene attributes to their bookmarked values.

A number of such discrete bookmark curves can also be chained to define a continuous piece-wise interpolation of the curves and their scene attribute values. Two discrete bookmark squidgets C_0, A_0 and C_1, A_1 for example, can be combined into a weight $w \in [0, 1]$ interpolated, continuous squidget curve $interp(C_0, C_1, w)$ with similarly interpolated attributes $interp(A_0, A_1, w)$ for a set A' that could be the union or intersection of attribute sets A_0, A_1. Continuous bookmark curves conceptually enable in-situ attribute keyframing (Figure 4.1(a)), visualized by a piece-wise linear path connecting a sequence of discrete bookmark curves (Figure 4.2(h)).

Any object selection and curve modeling tools can be used to create rig and bookmark curves with associated scene object attributes in 2D/3D. The implementation specifically facilitates the

drawing of bookmark curves on planar canvases in 3D (Figure 4.14), where each canvas additionally provides a grouping for bookmark curves and their associate scene attributes.

4.4 Implementation

The squidget framework supports a range of implementations for a variety of applications and workflows. This section presents details for an implementation within the 3D animation system *Maya* 2024. User stroke input is provided by a mouse, track-pad, or Wacom Cintiq 24HD Touch tablet.

4.4.1 Perceptual curve segment matching

A general similarity metric for curve matching is an ill-posed problem, depending on its use case and assumptions on the geometric properties of the curves. Curve matching for most interactive graphics applications are based on *corner*, *shape*, and *spatial* similarity [45]. Perceptually, we tend to align curves at corresponding sharp *corners* with matching smooth curve segments *shapes*. Depending on where the matching curves are drawn *spatial* transforms may be needed to better align corresponding shape segments. We distill these observations into a curve matching algorithm with weights that favor different criteria, making the algorithm easy to adapt to specific use cases, or fit to training curve data (Figure 4.6). We assume P is an open curve for now and adapt our solution to closed curves later.

Terminology

Curve: we represent a curve P as a poly-line sequence of points $P =< p_1, .., p_n >$ (common in interactive graphics).

Curve arc-length: computed for a curve P as the sum of poly-line segments $al(P, k) = \sum_{i=2}^{k} ||p_i - p_{i-1}||$.

Curve corners: for a curve P, let $C(P) =< c_1, .., c_m >$, be indices for internal corner points p_{c_i} in sequence. The corners thus induce smooth curve segments $P_l =< p_{c_l}, .., p_{c_{l+1}} >$ (for $l \in \{1, m-1\}$), the end curve segments being $P_0 =< p_1, .., p_{c_1} >$ and $P_m =< p_{c_m}, .., p_n >$. Segment arc-length for an internal curve segments P_l is thus $sal(P_l) = al(P, c_{l+1}) - al(P, c_l)$. At the extremes $sal(P_0) = al(P, c_1) - al(P, 1)$, and $sal(P_m) = al(P, n) - al(P, c_m)$.

Curve corner position+orientation: we define the position $o(P, i) = p_{c_i}$ for a corner point i in the set of corners $C(P)$ of curve P. A coarse tangent-like orientation at this point is given by the vector between its adjacent corners $t(P, i) = (p_{c_{i+1}} - p_{c_{i-1}})/||p_{c_{i+1}} - p_{c_{i-1}}||$.

Corner correspondence algorithm

Given two curves P and Q, we first find an optimal correspondence between the set of corners $C(P)$ and $C(Q)$, that act as sparse anchors in perceptual shape matching [45]. We use a dynamic programming formulation that minimizes the matching energy of corresponding corners on P and Q.

Let $M(P, i, Q, j)$ be the *corner matching energy* for the i^{th} corner in $C(P)$ to correspond to the j^{th} corner in $C(Q)$, defined as a weighted sum of a spatial alignment energy $A(i, j)$ needed to move and coarsely align the corners, and a coarse shape energy measured by a local disparity in curve

arc-length of their adjacent curve segments (end segments P_0 and P_m are ignored to allow partial curve matching). In other words:

$$M(P,i,Q,j) = w_t A(i,j) + w_s\Big(|sal(P_{i+1}) - sal(Q_{j+1})| + |sal(P_i) - sal(Q_j)|\Big). \tag{4.1}$$

The corner alignment energy $A(i,j)$ above is a weighted sum of a translation $Tr(i,j) = o(P,i) - o(Q,j)$, and a rotation $Ro(i,j)$ that rotates vector $t(P,i)$ to $t(Q,j)$.

Now let $E_C(P,e,f,Q,g,h)$ be the energy to match a sequence of corners from $e,\ldots,f$ in $C(P)$ to corners $g,\ldots,h$ in $C(Q)$. We can define this energy using dynamic programming as:

$$E_C(P,e,f,Q,g,h) = \min_{\substack{i\in\{e,\ldots,f\}\\ j\in\{g,\ldots,h\}}} \Big[E_C(P,e,i-1,Q,g,j-1) + E_C(P,i+1,f,Q,j+1,h) + M(P,i,Q,j)\Big]. \tag{4.2}$$

Curve segment matching algorithm

The matching corners provide anchors for a parametric correspondence between poly-line curve segments P_l (for $l \in \{1,m\}$) and their matching segments in curve Q (Wlog $|C(P)| \leq |C(Q)|$). In the event that P has no corners, we find a best-fit rigid transform that minimizes the distance between points on P and the curve Q using the iterative closest point ICP algorithm [97]. We then find the closest pair of points on ICP aligned P and Q and treat them as corners on P and Q respectively to provide an anchor for parametric correspondence. We also note that for the end segments P_0 and P_m, we use an arc-length parameterization to define a correspondence truncated by the segment of P or Q with a shorter arc-length. We now have two sets of parametrically matched curve segments in P, Q induced by P_l (for $l \in \{0,m\}$). Note again, for the case where P has no corners we re-combine the closest-point-pair induced segments after parameter correspondence, so they are treated as a single smooth curve segment. For each corresponding pair of segments P_l and Q_l, we resample the sparser poly-line to match the point count d of the denser segment. We now compare these segments simply using a matching set of corresponding points where $P_l = \{pl_1,\ldots,pl_d\}$ and $Q_l = \{ql_1,\ldots,ql_d\}$. The matching energy M_l can be combined as a single best-fit rigid transform A_l to align the point sets, and an average of residual distance-squared between corresponding points after the alignment transform. In other words,

$$M_l = w_a\|A_l\| + \frac{w_p}{d}\sum_{i=1}^{d}\|pl_i - ql_i\|^2. \tag{4.3}$$

The overall curve matching energy $E(P,Q)$ between curves P and Q is a sum of the matching energies of their corresponding curve segments $\sum_{l=0}^{m}(M_l)$.

For a given application, the best match between P and a set of curves $Q^1,\ldots,Q^k$ is then simply

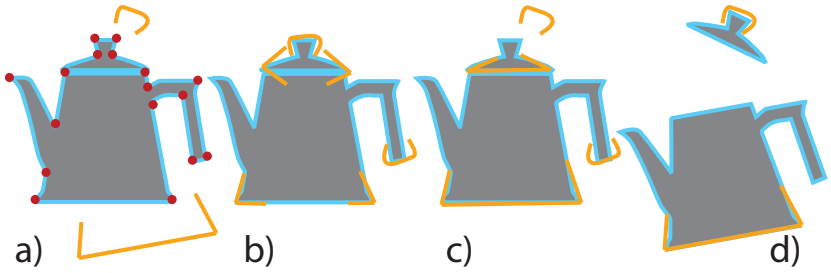


Figure 4.6: Strokes (orange) being matched to curves (cyan) with corners (red) (a). Strokes can be partially matched and aligned by ICP, and corner matching (b). Curves segments can be parametrically matched (c) and the best matched curves used to manipulate scene attributes (d).

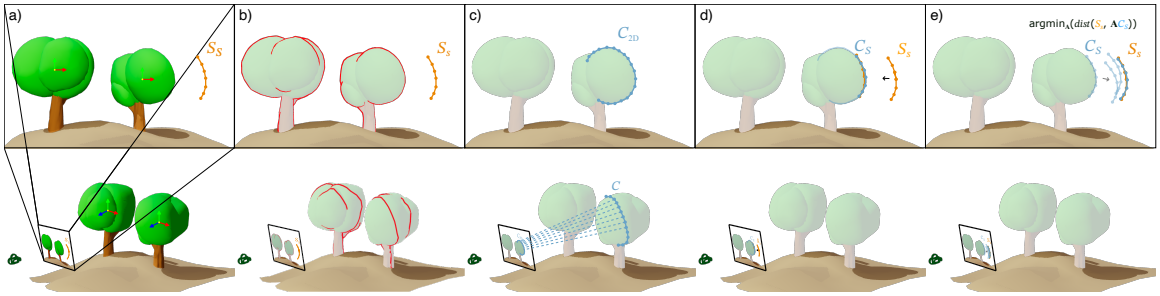


Figure 4.7: A user-drawn selection stroke is shown in screen space (top row, bottom row shows a world space view) (a). Maya toon render outlines are converted to abstraction curves in world space (b). Each abstraction curve C projected to screen space C_{2D} (c) is aligned with S (d), transforming C_S to S (e). The object corresponding to the best matching curve is selected for manipulation.

$\min_{i\in\{1,\ldots,k\}} E(P, Q^i)$, subject to setting the weight parameters to provide control over the impact of corners, spatial translation, rotation, and the shapes of the curves being matched.

For closed curves, we first perform a rigid ICP transformation to roughly align them. We then run the stroke matching above with a high spatial weighting to only allow small local movements of curve segments to align corners and curve segments.

4.4.2 Squidget curve creation

While rig curves are user pre-authored 3D scene curves, abstraction and bookmark curves need to be created or set-up by the system.

Abstraction curves

The Maya implementation uses NPR toon-rendering to create imagined curves from scene objects (Figure 4.7(b)). Arbitrary object attributes can visually change these toon curves. This implementation restricts these attributes to object transforms and object vertex positions for illustration. This section further notes that toon-rendering is view-dependent and toon curves can change discontinuously upon object or camera transformation. The system thus creates 3D curves $\mathscr{C}$ from 2D toon-rendered outlines given the current scene and camera parameters, as if they were pasted on their corresponding 3D objects, before any interactive scene manipulation begins.

Bookmark curves

Like rig curves, bookmark curves can be arbitrarily created 2D/3D curves in *Maya*. Once created, they can be associated with any set of scene attributes within the squidget framework.

Bookmark canvases: The system provides support for the streamlined creation and scene attribute association of bookmark curve groups using canvases. User strokes drawn over the canvas are projected onto the scene as 3D curves and automatically configured to the current values of the scene attributes associated with the canvas. Other attribute selection schemes were explored, such as automatically selecting recently changed attributes, but the canvas-centric approach provides a good balance between efficiency and flexibility in bookmark curve creation.

Discrete/continuous bookmark curves: Bookmark curves can be linked in sequence by drawing a stroke that crosses discrete bookmark curves. This continuously interpolated bookmark is visually indicated by a poly-line path through the mid-points of discrete bookmark curves in sequence. The path further acts like a UI slider that provides linearly interpolated control between the scene attribute values associated with adjacent discrete bookmark curves.

4.4.3 Squidget curve selection

The curve matching algorithm (Section 4.4.1) matches the selection stroke S_s to the best matching 3D squidget (abstraction/ rig/ bookmark) curve C projected into screen space Figure 4.7. Note that for 3D stroke input in AR/VR the matching takes place in 3D itself.

Corner, Spatial and Shape weights: The algorithm in Section 4.4.1 allows differential weighting of the importance of corners, a spatial transform, and the shape of curve segments in curve matching. A number of factors influence the weight settings, including: input device (trackpads and mice have greater noise than pen/tablet and thus a lower corner weight); separate select and manipulation strokes (the spatial weight for a dedicated selection stroke is low as users are expected to closely trace over the desired curve); nature of curves (for largely straight line segments in architectural applications the shape weight can be set as low, for known smooth curves the corner weight can be set low).

4.4.4 Squidget manipulation

Once a squidget curve is selected, the system computes the change to its associated scene attribute values, such that the resulting squidget curve best matches the manipulation stroke S_m. This search for optimal attribute values depends on the associated attributes. For example, for shape attributes like object vertex positions, the vertices can closely approximate the manipulation stroke by directly conforming object vertices to the shape of the manipulation stroke. For object transform attributes, there is an analytic best-fit transform that conforms a squidget curve to a manipulation stroke [109]. Discrete bookmark curves simply snap attributes to the bookmark curve closest to the manipulation stroke and continuous bookmarks are interpolated based on where the manipulation stroke intersects its poly-line path. Abstraction and rig curves can have complex relationships between their associated scene attributes and the curve, requiring a neighborhood search of attribute values, to find the curve that best matches the manipulation stroke.

For interactive control of squidget curves, users can employ a 2-stroke technique where S_s and S_m are separate strokes, a 1-stroke technique where $S_s = S_m$, or a hold-and-drag technique to interactively refine the attribute values after drawing S_m, like a virtual slider that incrementally

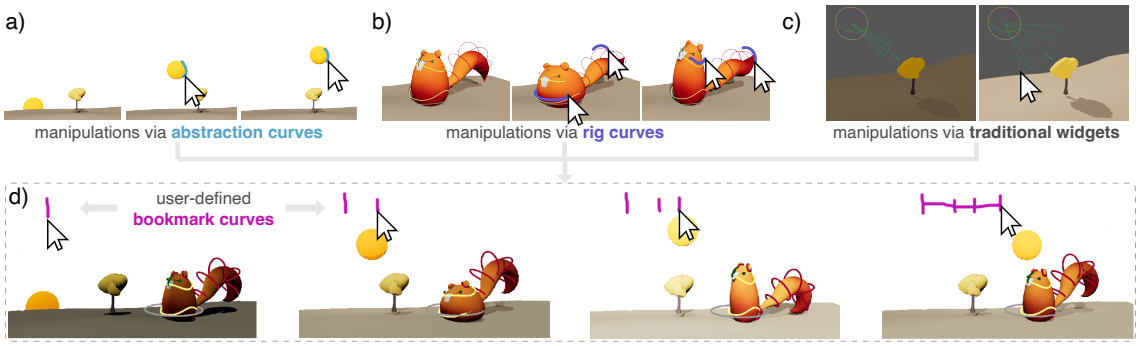


Figure 4.8: Top: To modify a scene, visual artists can freely choose between manipulating scene objects using abstraction curves (a), rig curves (b), or traditional widgets (c). Bottom: The artist can save the scene (and hence all the scene parameters) by sketching a bookmark curve (d, left). Iteratively modifying (a-c) and bookmarking scene configurations (d) allows the artist to create multiple versions, which can later be referred to, or interpolated by linking the bookmark curves (d, right).

changes scene attributes or interactively changes the object transform.

It is important to note that when squidget curves are matched in screen space, all manipulations happen orthogonal to the camera.

4.5 Applications

Most **2D design applications** already support sketch input, and can thus seamlessly incorporate squidgets for quick editing of scene parameters using abstraction curves, as illustrated in Figure 4.2(f). In **3D design applications**, in-situ control through squidgets can be a powerful approach compared to traditional widgets. For instance, in a *VR environment* sketching strokes that match the scene's abstraction curves can control characters, e.g. a stroke indicating where an axe should move Figure 4.9(a,b). Character and face rigs often have many rig curve handles for each part of the body, and posing often requires manipulating multiple handles together. Squidgets can facilitate *character and face posing* by allowing artists to directly work with face (Figure 4.2(g)) or character rig curves (Figure 4.8(b)), by drawing the shape of the rig handles. Using bookmark curves, artists can also author their own "rig" curves, as illustrated in Figure 4.10.

Combining squidgets enables **rich authoring workflows**. Figure 4.8 illustrates how an artist (a) manipulates the sun's location by drawing arc strokes to match the sun's abstraction curve, (b) poses the squirrel by sketching strokes that match rig curves of its tail, head and body, and (c) changes the scene lighting using traditional widgets. When satisfied with the scene, (d, left) the artist creates a bookmark curve, i.e. a simple tick mark for easy reference to this configuration. Then, (a-c) they continue to modify the scene, (d) adding a bookmark curve for each new configuration. Through this workflow, the artist authors key frames of a short animation where the sun rises and the squirrel bounces, which they can interpolate by creating a continuous bookmark squidget (d, right).

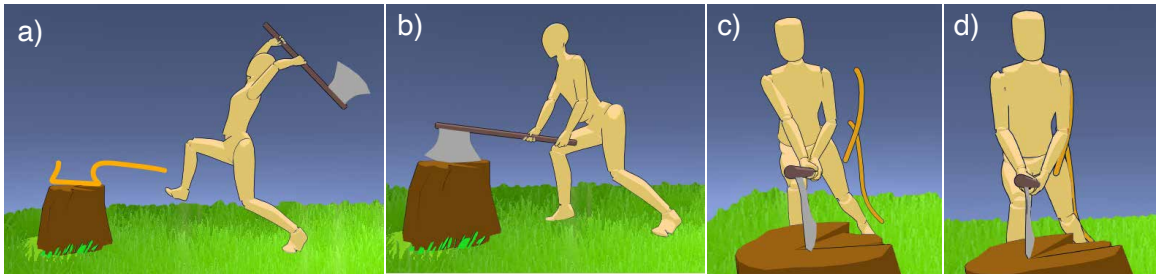


Figure 4.9: Within a VR environment, strokes are drawn to control a wood-chopper by moving the axe they hold (a-b) and adjusting the body pose (c-d).

Figure 4.10: User-authored bookmark curves can be associated with discrete configurations of face attributes to capture a smile (left) and a laugh (right); and further combined into a continuous bookmark curve squidget, where the curves and related face attributes can be interpolated (center).

4.6 Evaluation

The evaluation studies the squidget framework through a controlled experiment (§Section 4.6.1), and an informal qualitative evaluation with visual artists (§Section 4.6.2). In the interest of space, this section provides a summary; a thorough description and report can be found in the supplemental material for the original Squidgets publication [79]. The study was approved by the institutional ethics board.

Contribution Acknowledgment

The controlled experiment and informal qualitative evaluation described in this section were designed and analyzed by Dr. Fanny Chevalier. This section was written by Dr. Chevalier with review by the author of this thesis. The author of this thesis contributed to the following:

— The technical implementation of both studies.

— The conduction of the controlled experiment and co-conduction of the informal evaluation alongside Dr. Chevalier.

— The collection of data from both studies.

4.6.1 Controlled experiment: manipulate objects

The study used a within-subject experiment (n=12) to evaluate whether people understand the squidgets concept and the usability of using sketch input for manipulating scene parameters. Twelve participants were recruited through institutional channels and word of mouth. The study lasted ∼45min; participants received CAD$30. The study ran on a computer laptop plugged into a Wacom Cintiq 24HD screen tablet; and was implemented with the Maya software with most features stripped out from the view to best approximate a generic, traditional vector graphics editor.

Study Design

Participants completed two sets of scene manipulation `Tasks`: *translate* and *deform* a graphical object to match a target. The `Techniques` included a *baseline* with traditional graphical manipulation tools as found in vector graphics software, and different squidget variations: *1-stroke* (selection and manipulation are performed within a single input stroke), *select→drag* (selection with an input stroke, upon which the object sticks to the cursor to be further dragged around), and *select→manipulate* (an initial stroke is used for selection; subsequent drawn strokes perform manipulations to the selected object).

For the translation task, difficulty was varied according to the ambiguity that additional distractors would introduce for selection or manipulation: `SpatialDifficulty` refers to how far each distractor image is from the target axis location, whereas `ShapeDifficulty` determines how visually similar in shape the distractor is to the task image. For the deform task, `DeformDifficulty` modeled task difficulty as a function of type of applied deformations. See the supplemental material for the original Squidgets publication [79] for details.

Study procedure

(1) The experimenter first obtained consent, then explained to participants that they were tasked with helping design a summer drink poster by moving or changing the shape of fruit stickers, using different techniques. (2) Because the study did not evaluate discovery but rather comfort with the tool, the participants were allowed to practice for as long as desired with each `Technique` × `Task`. (3) Trial repetitions within each `Task` × `Technique` block were presented in a randomized order. For each trial, `Time`, `Operation`, and `Undo` were collected. After each block, participants were instructed to complete a NASA-TLX questionnaire [59]. (4) Finally, participants were invited to fill out a post-study evaluation asking them to rank the interactive techniques for each task, along with open comments.

Results

The analysis reports the 95% bootstrapped confidence intervals (CI) on means to assess effects [37] along with qualitative data. This section focuses on key insights; see the supplemental material for

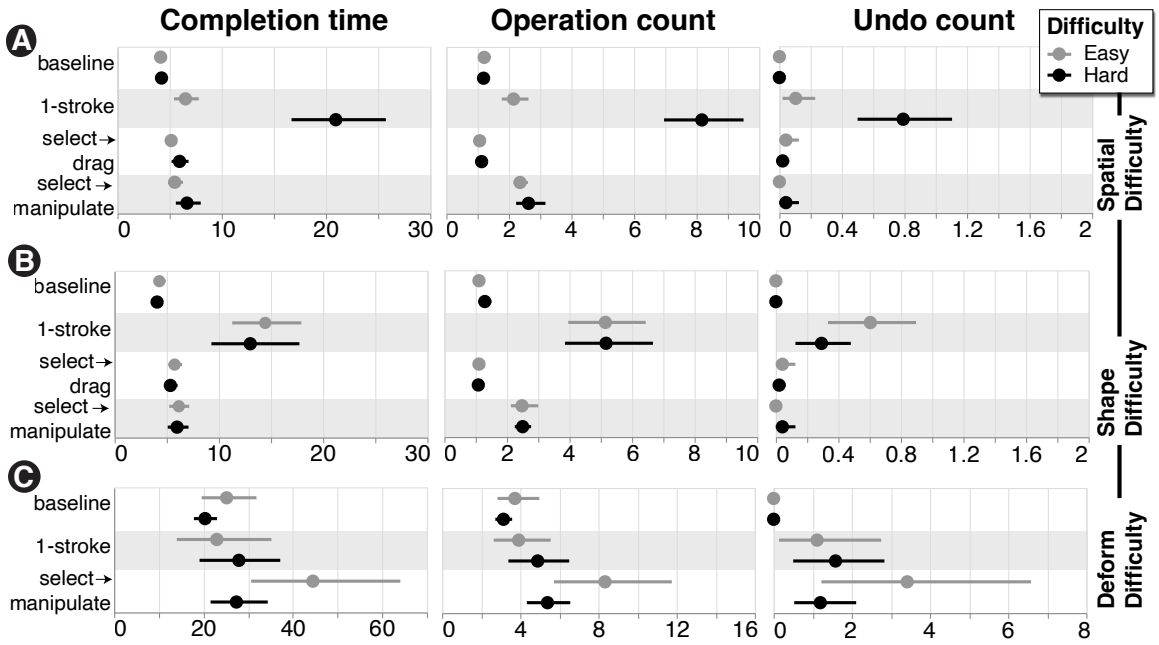


Figure 4.11: Effect of `Technique` and `Difficulty` on measures for the `translation` (A-B) and `deform` (C) tasks.

the original Squidgets publication [79] for the detailed report.

Performance: squidgets are comparable to the baseline most of the time. Figure 4.11 shows the effect of `Technique` and task `Difficulty` on performance. Overall, the baseline was generally comparable to squidgets for both completion time and operation counts, across tasks, with a few exceptions, discussed below.

For translations, distractors challenge *1-stroke*. As anticipated, the results show strong evidence that *1-stroke* presents notable challenges for move tasks with a *hard* `SpatialDifficulty`. The strokes the participants drew (Figure 4.12) are mostly arc-shaped strokes that match a small portion of the objects' silhouette. While this strategy largely works when strokes serve one of two functions (selection or manipulation), for *1-stroke* there is increased challenge in the presence of distractors. When distractors were close to the target, participants drew "incremental" simple strokes to move the object progressively. They did not attempt to fine-tune the stroke to incorporate distinguishable features for disambiguation. Participants rated *1-stroke* as more mentally and physically demanding, more effortful, more frustrating, and less performing than the other techniques for moving tasks. However, two participants rated it as the most preferred, because *"being able to do things in one stroke was overall much smoother compared to selecting."* (P3).

For translate tasks, different techniques will suit different people. The best technique for move tasks depends on personal preferences. The most popular, *select→drag* (7/12 rated as most preferred), was liked for its efficiency — *"The fastest and the most intuitive, and also had very good control in where you want to move the object"* (P2), and continuous interaction — *"You get to see real-time where the object is."* (P10). Those who preferred the baseline (5/12) also mentioned high precision and control. Participants appreciated both techniques *"were very simple and easy to understand right away."* (P9). They were more mixed about *select→manipulate*, pointing to downsides such as *"having the 2 stroke was very redundant compared to the other options"* (P8).

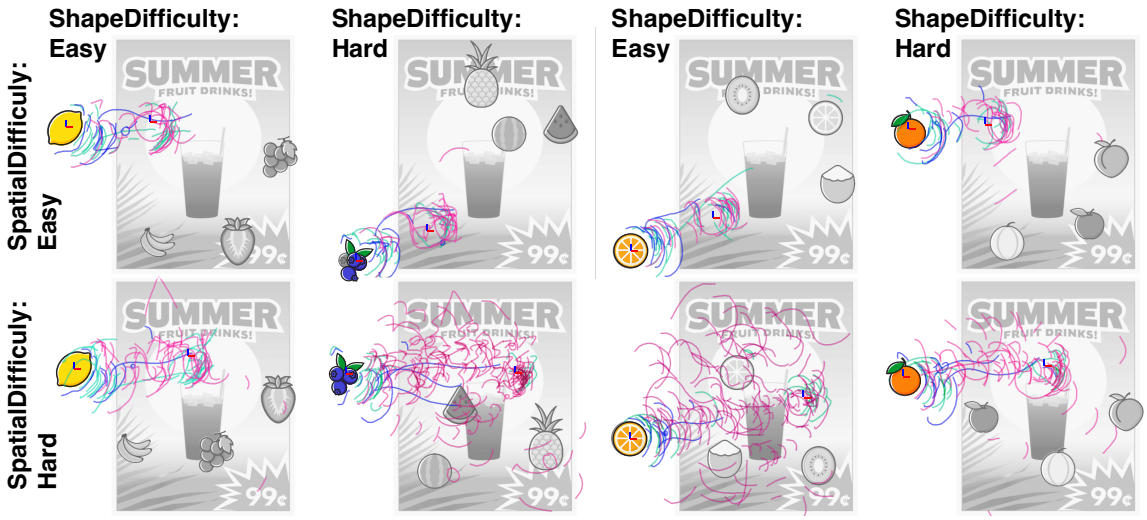


Figure 4.12: Move Task: Participants' sketch strokes for *1-stroke*, *select→drag*, and *select→manipulate*.

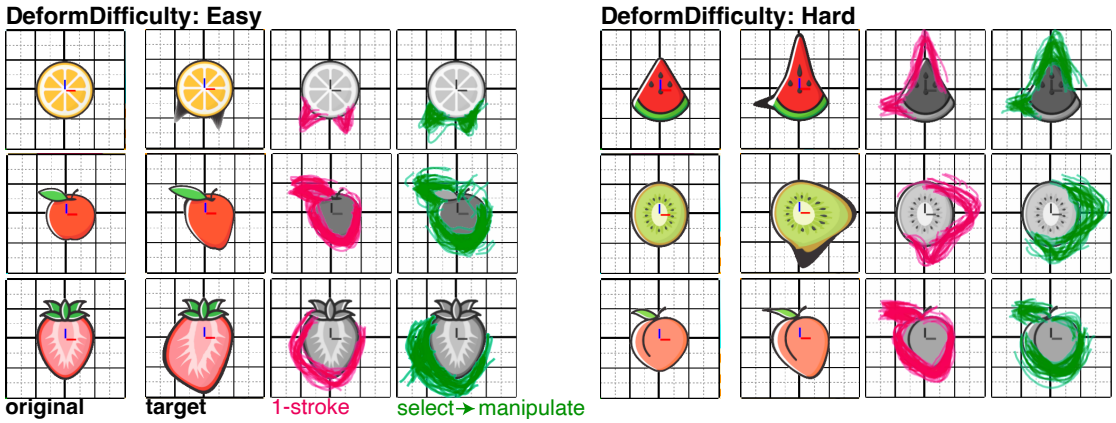


Figure 4.13: Deform Task: Participants' sketched strokes for *1-stroke*, and *select→manipulate*.

While separating the select and manipulation strokes offers advantages conceptually, participants did not recognize these benefits during the controlled experiment.

For deform tasks, a tool (baseline) is used iteratively while sketching the target (squidgets) is hit-or-miss. The *baseline* resulted in very few (if any) undo operations, but required about as many operations as *1-stroke* and *select→manipulate*. This indicates that, with the *baseline*, participants opted for *subsequent* manipulations instead of reversing the last operation. For squidgets, undo operations were more common, suggesting that participants imagined that the target deformation could be achieved in one go: *"The deform-by-stroke tool feels more natural because that's how I anticipate deformation in my head"* (P10). However, due to the challenges in drawing curves which perfectly match the intent, this required multiple attempts, especially for lay users with no particular expertise with digital drawing.

Deformations types do not correlate with task difficulty. `DeformDifficulty` had no effect, or even a countereffect on performance measures, suggesting that the metric is a poor proxy for anticipated difficulty. Participants struggled the most with trials for which they had a hard time identifying what exactly was deformed and how (i.e. peach and apple, see Figure 4.13). Note that this is an artificially introduced and extraneous step which was inevitable for a controlled experiment but would not occur in a real-world scenario, where people know what target shape they want.

For deform tasks, the sense of control is affected by the predictability of the technique and the dexterity with input. While some participants found that the baseline offers more control and predictability, others felt the opposite was true because vertex-based distortion was constraining: *"[the baseline] is my least favorite because you have to evaluate which width to use."* (P10), and having poor predictability: *"it's hard to map a larger deformation of the image to just a single vertex movement in my head."* (P11). Similarly, some found *1-stroke* offers control and is highly predictable, whereas others found it did not behave as expected, e.g. *"it moved parts that I did not necessarily want moved"* (P8). Opinions were also split for *select→manipulate*, with several finding that it gave *"finer control"* (P10), but a few finding it was difficult to understand.

For deform tasks, *select→manipulate* has strong potential. *select→manipulate* was a popular choice for deform tasks. One of its main advantages is fine control over the portion of the stroke concerned with the manipulation, which several participants noted was powerful. P11 articulated it best: *"I like it best because I feel like it provided the most amount of control - I can control the length of the curve that I want to deform as well as how much I want to deform it."*.

4.6.2 Impressions from visual artists

Animators were invited (n=6) for interactive sessions to gather their impressions of the squidget framework on 3D scenes (Figure 4.14). Four of them freely explored squidgets in informal sessions, whereas the last two (A1, A2) participated in a more formal 1-hour session where they explored all variations of squidgets while thinking aloud, followed by a semi-structured interview to discuss their experiences. A1 is a professional animator with ∼25 years of experience in the commercial and visual special effects industry, and whose primary tool is Maya. A2 is a researcher in graphics with 2 years of experience doing facial animation on Faceware[1] and Maya.

Conceptually, squidgets show promise. Animators found that squidgets were well suited for creative workflows, qualifying the techniques as "very useful", "intuitive" and "very neat". They

[1] https://facewaretech.com/

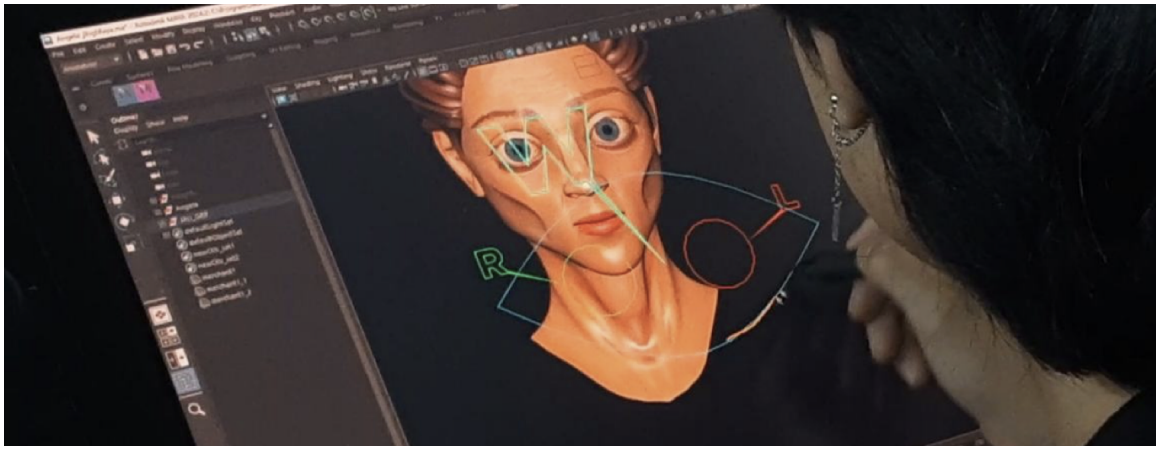


Figure 4.14: An animator uses rig curves to control a character.

found sketch input *in-situ* is a powerful mechanism that *"brings everything to the center of the scene"* (A2), which helps maintain the creative flow. This contrasts with traditional interfaces: *"as soon as you go into the interface, you're breaking what you're doing"* (A1). Animators found it to be "very satisfying" when the system correctly interpreted their intent from "just putting a stroke down". They showed excitement for the conceptual approach: *"a software that captures your strokes, and eventually learn to predict what you're trying to do [...] That's a perfect piece of software"* (A1), albeit currently, the implementation needs improvements (see below).

Projecting a 2D stroke in a 3D scene. In the controlled experiment, 2D scenes were used due to the difficulties of 3D navigation, especially for lay users. Animators worked with 3D scenes, providing insight into challenges and mitigating approaches. The implementation interprets 3D transformations with respect to the camera view, which animators found *"mentally confusing"* (A1) and misaligned with their expectations. They suggested alternative constraints to guide the transformations, such as axis constraints (x,y,z planar directions), or physics-based heuristics that result in *"plausible movements"* (A2) like moving along a wall or undulating on a water body. A1 indicated that *"even the stuff that is a little bit extraordinary is based on real world physics"*. Other scene objects were also indicated as useful guides to sketch over, for instance for assembly models, where sketching over objects will allow things to fit perfectly together *"like a Tetris"*.

Disambiguation. Animators commented on disambiguation challenges exacerbated by near-by or too similarly shaped squidgets. They strove to balance the effort required to draw enough details or unique features for disambiguation. *select→manipulate* and *select→drag* achieved this goal successfully. One observed strategy aimed to optimize for selection, then use minimal strokes for translations: *"I tried to draw over the object; and then, it would just be like dashes. Very quick ones. [...] I like how even if you were lazy and don't want to sketch the full curve, you can also get there" (A2)*. Animators also strategically picked parts that are simple to replicate to achieve precision in a two-stroke workflow. The squidget paradigm acts *"differently than how you've selected for, you know, 30 years"* (A1), which can hinder adoptability. But animators indicated that sketch selection would work well to select distant objects when these are visually distinct.

Squidgets workflow #1: Coarse and granular changes. A1 discussed combining squidgets for coarse and granular changes, noting that *1-stroke* and *select→manipulate* are efficient for large

movements but lack granular precision, while *select→drag* affords precision but is laborious for large manipulations. They described an envisioned workflow where *1-stroke* or *select→manipulate* interaction would first *"snap the object in position"*, followed by refinements by dragging while swapping between translation and rotation to *"just slightly move it exactly where it needs to be"*.

Squidgets workflow #2: Pose and bookmark in-situ. A1 and A2 quickly picked up on the potential of squidgets for animation. They experimented with posing a character [54] or a face using abstract or rig curves, keying that pose with a bookmark curve, repeating the operation multiple times with a new pose and key. Then, they linked the bookmark curves and scrubbed through the interpolated animation using *select→drag*. Within this workflow, the ease with which they could "re-key" or add a new pose without having to switch between the scene and a timeline would *"save artists hundreds of hours"*. A2 appreciated how the connected bookmark curves *"visualizes the timing in the spatial domain"* akin to notations that artists use to indicate durations between frames; also reminiscent of timelines based on motion flow [38, 76, 127].

Bookmark curves as a personalized language. Animators quickly appropriated bookmark curves as a personally-defined visual language. When bookmarking different character poses A1 and A2 drew the lines of action [54] *"following the spine and the legs of the character"* (A1), but later adopted minimal lines, as if the pose of the character was not necessary to fully "encode". Indeed, *"making curves simpler will make it easier to query [...] I think just having the position is enough for me to specify which frame I want" (A2)*. They also mentioned that custom markers would be useful for objects whose absolute position does not change, like faces.

Scenarios of usage. Concrete scenarios where animators see squidgets as a valuable alternative to their current practice include modularizing keyframe animation by creating sequences for smaller modules and controlling each squidget module with a meta-squidget; and the creation of bookmark curves for intangible, difficult to reach scene parameters (eg., lighting). A2 also felt that *select→manipulate* interaction with abstraction curves would be particularly suitable for exaggeration effects, like squashing and squeezing.

Areas for improvement. Squidgets have the potential to cause visual clutter if too many are created. Finally, because multiple different squidgets can be coupled to the same scene attributes, changing one squidget may affect the states of the others, causing concerns that such cases could yield undesirable results.

4.7 Discussion

The prototype implementation and user studies affirm the feasibility of squidget interaction and that the concept of squidgets, as a powerful and natural mechanism to express scene edits, was understood and appreciated by both experts and lay users. The studies confirmed the design guidelines and provided further insight into the squidget paradigm: (i) input device fidelity, drawing skill, and the choice of strokes for selection and manipulation, all affect the ability to select and perform desired complex scene edits; (ii) oversketching naturally aligns with 1-stroke interaction, but is only effective in simple scenes where disambiguating selection is easy, or when the desired edit is small (i.e. the selection and manipulation strokes are very similar); (iii) the *select→drag* workflow was appreciated for allowing a quick, coarse stroke for selection and manipulation that can be interactively refined by dragging; (iv) An important difference between novice and expert users lied in their ability to draw

uniquely identifying parts of strokes that aided selection and manipulation of scene objects. Experts also liked the gestural nature of squidget interaction for quick scene exploration and bookmarking, and saw it as complementary to existing tools.

Squidgets are presented as a conceptual framework for interaction. The Maya implementation of squidgets is but one instance of a squidget workflow. As the results show, there is potential within the framework to fine-tune the use of multiple strokes to disambiguate selection, vary the associated scene attributes, and the resolution and precision with which they are edited.

4.7.1 Limitations

Conceptually, squidgets capture an ill-posed and ambiguous inverse control problem. Despite this, constraining the number of squidget curves and their mapping to scene attributes can make interacting with squidgets predictable and satisfying. There may be inherent redundancy in some scene attributes. For example, a stroke indicating a larger sphere in *Maya* can be realized by a change to the sphere's radius, its scale transform, or an infinite combination of the two. Such redundancy can be mitigated by regularization or authoring explicit bookmark curves that clarify the attributes to be controlled by the squidget. The prototype implementation also makes a number of simplifying assumptions such as only controlling transform and shape attributes via abstraction curve squidgets. Given recent research on data-driven inverse rendering, future work could solve general scene attribute manipulation for abstraction curve squidgets as an optimization learned from forward rendering simulations.

While the framework supports 2D/3D scene curve manipulation, users interact with 2D strokes drawn in screen space (with the exception of mid-air strokes drawn in AR/VR). As a result, scene attributes are edited to produce view-depth preserving edits of the selected scene curve. The onus is thus on a user to pick good views to draw strokes, avoiding views where desired attribute edits significantly impact the scene depth.

4.7.2 Future work

Multi-stroke input: Squidgets, like single-stroke gestures, currently use a single input stroke for selection/manipulation. Given the ability to draw multiple strokes before they are processed for selection/manipulation can greatly increase the expressive power of squidget interaction. For example, a pair of parallel strokes could better select thin tubular objects (a wine glass stem), or three intersecting strokes could indicate a 3D frame of reference providing better control over attributes that change scene depth.

Stroke attributes: The experiment only focused on geometric shape matching for scene curve selection and manipulation. Sketch strokes themselves have a rich set of attributes like weight, color and style, that could be exploited to further control the selection and manipulation of scene curves and associated attributes.

Bookmark Curve extensions: Many of the animators in Section 4.6.2 saw the value of bookmark curves to bootstrap and improve in-situ graphical interfaces. Bookmark curves, currently used to represent discrete scene configurations, or interpolated in sequence can be embedded in-situ on manifolds to allow a richer exploration space of scene attributes [8]. Bookmark curve templates that transfer curves and associated scene attributes across objects is also exciting.

4.8 Conclusion

Squidgets present a grand unified vision for stroke-based interaction that leverages human perception of images as a collection of real or imagined curves. The prototype only implements a small subset of squidget interactions in 2D/3D scenes with 2D/3D stroke input.

While the studies do also point out challenges in squidget interaction for scene attribute control, these can be addressed by better constraining curve selection and attribute manipulation to the task context and application domain. Squidgets do place expectations on user drawing skill, but these can be reduced by simple and unique squidget curve shapes, stroke beautification techniques, incremental refinement via multiple strokes, and visual feedback that guides and snaps users towards better strokes.

Squidgets are a novel interaction framework that couples the natural tendency to indicate changes to a visual scene by drawing strokes over it. The implementation and studies show that such an approach is both viable and promising, and this work may fuel further work in direct, in-situ, sketch-based scene interaction.

Acknowledgements. I would like to acknowledge the user-study participants for providing insightful feedback on the tool; Jenny Yujeong Oh for helping run the user study; Damien Masson for helping create data figures; and Zhecheng Wang for helping with the curve matching algorithm. The Lou Rig character was modeled by Dmitry Danilov and rigged by Boris Klimov. The *Park environment* and teaser image furniture were modeled by anim_matt. The *Low Poly Benchy* model was created by Null_Hypothesis. VR environments were rendered in Gravity Sketch.

Chapter 5

Optimizing UI Layouts for Deformable Face-Rig Manipulation

The previous chapter turns visual evidence into transient controls for focused attributes. This chapter considers a complementary case: the relevant parameters are already known, but their number and spatial effects still make them difficult to access through a detached interface.

Face rigs provide a concrete example of this problem because they contain many parameters whose names and detached slider positions do not immediately communicate where their visible effects occur. This chapter uses sampled deformation trajectories as visual evidence for organizing persistent in-situ controls near the facial regions they affect. It completes the technical progression of the thesis by moving from attribute discovery, to transient direct manipulation, to persistent in-situ control layout.

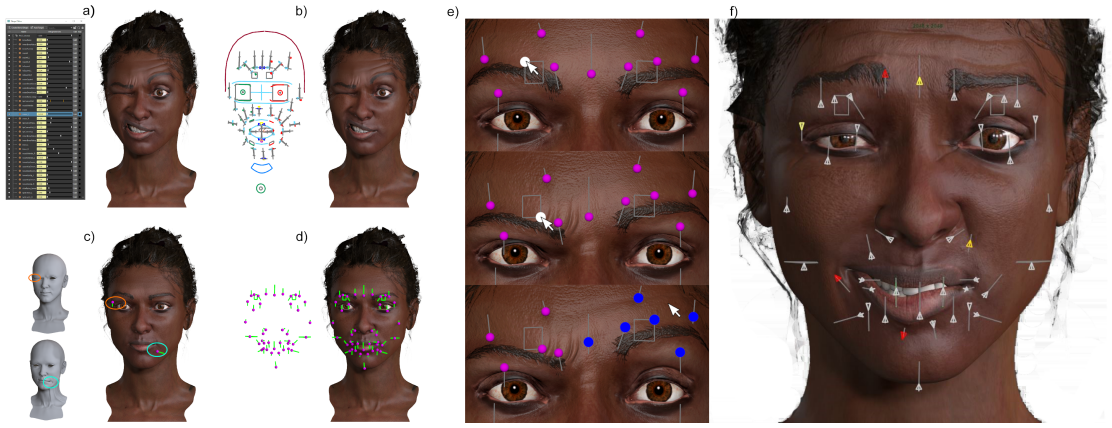


Figure 5.1: Expressive deformable models like faces have many independently controlled rig parameters (a). Animators increasingly hand-craft in-situ UI layouts for such face-rigs (b). This chapter distills their design choices into a set of layout principles (c). It then presents an algorithm to produce optimal in-situ UI layouts that match animator expectation (d); enabling a direct on-shape deformation interface (e), where rig parameters can be both independently manipulated (cursor on white control, top to middle), and best-fit computed (blue controls) to match the direct manipulation of the shape (cursor on face, bottom) [93]. The generated in-situ UI layouts can be further customized and refined by animators (f).

5.1 Introduction

Character rigging is an art that provides a number of meaningful interactive controls (like strings of a puppet) with which to bring a character to life [5]. The creative flow and expressivity of a puppet-master (animator) is impacted not only by how the strings manipulate a puppet (control parameters), but by the relationship between the strings and the puppet-master (control UI and its layout). While much CG research has focused on the anatomic, geometric, and dynamic aspects of character rigging, research into the layout and interactive control of rig parameters is relatively less explored.

This chapter introduces the problem of creating animator-optimal, in-situ, layouts of UI controls to facilitate the manipulation of a deformable character rig (Figure 5.1).

Motivation. Professional character animation rigs have hundreds of interactive controls. A Facial Action Coding System (FACS)-based face-rig [41] for example has upwards of 50 independently controlled Action Units (AUs) that define a facial expression. Modern face rigs often go well beyond FACS to model sub-muscular behavior, simulate aging, or represent families of faces [132, 133], resulting in rigs with hundreds of parameters [131].

These rigs are animated by traditional keyframing [113], performance capture [131, 154], or even speech audio [39]. While rigs are increasingly driven by performance capture, the role of an animator remains as important as ever: the popular genre of stylized animation is still tediously keyframed [113], and performance captured animation is still cleaned-up and embellished by animators working with interactive deformable rigs [132]. Film rigs are also ephemeral, with base rigs continually evolving to account for context specific corrections and shot customization [94].

The default UI for such rigs in commercial animation software like *Maya* is a disparate and long scrollable list of deformation parameter sliders, only a fraction of which might be concurrently

viewable (Figure 5.1(a)). Interacting with these sliders is woefully inadequate: knowing which AUs are needed to effect a desired change to the face is not obvious. These AUs must then be found by name in the slider list and manipulated while shifting visual focus between the 2D UI and the 3D face.

Manipulating such a large number of control parameters is challenging. Direct manipulation techniques such as [93] inverse-fit a configuration of rig parameter values, whose deformed shape best-matches a user manipulated target shape. Low-dimensional data-driven manifolds allow users to rapidly explore a meaningful subspace of the high-dimensional rig parameter space [1]. While animators appreciate the simplicity and efficiency of such approaches, they also demand independent control of rig parameters for the fine control needed to showcase their skills (similar to the complementary use of both Forward and Inverse Kinematics in skeletal animation) [117] (Figure 5.1(e)). As a result most riggers and animators meticulously hand-craft in-situ UI layouts to drive their character rigs (Figure 5.1(b)). Such in-situ/on-screen interfaces (Figure 5.2) are also increasingly prevalent in game engines, and mixed reality platforms: to support expressive character posing and animation for film, games, social media and avatar customization.

Each UI control in Figure 5.1(b) was hand-crafted in a 2D frontal view, designed to largely emulate the view-projected deformation trajectory[1] of a representative shape vertex, across extreme values of a corresponding rig parameter. The UI layout took the rigger about a day to create, intuitively picking a visually meaningful trajectory for each parameter, and laying it out respectful of symmetry, occlusion, and maximizing the use of space around the shape. Creating such a layout by hand gets significantly harder with 3D curve trajectories (Figure 5.16) and evolving rig parameters (Figure 5.12). Hand-crafted rig layouts are also brittle to layout re-targeting across characters with diverse proportions, topology, and rig parameters.

The primary contribution of this chapter is thus: an analysis of the artistic insights in creating hand-crafted face-rig UI layouts and, subsequently, an optimization algorithm that fully automates the creation of such UI layouts. The resulting UI layouts have many applications: they validate manual layout design choices and can optimally complete a partially hand-crafted rig; provide an in-situ UI layout for one-off shot-specific/layered/corrective shapes common in performance capture; provide in-situ UI layouts for novices, and one-off or non-standard characters with stylized proportions (animal in Figure 5.18); and generalize to arbitrary parametric deformations (Figure 5.16).

Problem Statement and Overview. The approach observes that the feel of direct manipulation can be provided by mapping UI handles to the deformation of a representative vertex of the shape. Defining an in-situ layout can thus be cast as a mapping $L : C \longrightarrow V$, where C is a set of rig parameters/controls, and V a set of 3D shape vertices. The problem is then formulated by the various aesthetic and usability properties that hand-crafted layouts imbibe (Figure 5.1(c)) as an extendable set of energy terms, defined for any given layout L. The optimal in-situ layout is thus found by computing the energy minimizing control to vertex mapping (Figure 5.1). While the approach is focused on face-rigs (Figures 5.17 and 5.18), the technique is designed to be generally applicable to parametric deformations (Figure 5.16).

A review of related work (Section 5.2), is followed by a discussion of the overall problem space, and development of an extendable set of in-situ UI layout design principles (Section 5.3). Section 5.4

[1]The blendShape weight parameters in Figure 5.1 produce linear trajectories, but in general this trajectory is a parametric 3D curve.

presents the details of the optimization algorithm. Section 5.5 presents a user study comparing the generated UI layout to a baseline slider interface, and artist hand-crafted rig UI layout. Further evaluation, limitations and directions for future work are presented in Sections 5.6 and 5.7.

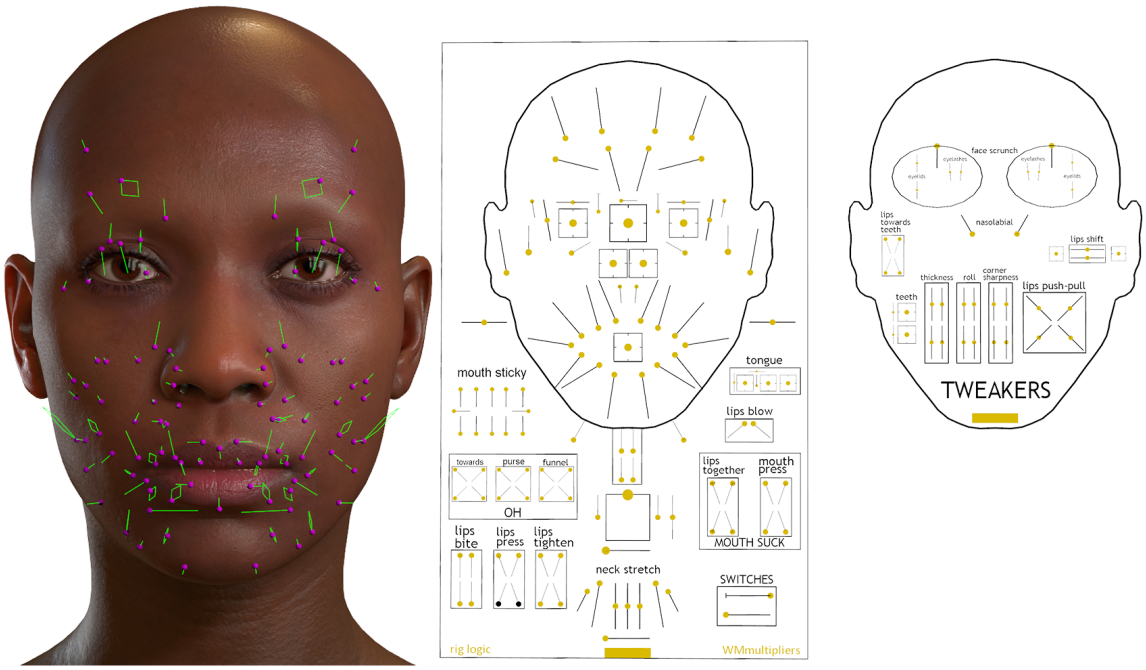


Figure 5.2: Generated rig UI layout alongside hand-crafted UI layout for the 24K vertex Metahuman with 142 parameters ©Epic Games, Inc.

5.2 Related Work

The problem of rigging and motion control for interactive character animation is at least three decades old [9]. This chapter focuses on character rigging: the interactive set-up that allows animators to manipulate parameters that control the deformation of character geometry. The majority of rigging research concerns algorithms that actually deform a 3D model, based on configurations of rig parameters. Work relevant to this chapter instead addresses the user interface between rig parameters and the animator, broadly classified under: deformation proxies and widgets, high-dimensional UIs, and direct manipulation of deformable shape.

Deformation Proxies and Widgets. UI widgets are visual 3D elements designed to provide an in-situ interactive interface to manipulating objects and aspects of a virtual scene [20]. Widgets are typically hand-designed to capture the form/function of parameters they control, like a rotation arc-ball.

A variety of general frameworks for 3D widget design have been developed [140], including approaches to interactively combine simple widgets into composite widgets for complex 3D object manipulation [129]. The method automatically creates such a composite widget for deformable shapes. Deformable objects are often interactively manipulated using proxy objects like lattices [130] or wires [137], whose direct manipulation is intrinsically responsible for the resulting shape

deformation. The approach can augment such deformable proxies by automating the UI layout of various deformation parameters. While the optimization objectives are visual design and interaction based [3], this work also draws inspiration from research on computing optimal marker configurations to accurately capture facial deformations [89].

High-Dimensional User Interfaces. Exploring high-dimensional design spaces in a creative context is a challenging task. Data-driven techniques, often aimed at novice users, reduce the dimensionality of the parameter space [86], employ design galleries of good representative examples [24, 49], or use crowd-sourced visualizations usually to aid novice users [85, 152]. Recently data-driven manifolds of deformable shape [11] have also been used in the context of rapidly posing expressive faces [1]. Expert animators are not intimidated by a large number of parameters, as long as their interface is streamlined for efficient interaction that maintains their creative flow. Hand-crafted in-situ rig UI layouts (Figure 5.3) exemplify such an interface. This work introduced the first automatic approach to creating such rig UI.

Direct Manipulation of Deformable Shapes. Sketch-based and direct manipulation alternatives can streamline spatially goal-directed animation [50] and reduce the tedium of sequentially manipulating individual parameters of a high-dimensional rig. Inverse Kinematics techniques (IK) for constrained goals, like walking or grasping [121], or aesthetic line-of-action sketching [55] position multiple joint parameters of a deformable character using high-level animator input. Facial domain systems like Face-Poser [88] use screen-space input as 2D point, stroke, and curve constraints with a deformation prior learned from a prerecorded facial expression database. Other approaches [106, 142] use 2D drawing and curve-editing to drive 3D marker-based facial expressions. Performance capture of the face or body [161] is a popular approach to driving a 3D deformable shape from markers or video of an actor. Much of this work is not animator-centric, but techniques such as [160, 167] can fit performance capture to the parameters of a deformable rig. In this spirit direct manipulation methods [27, 93] compute rig parameters from the manipulation of vertices on a 3D face, with varying degrees of local spatial control [153]. Direct manipulation techniques perfectly complement this work, for the same reason that both inverse and forward kinematics are necessary for animation. Figure 5.1(e) shows the combined interface, where selecting a UI element controls specific parameters (top, middle), and selecting the shape elsewhere sets parameters best-fit to direct manipulation.

5.3 Design Principles

Direct, in-situ interfaces keep users immersed and focused on their creative task in a 3D scene. The general problem space is vast, with many design dimensions, including:

— *Rig Parameter Structure and Mapping*: clusters and hierarchies of rig parameters, and their mapping to UI elements can provide a trade-off between visual clutter and coarse-to-fine parameter interaction. Additionally, parameter structuring can greatly reduce the computational complexity of layout optimization.

— *UI Element Form*: the geometric shape of a UI element/widget is both: an interaction handle for its rig parameter(s), and a visual proxy for the resulting deformation (for eg. the straight sliders

convey a linear blendShape trajectory, and the eyelash/chin curves convey the deformation due to *squint*/*jawOpen* parameters in Figure 5.1(b)).

— *UI Element Function*: the interaction workflow with UI elements can address design issues ranging from UI clutter, to providing visual feedback and control over deformation properties and constraints.

— *UI Element Layout*: a successful layout, for a given mapping of rig parameters to UI elements, should be optimized for visual meaning, aesthetic, and ease of interaction.

While parameter clustering is used to aid layout optimization and three different rig parameter to UI element mappings are shown, this chapter remains largely focused on UI layout, leaving detailed explorations of other design aspects to future work.

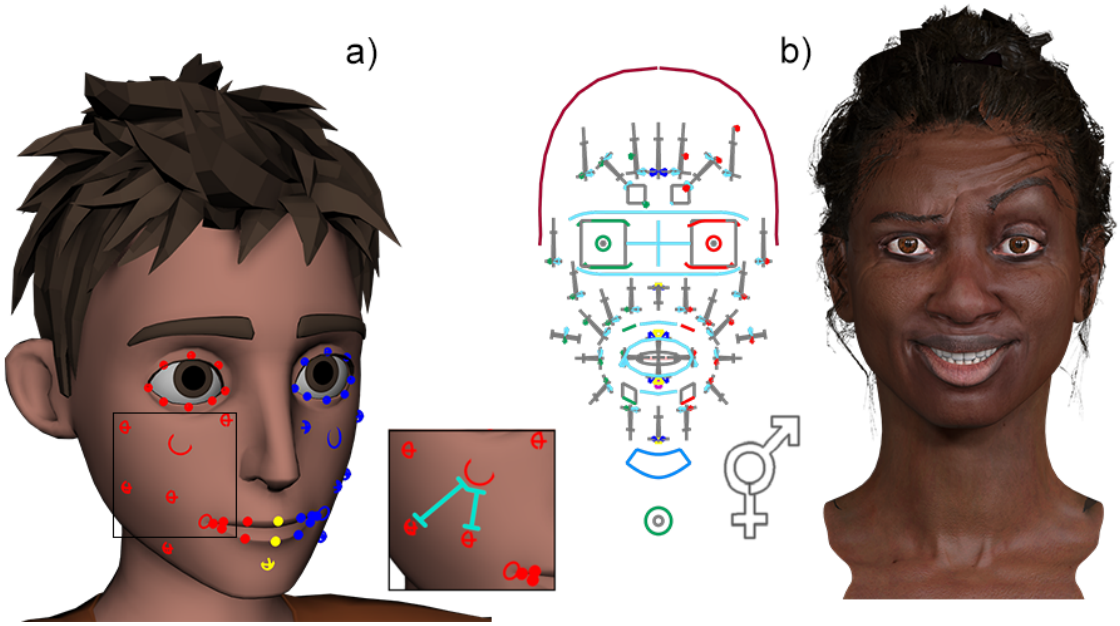


Figure 5.3: Hand-crafted rig UI layouts: the control UI is a mix of manipulation handle shapes, line/curve sliders, and patches in 3D (a) or 2D (b). The layouts are symmetric, well spread in space, minimally occluded and correspond directly to the shape deformation. The layouts can be placed on (a) or off the face (b) and UI elements for global rig parameters like gender, are typically placed beside the rig UI (b). Ray Character Rig by CGTarian ©UAB MOCAP.LT.

This section distills an understanding of animator intuition in rig UI creation from conversations with animators and an analysis of hand-crafted rigs (Figure 5.3 and near a dozen proprietary professional rigs). In all in-situ rigs the manipulation handle of a parameter was aligned to at least approximately track the deformation of a point on the face resulting from manipulating that parameter. In many instances 2D handles matched the deformation projected in a front view (Figures 5.7 and 5.17).

5.3.1 Feeling of Direct Manipulation

The UI layout problem is thus formalized as a mapping $L : C \longrightarrow V$, where C is a set of m rig parameters $c_1..c_m$ and V the set of n 3D model vertices $v_1..v_n$. Let $P = \{p_1..p_m\}$ be a configuration

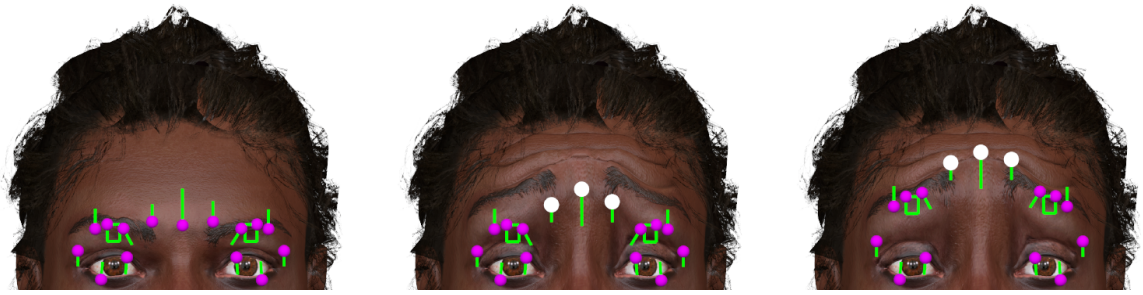


Figure 5.4: UI elements on the eyebrows (left) can remain static (middle) or interactively adapt (right) to parameter manipulation (white controls).

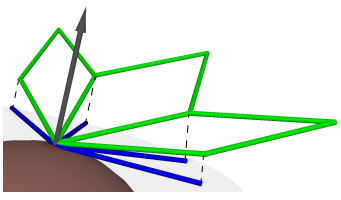
of the rig parameter values, where p_j lies between min_j and max_j values for control parameter c_j. Let $l_j \in \{1..n\}$ be the vertex index that c_j is mapped to under some layout $L = \{l_1..l_m\}$, and $d_j(t)$ the parametric 3D trajectory of vertex l_j, as it deforms under some parameter configuration P (for eg. the default parameter values) with only c_j changing from $t = min_j$ to $t = max_j$. For the blendShape weight parameters seen in Figure 5.1, this trajectory d_j is a straight line (in general $d_j(t)$ is a parametric 3D curve).

Direct manipulation dictates that the UI element E_j for any control c_j should be shaped by d_j. The UI elements in Figures 5.1, 5.2 and 5.17 are mostly 3D line sliders, but can also conform in shape to the region being deformed (for eg. eyelash/lip/chin in Figure 5.1(b)). The formulation therefore needs to support general element shapes and trajectories, like parametric curves, patches, and fans (Figure 5.5), and the goal is to find a layout L, whose UI element shape and trajectory optimize certain functional and aesthetic criteria.

Note that the trajectory $D = \{d_1, ..d_m\}$ for any layout L, also depends on the parameter values P. Conceptually the layout can either be optimized for some default parameter configuration, or constantly recomputed as parameter values P change. Animator consultations and HCI principles [48] confirm that persistent layouts become familiar and more predictable for users. Optimal layouts, like hand-crafted layouts, are thus only computed once, for some neutral/default or user-defined parameter configuration P. While the layout mapping remains fixed, all UI elements should deform to remain attached to the interactively manipulated shape (Figure 5.4), to provide a feeling of direct manipulation.

5.3.2 UI Elements

Figure 5.5 shows various linear UI elements. A *single-parameter slider* independently manipulates a rig parameter. A *two-parameter patch* allows simultaneous 2-DOF manipulation of two parameters. A *multi-parameter fan* (inset) extends a patch to allow simultaneous 2-DOF manipulation of adjacent fan parameters. The trajectory tangents (green) for the parameters at the common layout vertex are projected (blue) onto the tangent plane at the vertex, and the radial angular order of the projected tangents in this plane defines the adjacency of parallelogram patches (angle $\leq 180^o$) of the fan.



The rig parameters that define multi-parameter elements typically deform the same region somewhat orthogonally, such as a *jaw-open-close* and *jaw-left-right*. While multiple rig parameters are used to define patch and fan UI elements, such elements should be represented by a single representative control in the layout C, so that the element can be created at a common vertex in the optimal layout L. Below the formulation assumes each control c_j to map to a unique UI element E_j.

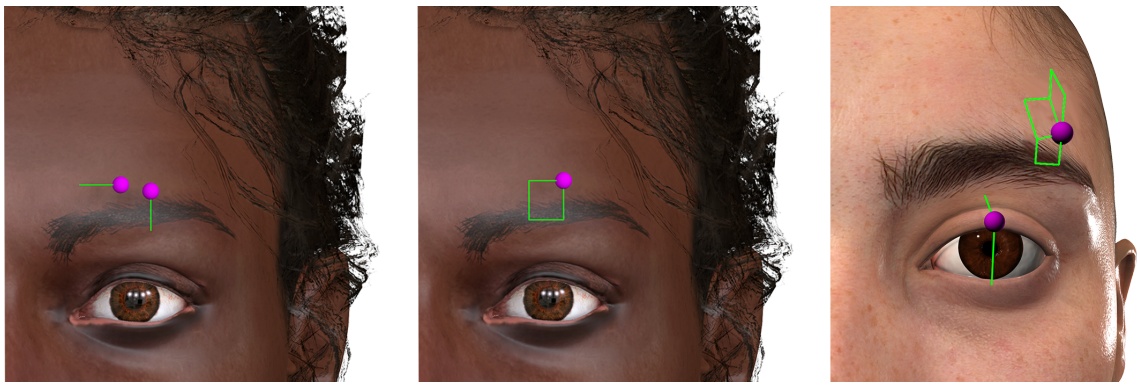


Figure 5.5: Rig UI layouts can be optimized for arbitrary UI elements, such as the sliders, patches and fans shown. Further, the algorithm can automatically group rig parameters with large overlapping regions of deformation, to define 2-parameter patches, and multi-parameter fans.

5.3.3 UI Layout Design Objectives

The following extendable set of objectives guide the functional and aesthetic quality of a UI layout. These objectives are discussed for single-parameter sliders. For a multi-parameter UI element (patch or fan), the objective is set to be the worst objective value computed for each of its mapped parameters independently.

Maximal Displacement. The visual impact of manipulating a rig parameter is best observed in regions where it invokes large deformations. Functionally, larger deformation trajectories also produce larger UI elements that are easier to manipulate with better resolution. A user wishing to *squint* an eye in Figure 5.1 for eg., would expect to tug directly on a UI element that tracks a vertex on the eyelid, instead of a visually confusing and unusable zero-length UI element attached to say, a vertex on the chin. As a down-side, a large UI element may disproportionately occupy space and interfere with the 3D shape and UI elements of other rig parameters. This objective is modeled for a candidate layout L as a function of the arc-length of its deformation trajectories D.

Minimal Inter-Element Overlap. The UI elements in hand-crafted layouts (Figure 5.3) are spatially well-spread. Elements that intersect or have significant spatial overlap are unattractive, visually cluttered and difficult to select unambiguously. The objective aims to minimize inter-element overlap (or alternately maximize inter-element distance).

Figure 5.6 shows the trade-off between the use of maximal displacement, and minimal inter-element overlap in the approach.

Element-Model Occlusion. It is visually desirable for UI elements to remain largely un-occluded by the 3D shape during interaction (even though they can always be drawn un-occluded as an overlay). Locally, the objective favors trajectories for control c_j whose tangent d'_j at the default configuration, aligns well with the 3D vertex normal for the layout vertex l_j. This alignment can be measured as a function of the dot product of the vectors. Element-model occlusion can also be addressed in a view dependent fashion by limiting candidate layout vertices, to those that are visible from a given view in the default configuration. Finally if a view-dependent 2D layout is desired, the optimal layout can further be computed after view-projecting elements in 2D, to penalize strongly foreshortened UI elements (Figure 5.7).

Symmetric Elements. As characters are inherently symmetric, most rigs have parameter sets that deform shape symmetrically (for eg. a *left-squint* and *right-squint* control). Such symmetric controls are readily inferred automatically [108], or can be user annotated. UI elements for symmetric parameters should convey that symmetry visually. Given controls c_j, c_k subject to some symmetry transform S, precise symmetry is enforced by removing c_k from the layout optimization, but retaining its UI element as part of c_j (i.e. the UI element for c_j is defined as both E_j and its symmetric element $S(E_j)$). This ensures that the optimal l_j will account for both UI elements it is responsible for creating, and that the UI elements for c_j and c_k are symmetric. The above trivially generalizes to multi-fold symmetry by optimizing the UI layout of one representative control c_j for each set of symmetric controls.

Symmetric Deformations. Characters often have individual parameters c_j that deform the shape symmetrically (for eg. a *brow-raise* control that wrinkles the forehead symmetrically). Such controls should visually lie along their plane or axis of symmetry. This can be enforced by constraining the set of candidate layout vertices for l_j to those that lie precisely on the plane or axis of symmetry. In the absence of such vertices, the hard constraint can be replaced with a function that penalizes the distance of a candidate vertex l_j, from the plane or axis of symmetry.

Global Deformations. While most rig parameters are manifested as localized shape deformations, some rig parameters, such as one to customize gender may deform all vertices of the shape significantly. Placing a UI element for such a control locally on the shape can be misleading, and the UI elements for such controls, such as the gender symbols in Figure 5.3(a), are often presented as global and off-shape.

5.4 Layout Optimization Algorithm

The search for a layout (mapping m rig controls to n shape vertices) L^*, that optimizes the design objectives in Section 5.3.3 is an instance of an integer quadratic programming problem, known to be NP-Hard. This section first describes the quadratic programming formulation below, and then looks at various pre-processing steps in Section 5.4.2, that reduce the computational complexity, and make the optimization practical for state-of-the-art industrial rigs (Figures 5.2 and 5.12).

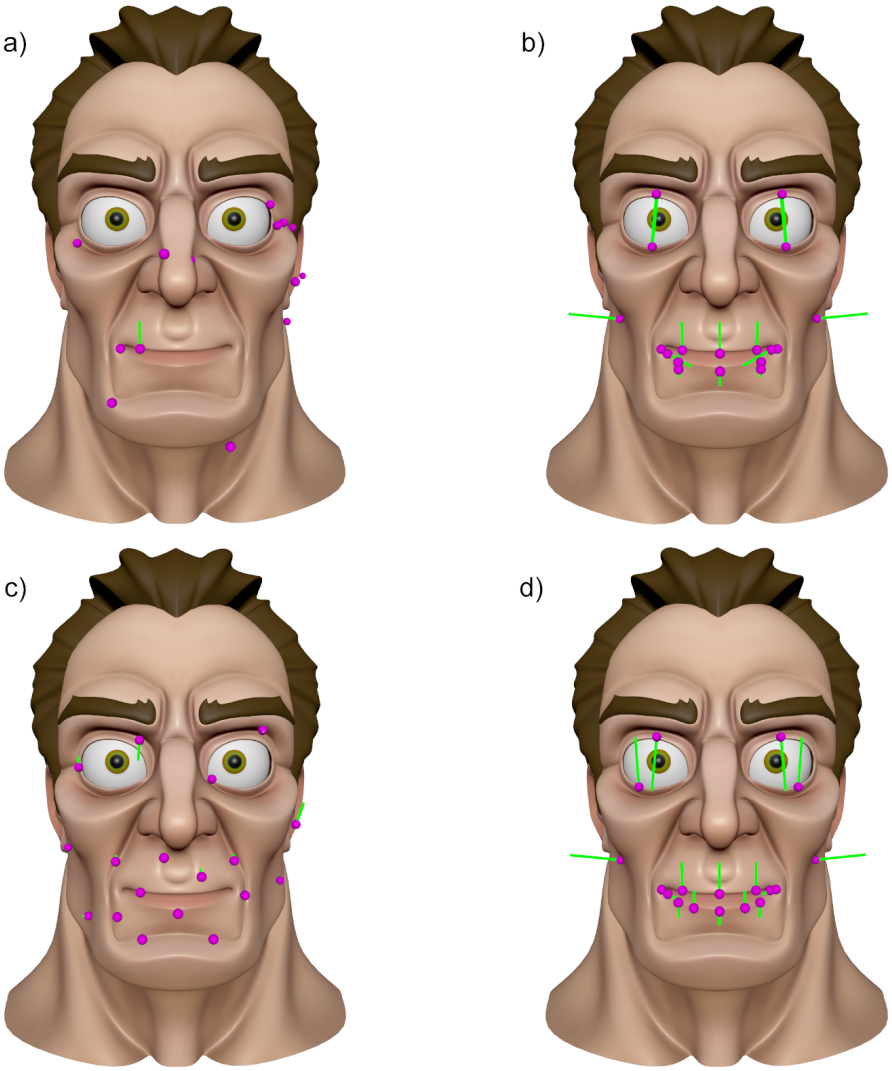


Figure 5.6: Design Choices: a random control to vertex mapping produces visually meaningless and unusable UI layout (a); a layout based on maximal displacement can result in visually cluttered and occluded UI elements (b); a layout based on minimal inter-element overlap creates well spread but small UI elements (c); the approach jointly optimizes displacement, overlap, occlusion and symmetry criteria (d). Rig by ©Antony Ward (3D World)

5.4.1 Integer Quadratic Programming

A candidate layout L is represented using an $n \times m$ binary matrix $\mathbf{X}$ where $x_{ij} = 1$ iff control c_j maps to vertex v_i in L (i.e. l_j is v_i). Each column j of $\mathbf{X}$ is thus all zero elements and a single 1 in the row for vertex l_j. The matrix $\mathbf{X}$ is interchangeably written as a concatenation of rows into an $n * m$ column vector $\vec{x} = [x_{11}, x_{12}...x_{nm-1}, x_{nm}]^\intercal$. The notation uses x_{ij} equivalent to $x_{(i-1)*n+j}$, and x_k as equivalent to $x_{row(k)col(k)}$, where $col(k) = (k-1)\%m + 1; row(k) = (k - col(k))/n + 1$.

The maximal displacement objective for a given rig parameter configuration, is also encoded as an $n \times m$ column vector $\vec{d}$, where d_k represents the arc-length of the deformation trajectory of vertex $v_{row(k)}$ controlled by UI element $E_{col(k)}$.

Inter-element overlap is encoded as a symmetric matrix $\mathbf{Q}$ of size $nm \times nm$ where q_{rs} is the

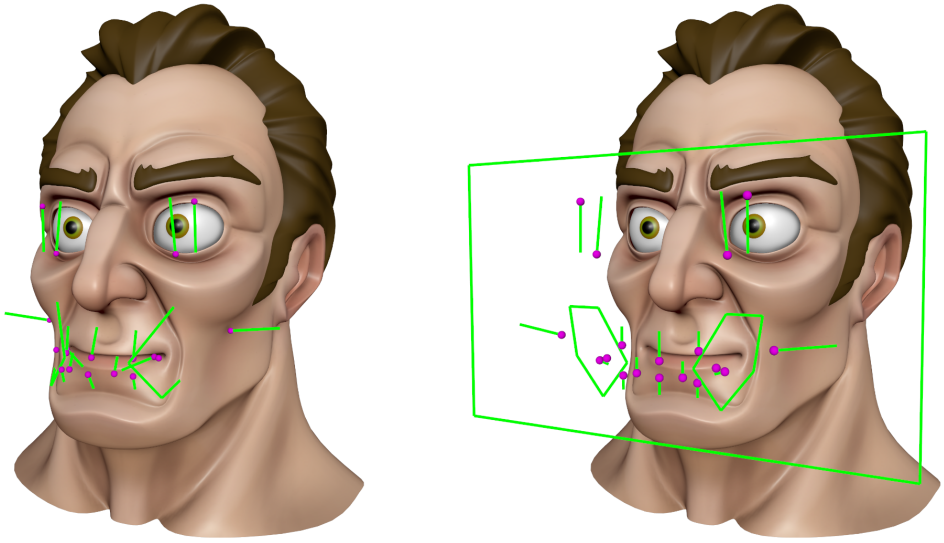


Figure 5.7: UI layouts can be optimized in 3D, or 2D from a given view.

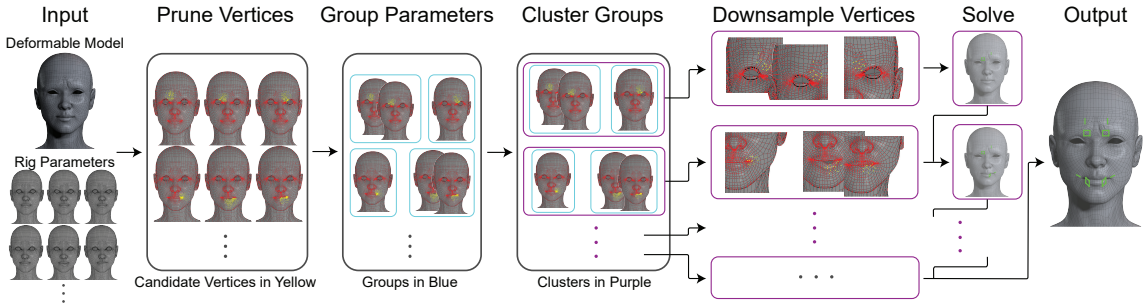


Figure 5.8: Algorithm Overview: model vertices are first pruned based on design criteria to a candidate set of layout vertices for each rig parameter; the parameters are then grouped and clustered based on the overlap and separation of their candidate vertices; candidate vertices in each cluster are then sampled and used to compute an optimal layout using integer quadratic programming.

inter-element overlap $\phi(dist(x_r, x_s))$, where $dist(x_r, x_s)$ computes an inter-element distance between element $E_{col(r)}$ mapped to vertex $v_{row(r)}$, and element $E_{col(s)}$ mapped to vertex $v_{row(s)}$, and ϕ is a sigmoid function.

The method calculates the inter-element distance to capture the difficulty of selecting a point on a UI element due to spatial interference with another element. The method first discretely samples any x_r (uniform parametric sampling): the UI element $E_{col(r)}$ placed in 3D at shape vertex $v_{row(r)}$ is sampled at u points $W_r = \{w_r^1 ... w_r^u\}$. The shortest distance is then computed from every point in W_r to the point sample set W_s for x_s, and vice versa. The overall average of this shortest distance is a symmetric distance measure $dist(x_r, x_s)$. This distance is then inverted to capture overlap using a function $\phi(z)$:

$$\phi(z) = a - \frac{a}{1 + e^{s(z-\tau)}} \tag{5.1}$$

where a is the maximum overlap error, τ is the distance for z which gives $a/2$ overlap, and s scales how quickly the overlap penalty vanishes with increasing distance. In this implementation values $a = 5$, $\tau =$ average edge length between adjacent shape vertices, and $s = 2$ were used.

These objectives are cast as Equation (5.2) in a quadratic program:

$$\arg\min_{\vec{x}} \quad \frac{1-\lambda}{2}\vec{x}^{\intercal}Q\vec{x} - \lambda d^{\intercal}\vec{x} \tag{5.2}$$

$$\text{s.t.} \quad \mathbf{A}\vec{x} = \vec{1} \tag{5.3}$$

$$\mathbf{B}\vec{x} \leq \vec{1} \tag{5.4}$$

$$\vec{x}_{1..nm} \in 0, 1 \tag{5.5}$$

The relative importance of large displacements vs. inter-element overlap in Equation (5.2) can be controlled using λ (default $\lambda = 0.85$). Additional design objectives can be similarly added to Equation (5.2) as quadratic or linear functions of $\vec{x}$. The goal of Equation (5.3) is to ensure that each of the m controls gets mapped to exactly one of the n vertices. $\mathbf{A}$ is thus an $m \times nm$ matrix. Row j of $\mathbf{A}$ thus needs to isolate all variables x_{ij} for $i \in \{1, ..n\}$. Row j of $\mathbf{A}$ is thus all 0's except for n 1's every m elements, starting at index j. Equation (5.4) ensures that each of the n vertices is mapped to by at most one of the m controls. $\mathbf{B}$ is thus an $n \times nm$ matrix. Row i of $\mathbf{B}$ is filled with 0's except for a block of m 1's starting at column $i * n$. Equation (5.5) ensures that all variables in the quadratic program are binary. A quadratic program was found to be significantly faster and space efficient than a linear program that needs nm^2 additional variables to capture the inter-element overlap.

Figure 5.9 further shows how the algorithm parameters can impact the optimal rig UI layout, by varying the contributions of inter-element overlap, and occlusion culling (particularly useful when many parameters are concentrated in local regions of high occlusion like lip corners).

5.4.2 Optimization Pre-processing

This section now discusses four independent grouping and pruning strategies that can radically reduce the number of variables input to the quadratic layout optimization above.

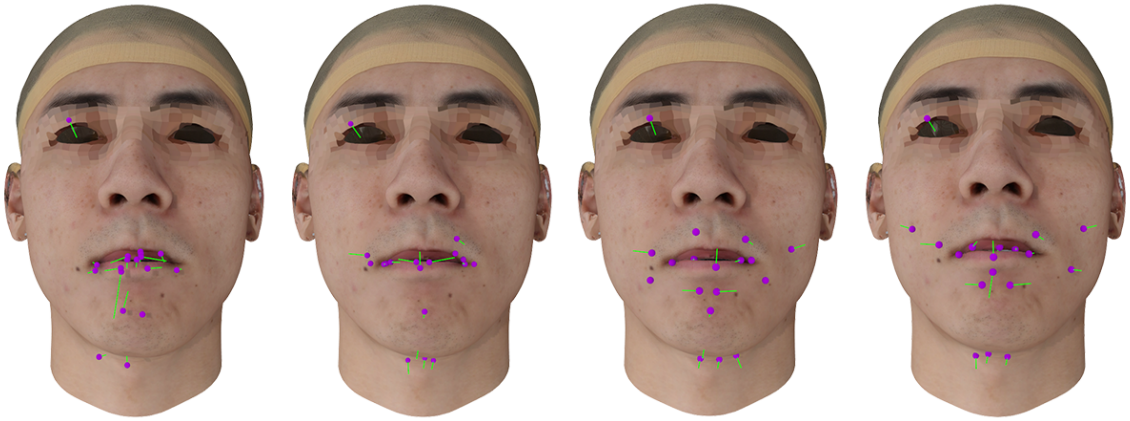


Figure 5.9: FaceScape #393 ©The CITE LAB. Left to right: $\lambda = 1$ (maximal displacement), $\lambda = 0.85$ (default), $\lambda = 0.4$ (higher inter-element penalty), occlusion culling.

Pruning Layout Vertex Candidates The optimization variables x_{ij} consider all vertices v_i as layout mapping candidates l_j for parameter c_j, yet many of these variables would produce undesirable UI elements, and can be omitted from the pool of candidates. The design principles are used based on the following criteria, to produce a pruned set of candidate vertices and associated variables $\vec{x}_{pru}$:

— *Minimally deformed vertices*: Variable x_{ij} is pruned if the length of v_i's deformation trajectory for c_j is below a threshold (default set to the avg. length of all v_i with non-zero trajectories). Deformation controls c_j for which the minimally deformed vertices are a small fraction ($< 10\%$) of model vertices are termed global. Such controls, like the gender control in Figure 5.3(a), are often laid out adjacent to the shape, and can optionally be removed from the optimization.

— *Occluded Vertices*: Variables x_{ij} whose deformation trajectory for parameter j is largely occluded by the model are pruned. All x_{ij} that are invisible from a given view can also be pruned.

— *Symmetric Deformations*: For any parameter j that symmetrically deforms the shape, variables x_{ij} whose vertices v_i do not lie on the plane/axis of symmetry can be pruned away.

Multi-Parameter Grouping Rig parameters that deform very similar regions of the model are often complementary and may benefit from UI-elements like patches and fans (Figure 5.5) that provide concurrent multi-parameter control (Section 5.3.2). To detect possible parameter groupings of size k (for a k-parameter fan, default $k = 2$ for a patch element), the method initializes a set of groups $G = [g_1..g_m]$, associated with each rig parameter $c_1..c_m$ and a vertex set $g_j.v$ that comprises candidate layout vertices for c_j after pruning. Control groups g_i and g_j are then iteratively combined when they together can be represented by a k-parameter fan (i.e. $|g_i \cup g_j| \leq k$), and whose Jacaard index J, where $J = \frac{|g_i.v \cap g_j.v|}{|g_i.v \cup g_j.v|}$ is the largest (i.e. the method picks the i, j pair with he biggest $J > \epsilon$ (default $\epsilon = 0.85$). The two control groups are combined by setting $g_i = g_i \cup g_j$ and removing g_j from G. Likewise, $\vec{x}_{pru}$ is reduced to only retain variables corresponding to vertices $g_i.v \cap g_j.v$ for combined controls $\in (g_i \cup g_j)$. The final set of parameter groups in G, with its corresponding associated variables termed $\vec{x}_{grp}$, can be further reduced by clustering and down-sampling below, or serve as input to the quadratic optimization in Section 5.4.1.

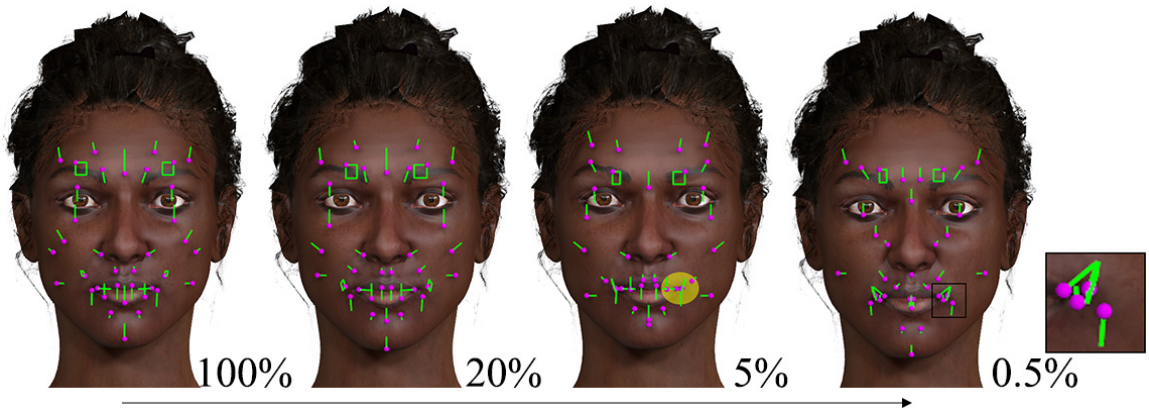


Figure 5.10: UI layouts resulting from weighted random down-sampling of vertices. The UI layout created using 20% of the model vertices is qualitatively similar to the optimal layout (100%). Quality degrades with visual clutter at 5% and inter-element overlap at 0.5% down-sampling.

Rig Parameter Clustering The optimization space of candidate control layouts for n model vertices and m rig controls grows exponentially with m. Many pairs of controls, however, locally deform spatially distinct regions of the model, and their respective UI elements are unlikely to overlap. Partitioning the controls into such spatial clusters that can be independently optimized can thus greatly improve efficiency, with little detriment to the overall UI layout. For example, UI elements for controls that deform the eyes and mouth are spatially distant in Figures 5.1 and 5.11, and could thus be optimized separately.

The implementation uses a simple single-link clustering [110] on the rig control parameters where the distance metric between two clusters of parameters U and V:
$Dist(U,V) = \sum_{c_u \in U} \sum_{c_v \in V} \sum_r \sum_s dist(x_{ru}, x_{sv})$,
where $x_{ru}, x_{sv} \in \vec{x}_{grp}$. In other words the distance between two clusters U, V is the total inter-element distance between all candidate vertices for all pairs of controls belonging to U and V respectively.In practice, replacing the inter-element distance $dist(x_{ru}, x_{sv})$ by the distance between vertices $||v_r - v_s||$ produces similar results with better efficiency. Let $S = \{S_1..S_o\}$ be the resulting clusters that partition the set of rig controls. Consequently, variables $\vec{x}_{grp}$ are partitioned into $\vec{x}^1_{grp}..\vec{x}^o_{grp}$, where $x_{ij} \in \vec{x}^k_{grp}$ iff $c_j \in S_k$.

Figure 5.11 shows the clustering of rig controls for different values of o. A default of $o = 2$ is used to capture the typical upper/lower half split on faces. The UI layout of each cluster of controls is then computed in sequence, starting with the largest cluster. The UI elements computed for former control clusters contribute fixed inter-element distances as constraints for subsequent clusters and thus impact the optimal layout when optimizing latter clusters. As shown in Figure 5.11, increasing the number of clusters can substantially improve efficiency, with minimal impact on the optimality of the control UI layout.

Vertex Down-sampling High-end production models such as Figure 5.12 have upwards of 100K vertices. Rig deformations however, are mostly smooth, and regions of neighboring vertices tend to have similar deformation trajectories. Lower resolution proxies with a fraction of the original shape vertices can produce near optimal UI layouts more efficiently (Figure 5.10). There are a number of ways to down-sample the shape including random vertex sampling and model decimation that retains

correspondence to a subset of the original shape vertices [22]. Unlike typical mesh simplification that optimizes geometric fidelity to the original model, the sample-importance criterion for a vertex favours large displacements (Equation (5.2)). The method performs a weighted random sampling on all $\vec{x}$ variables $\vec{x}^k_{grp}$ in each cluster $k \in\in \{1..o\}$ above. The weights for each variable are associated with the displacement d of its corresponding UI element. The final set of variables down-sampled from $\vec{x}^k_{grp}$ is termed $\vec{x}^k_{dsp}, k \in\in \{1..o\}$. Figure 5.10 shows that 5K+ vertex shapes can be down-sampled 5-10x before there is noticeable visual degradation in the control UI layout.

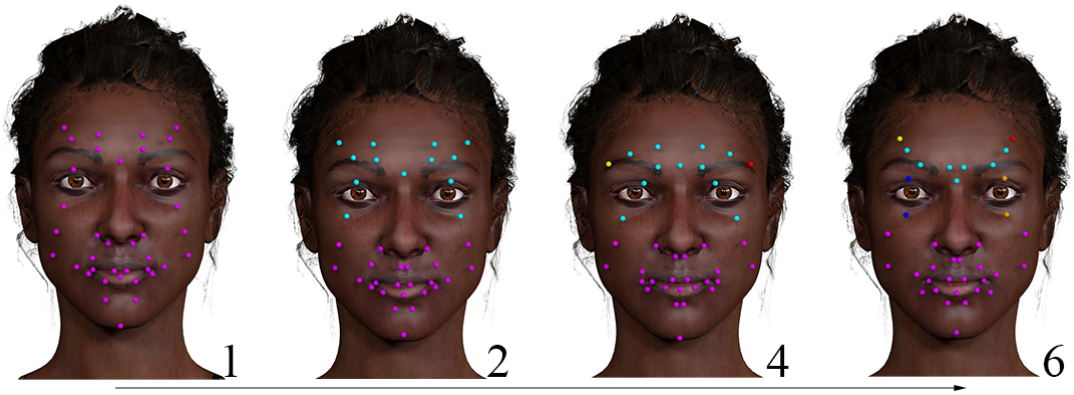


Figure 5.11: Partitioning rig parameters significantly improves performance, while preserving the visual quality of the UI layout. Computation time (seconds) for 1 (164.43), 2 (13.6), 4 (11.41), and 6 (9.95) clusters.

5.4.3 Implementation Details

The approach was implemented using a quadratic programming solver in C++ using IBM CPLEX Optimizer library, and the models were controlled in Autodesk Maya®2020. The results shown in this chapter were generated on a Windows 10 machine with an Intel Core i5 3.6GHz processor and NVIDIA®GeForce GTX 1080.

5.5 User Study

A within subjects user study was conducted with 12 (8 amateur, 4 professional) participants (contacted via email), to evaluate the user experience of the generated UI layouts. All participants had access to and experience with Autodesk Maya®, in which the study was conducted. Participants were identified as amateur/professional, specific to their experience animating 3D face-rigs. While the in-situ layouts are designed to be used with direct manipulation [93] within a single interface (Figure 5.1(e)), the study disabled direct manipulation to focus on a principled comparison of independent parameter control: between a baseline slider *list*, an *off-face* animator created gold-standard layout, and the *on-face* automatic UI layout, for the model in Figure 5.13. The artist layout and the automatic layout could theoretically be interchangeably used as *on* or *off* face layouts.

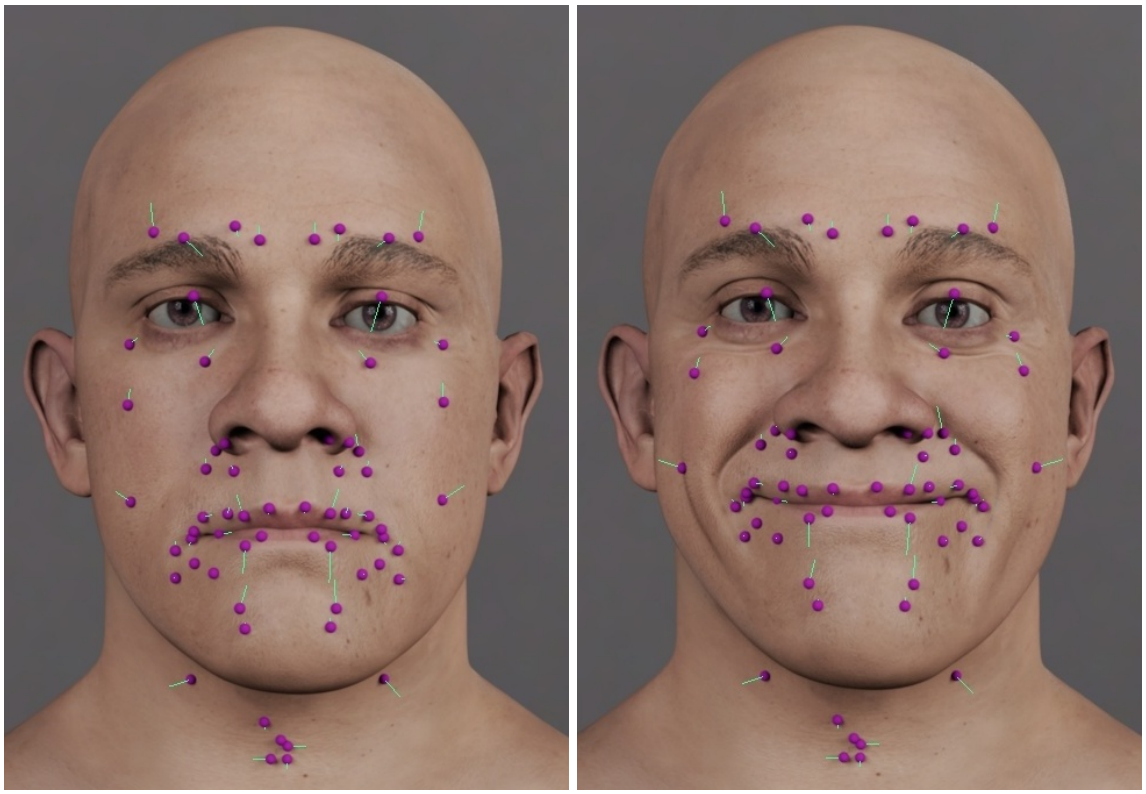


Figure 5.12: UI layout for a 85K vertex, 65 control Allan Henry face-rig ©Weta Digital with neutral (left) and posed parameters (right).

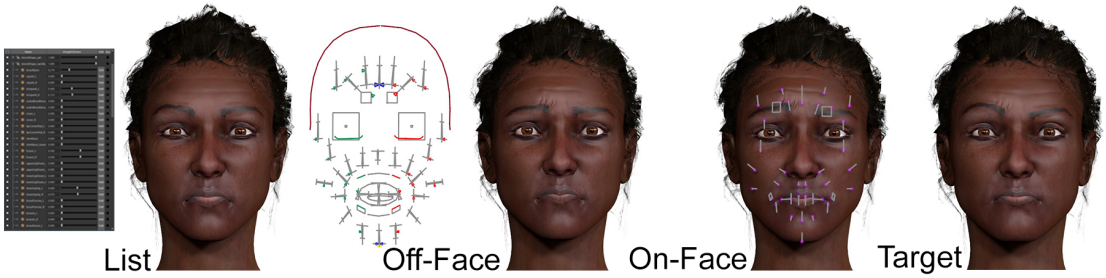


Figure 5.13: Expressions set by user $P8$ using the three compared interfaces, for the common target expression.

5.5.1 Experiment Setup and Protocol

The study presented a subject with a posed target face, adjacent to a neutral face that was interactively manipulated by the subject using one of the above three interfaces, to visually match the target, until satisfied (Figure 5.13). Each participant was presented with the same sequence of 15 faces to match (3 interfaces * 5 faces). The interfaces were tested in order (one of 6 permutations of *list*, *off-face* and *on-face*), fully counter-balanced across the 12 subjects. 12 of the 15 faces were unique and of varying complexity (activating 1-13 rig parameters) spread across the three interfaces tested. One expression (13 activated parameters) was repeated for all the 3 interfaces (Figure 5.13). Participants were presented with the faces side-by-side in a frontal view, but were free to manipulate the camera,

as desired.

The study instructions and testing itself were driven from a UI control panel within Maya. Each interface test started with 2 simple expressions (1-2 rig parameters) followed by 3 more complex expressions (6+ rig parameters). The study was run uncontrolled, and completed on average in 28 minutes by participants remotely in Maya. After the study participants filled-out an online questionnaire on their experience using the three interfaces (Figure 5.14).

5.5.2 Analysis

All subjects had some familiarity with Maya, and were able to complete the study without any reported difficulty. The summary finding was that qualitatively, the overall user experience of the on-screen interfaces *off-face* and *on-face* (both with avg. 4 out of 5) was much better than the slider *list* (avg. 2.25 out of 5) (Figure 5.14(left)). The *on-face* interface also scored highly (avg. 4.83 out of 5) on important questions like "The system gave me a feeling of directly manipulating the face?" (Figure 5.14(right)). Quantitatively, the study data corroborated user response: users were significantly slower with the slider *list* (46.87% of their total time on avg.), compared to the artist created *off-face* (25.62%), and the *on-face* interface (27.51%) (Figure 5.15(right)); while accuracy in matching target faces was similar, since users manipulated the face until satisfied. The *list* was marginally, but not consistently more accurate than the on-screen interfaces, possibly due to finer control resolution of the longer *list* sliders (Figure 5.15(left)).

Quantitative Expression accuracy is measured in both rig parameter space (wt. error sq.) and model space (vtx. error sq.), as the squared difference between the values set by the user and the target expression. For reference, these error values are reported as normalized relative to the difference between the target and neutral expression. In other words, the relative vertex error squared for a user set expression u, target t and neutral n is: $\sum_v(||v_u - v_t||^2)/\sum_v(||v_n - v_t||^2)$. The rig parameter weight error is formulated similarly. Rig parameter weight errors better reflect perceptual changes in expression, but these weight errors can sometimes be misleading due to redundancy in rig parameter space (i.e. a user may create a similar visual expression to the target by using a different rig parameter that produces a similar visual change in expression). Time spent on an interface is also normalized as a $\%_{age}$ of total time taken by a user (Figure 5.15).

Figure 5.13 shows the complex common expression posed in block order (*list*, *on-face*, *off-face*) by user $P8$. The actual time and $\%_{age}$ time spent on each interface in block order was (223s, 119s, 80s) and (17.5%, 9.3%, 6.3%), and the relative weight and vertex sq. errors were (1.0, 1.01, 1.34) and (0.43, 0.47, 0.57), respectively. These statistics might suggest a learning effect with users getting quicker with each block, and that accuracy depends on time spent with the interface. This however, was not a general trend: for eg., the same statistics for $P12$ (*on-face*, *off-face*, *list*) were (146s, 153s, 186s), (9.9%, 10.4%, 12.6%), (0.35, 1.36, 1.45) (0.33, 0.9, 0.62). In general for the common expression, overall participants spent 36% and 12% more time using *list* and *off-face*, than the on-face interface.

Judging, from user comments, the off-screen parameter *list* takes consistently longer to find and manipulate than on-screen controls ("moving back and forth between the face and the slider GUI takes time" $P8$, "slider attribute editor interface does not scale, need to hunt for controls in a scroll list and controls are sometimes not in list close to similar ones" $P6$, "when working with an animated face in a dynamic scene it is better to have controls close at hand", $P7$).

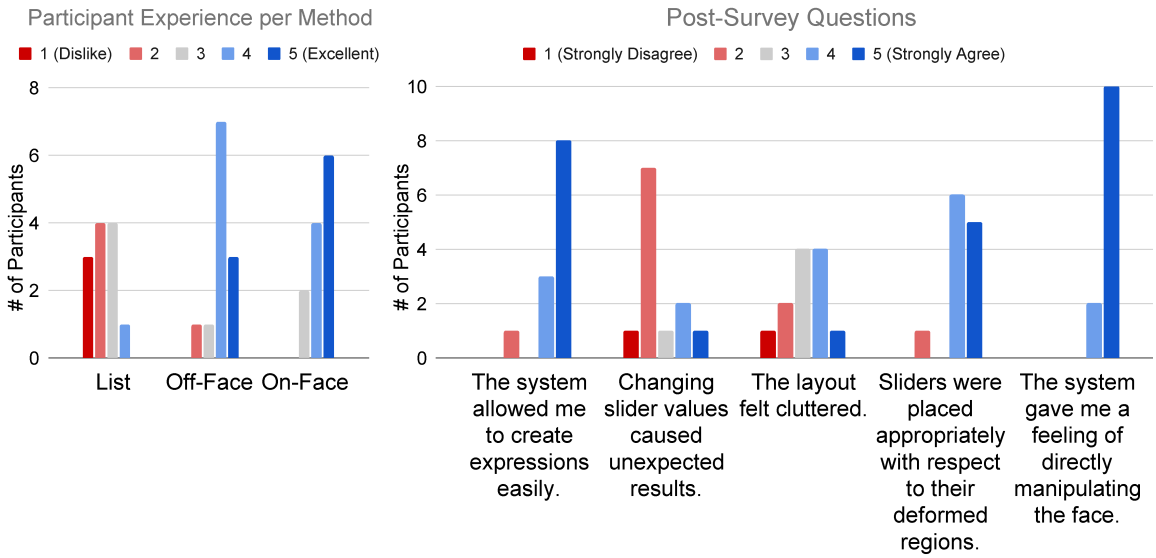


Figure 5.14: User Study: overall experience with the three interfaces (left); questions pertaining to the *on-face* UI (right).

The marginally better accuracy of the slider *list* could be due to resolution ("one advantage of the slider interface is that it does seem to provide finer resolution when interacting with the controls" $P7$). The resolution of on-screen controls can be made finer by zooming into the model but users did not appear to manipulate the camera much ("...this fixed model and camera test setting" $P7$).

Qualitative Figure 5.14(left) shows that the on-face interface was well-received, relative to the *list* and *off-face* interfaces. The responses to questions (below) focused on the on-face interface were also largely positive Figure 5.14(right):

— The system allowed me to create expressions easily.

— Changing slider values caused unexpected results.

— The layout felt cluttered.

— Sliders were placed appropriately with respect to their deformed regions.

— The system gave me a feeling of directly manipulating the face.

The study also produced a wealth of both positive and critical feedback:
General (Positive)

— "I really liked the on-the-face slider a lot." $P4$.

— "Am not an expert on faces but such an interface (on-face) can be very useful for any on-object parameter interaction." $P11$.

— "The on-face patches are very cool and it would be great to see more such coupled controls". "Maybe this grouping can even be dynamic, where I select two or more controls..." $P8$.

Direct Manipulation (Positive)

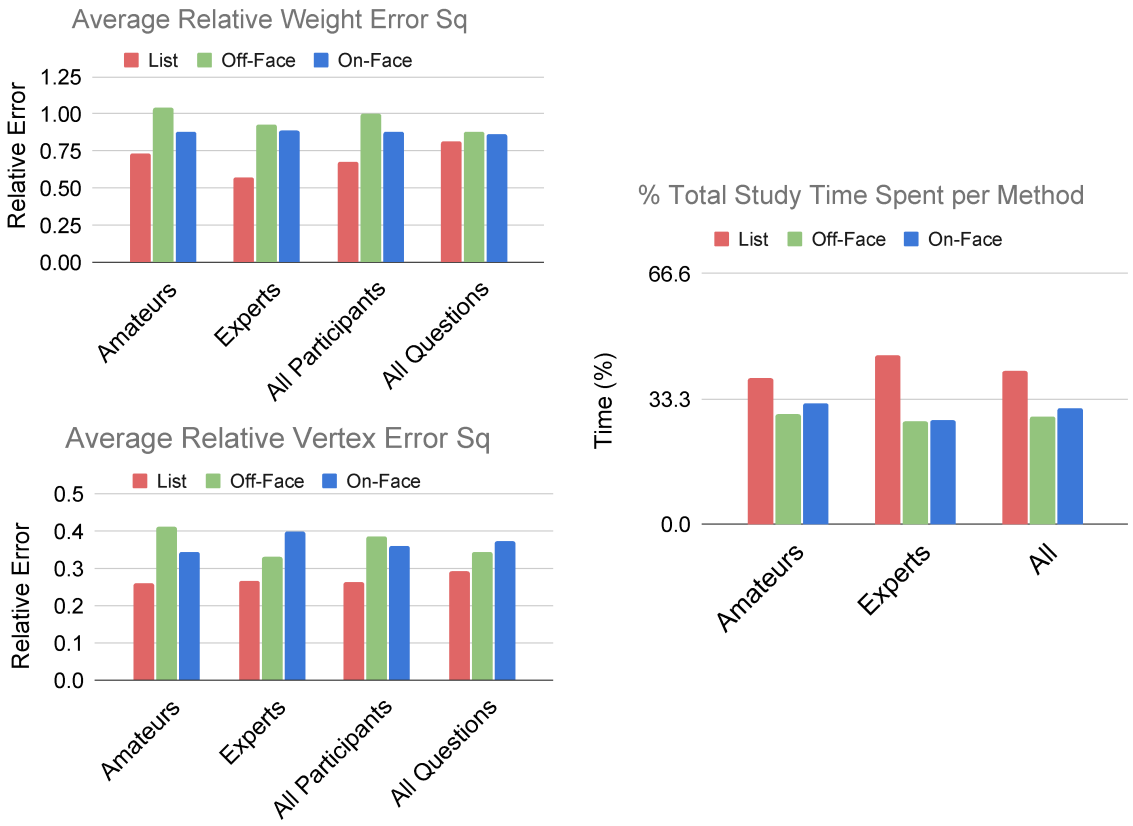


Figure 5.15: User Study: avg. relative weight—vertex sq. error (left top—bottom) and % time the users spent on the three interfaces (right).

— "On-face sliders are great and direct" *P*6.

— "The on-face sliders feels very direct because all the sliders move and stay stuck to the face. It would be cool to augment the on-face slider interaction with directly pulling on parts of the face." *P*7.

— "Precise and local control with direct face grabbing is a problem, and the on-face sliders complement direct manipulation beautifully. I would love to see them work together." *P*8. The layouts are indeed designed and already implemented to be used in conjunction with direct manipulation (Figure 5.1(e)).

Face specific (Critical)

— "It would be useful to show the area of influence when you select the control to give a sense of what area is affected (similar to skin weights map). Or the inverse: clicking anywhere on the mesh would highlight which controls have influence on that area." *P*5.

— "All 3 interfaces could benefit from symmetric control toggle." *P*6.

Model occlusion (Critical)

Many users found that *on-face* visually obscured evaluating the manipulated expression, suggesting off-face transport, and UI element toggles/transparency as solutions:

— "Perhaps working on a duplicate copy of the face will be better? So your manipulations feel direct, but you can still see the actual result well." $P2$ (also $P1$).

— "It would be useful to be able to move the off-face controls on-face and vice versa for the on-face sliders." $P7$.

— "It felt the most natural to use the on-the-face slider layout, but I wish I could toggle the visibility of the sliders in this format." $P3$ (also suggested by $P5$, $P10$ and $P6$ actually "added a toggle to hide/show them").

— "On-face sliders need to be very transparent but if not visually occluding can save time keeping the focus on the face." $P11$. (also $P8$ did likewise "I made the on-face slider shader very transparent so I could grab them and evaluate the facial pose simultaneously.").

5.6 Results and Discussion

Direct Shape—Parameter Control Interface While the user study (Section 5.5) only compared independent parameter control using the *on-face* UI layout with an *off-face* hand-crafted UI layout and a baseline slider *list*, the generated layouts provide a homogeneous interface to complementary direct-shape and direct-parameter manipulation (Figure 5.1(e)).

Gallery of Rig UI layouts Figures 5.1, 5.2 and 5.17 show a number of rigs, for which the approach produced UI layouts that matched the aesthetic and functional choices made by artists in hand-crafting UI layouts for the rig. Figures 5.9, 5.12 and 5.18 show further UI layouts for a number of face rigs for which no hand-crafted comparison was available.

Table 5.1 shows the input vertex and control complexity, algorithmic parameters, and computation times, for all the face-rigs shown in this chapter. While the core optimization algorithm (Section 5.4.1) can be exponential in complexity, the pruning, grouping, clustering and down-sampling stages in Section 5.4.2, make it practical to compute rig UI layouts for complex high-end face rigs.

Animator critique A professional animator was consulted to critique the rig layouts in Figures 5.17 and 5.18. He found the generated rig UI layouts to be compelling, and was easily able to refine the layout aesthetics in Figure 5.1(f). He further commented on various aspects of the problem space, opining that the interface was (or could be):
discoverable and could be manipulated instinctively, without instruction or labels, in contrast to a baseline list of parameter names;
direct, keeping immersive focus on the face, with UI elements that tracked the face during manipulation (Figure 5.4(right));
aesthetic, and produced well-spaced layouts, respecting symmetry and occlusion handling;
localizable, in that the UI layout of a parameter could be forced to lie in proximity to scene element, for eg., the UI element for a joint related rig parameter should be mapped to the shape in the vicinity of the joint Figure 5.16(right) (also see future work);
customizable, in that animators should be able to provide a partial rig UI layout for some parameters and let the algorithm automate the others, which can be handled by simply pre-constraining the variables pertaining to the hand-crafted controls.

General Deformable Rigs Conceptually, the approach is rig agnostic. Shapes can be driven by skeletal skinning, blendShapes, or other geometric/data-driven deformers, as long as rig parameters can be independently sampled to construct the piece-wise linear deformation trajectories of rig vertices, required by the algorithm (Section 5.4). For example, Figure 5.16 (left) illustrates a simple rig UI layout from sampling rotation parameters of the shoulder (green), elbow (red) and wrist (blue) skeletal joints, and Figure 5.16 (right) a pose-based UI element.

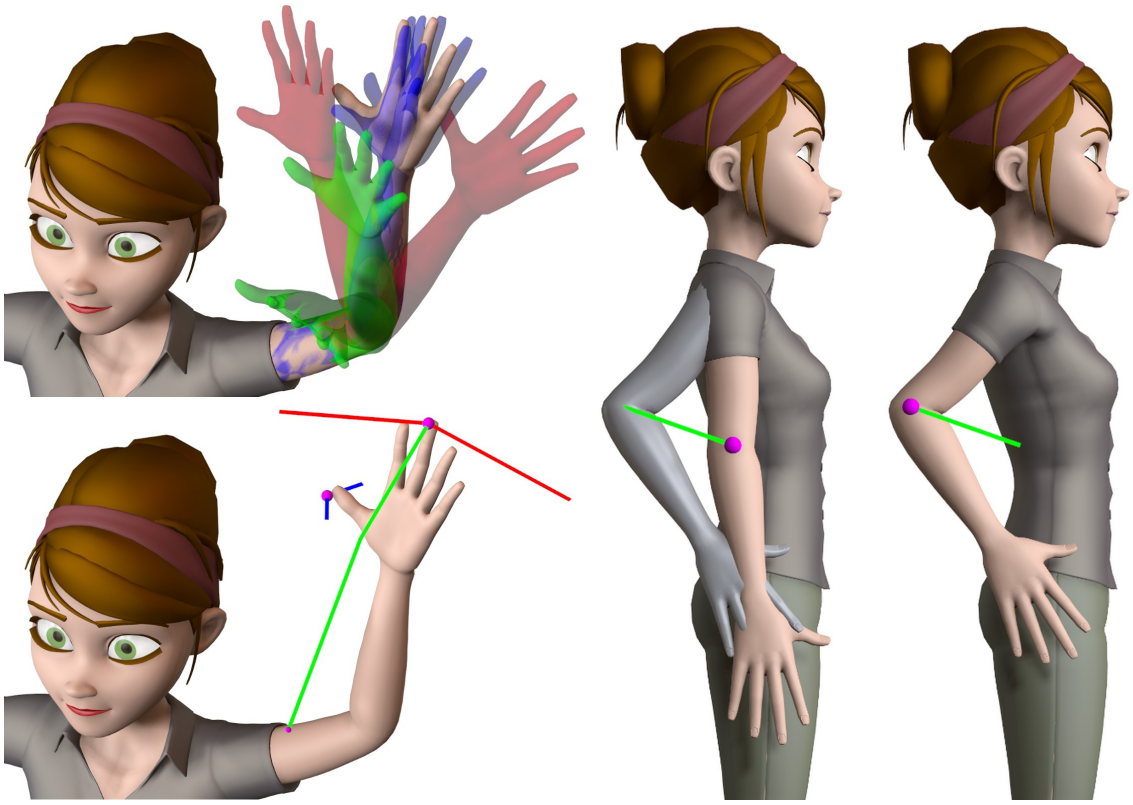


Figure 5.16: Pose-Pose control using rig UI layouts.

Limitations and Future Work Being an early investigation into automated in-situ UI generation for 3D deformable models, this chapter only touches upon some aspects of the problem space, like automating the design of the visual form and function of UI elements/widgets (Section 5.3). The focus of this work, optimizing rig UI layout also has inherent limitations.

— In-production rigs can have highly localized and/or redundant parameters that cause inevitable UI crowding (Figure 5.12). While penalizing inter-element overlap can partly alleviate this problem (Figure 5.9), the problem can also be addressed by dynamically spreading the UI upon interaction, using layouts with diagrammatic arrows [3], or by a multi-level UI.

— Rig parameters controlling subtle deformations, even when mapped to their maximally displaced vertex, may produce tiny UI elements that have poor resolution for interaction.

— Global rig parameters and those controlling largely hidden regions of a face like the tongue, may not be ideally suited to the proposed on-face UI layout.

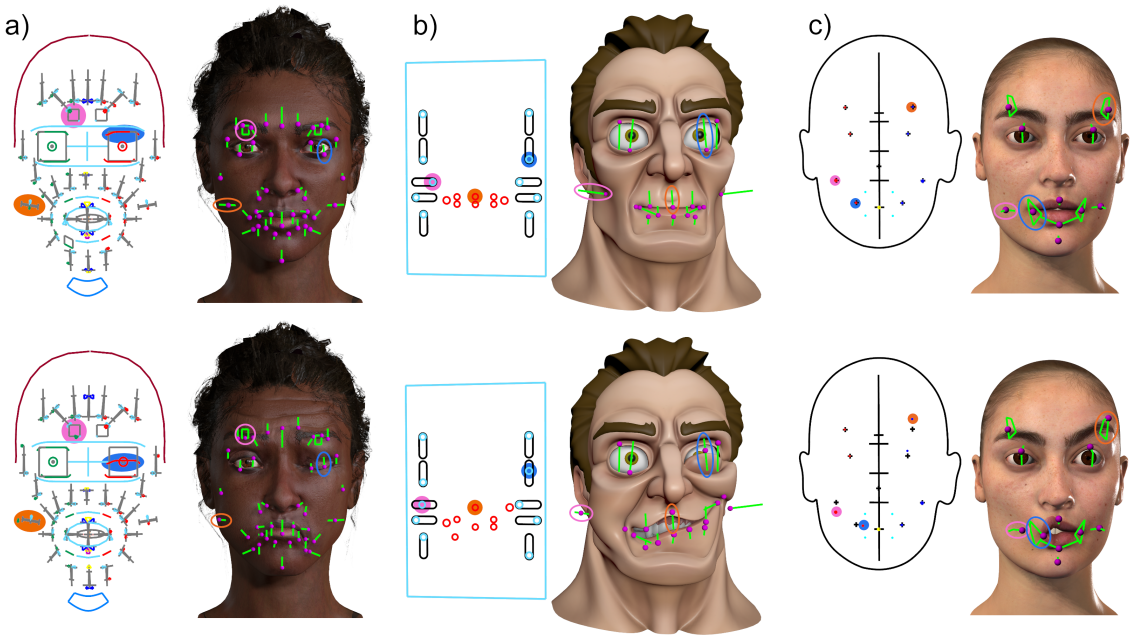


Figure 5.17: Hand-crafted layouts compared to optimized rig UI layouts. The top row shows neutral faces and the bottom row shows posed faces. a) Dakar Valley Girl ©Chris Landreth b) Rig by ©Antony Ward (3D World) c) Animatable Digital Double of Louise by Eisko© (www.eisko.com)

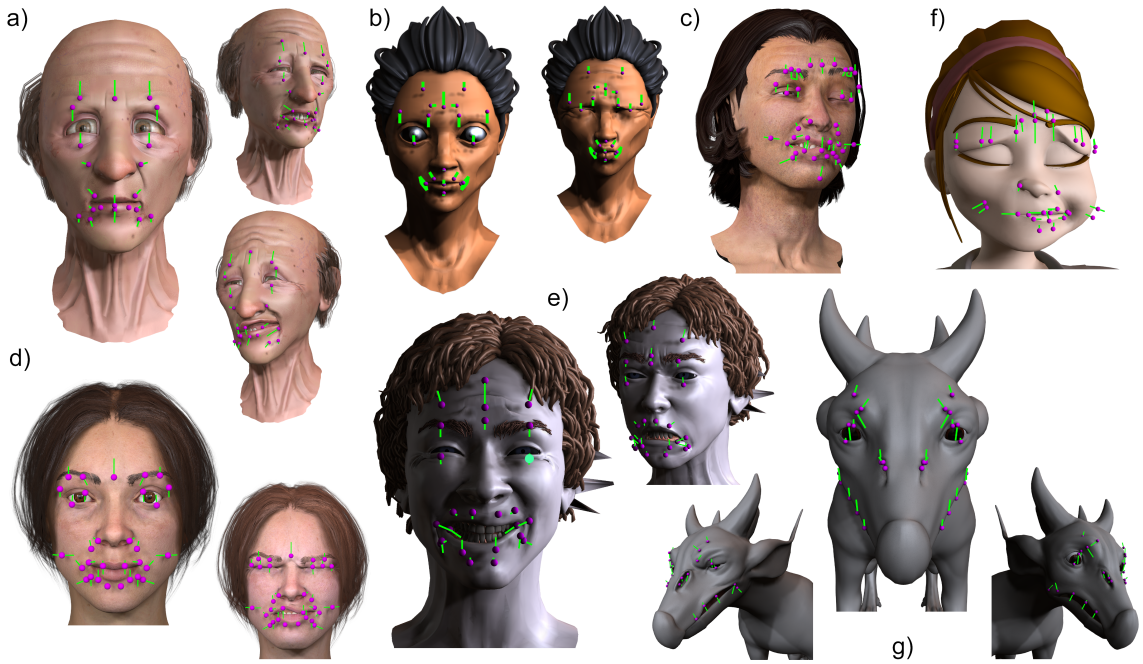


Figure 5.18: Rig UI layouts on models for which no hand-crafted comparison is available. a) Dan b) Angela c) Prague Boy d) Clara e) Pidoras all by ©Chris Landreth f) Mery Project by ©{José Manuel García Alvarez and Antonio Francisco Méndez Lora} g) Nico by ©Chad Vernon (chandmv).

Table 5.1: Shape vertex and control count, algorithmic parameters, and computation times, for all the face-rigs as shown.

Model	Vertex Count	Controls	Clusters	Sample %	**Time (s)**
Dakar Valley Girl	5064	47	2	100/20/5	249/45/11
Head Rig	1186	26	2	20	<1
Louise	5034	28	2	20	3
Mery	12942	28	2	20	3
Nico	27434	28	2	20	19
Dan	5706	23	2	20	62
Clara	5064	45	2	20	15
Prague Boy	5064	47	2	20	18
Pidoras	5064	25	2	20	62
Angela	4002	22	2	20	<1
Allen Henry	85753	65	4	10/5/1	642/243/108
Metahuman	24049	142	4	5/1	553/162
FaceScape #393	16437	20	2	20	210

— Sometimes UI elements enable constrained parameter manipulation such as the focus point cross-hair of an eye-gaze controller. While this chapter does not design such UI elements, artists can hand-craft UI elements for some rig parameters, and use the approach to optimize the remaining rig UI layout, subject to a partial rig UI.

— Some rig parameters, like a sculpting tool deformation radius, are better conveyed by a circular UI element and not a UI element that tracks the deformation trajectory of a vertex of the shape.

There are many avenues for future work. The multi-parameter groupings at present are simple: either user specified, or simply based on overlapping spatial regions of influence. Inferring data-driven (rig + animation data) grouping, correlation, and independence between rig parameters is subject to future work. The user study participants further suggested dynamic parameter groupings, and suggested that the approach can form the building block of a more complex posing system where multiple rig parameters can be posed and controlled in-situ using the generated rig UI layouts (Figure 5.16). Another interesting avenue for research is customizing the UI elements of the rig parameters to be visually representative of the region of the shape they deform, or other rig entities like bones or joints (for eg. the hand-crafted jaw or eyelid UI in Figure 5.1(b)). Finally, while the approach can conceptually define a UI layout for any deformable rig, such as Figure 5.16, the claimed contributions of this chapter to the UI layout of face-rigs, remain to be validated for general deformable rigs.

5.7 Conclusion

Inspired by animator hand-crafted UI layouts, and increasing on-screen UI elements seen in games, mixed reality, and graphics applications, this chapter proposes the novel problem of automating rig parameter UI layouts. It then presents an automated solution to creating such UI layouts for

deformable rigs. The approach seamlessly integrates into current animation pipelines, in game engines and commercial software, like Autodesk Maya. Character rigs form the basis for expressive animation. Automatically streamlining the interface between the rig and the animator may positively impact the throughput and quality of animation produced. Beyond character animation, this work may inspire future research in automatic creation of in-situ user interfaces.

Acknowledgements. I would like to acknowledge Chris Landreth (JALI Research) and Steve Cullingford (Weta Digital) for providing facial rigs, animations, and invaluable feedback. I am also grateful to the user study participants, and to Metahuman (Epic Games), CGTarian, Antony Ward (3D World), CITE LAB, Eisko, Mery Project, and Chad Vernon for face rigs.

Chapter 6

Toward Adaptive In-Situ Workflows

The previous chapters suggest three different ways to access hidden graphical structures using visual context. Chapter 3 analyzed how visible differences from a procedural result can produce in-situ navigation controls for dependency graph structures. Chapter 4 interpreted visual similarities between drawn strokes and scene curves as a way to generate in-situ selection and manipulation controls. Chapter 5 used visual updates from rig parameters as evidence for where to place in-situ controls on a face rig. These systems are different in method, but they suggest a common future direction: graphical tools should let artists start from what they see, then reveal or construct the controls needed to act on that visible intent.

In a more adaptive in-situ workflow, the interface would respond to the user's current scene context, selected region, visible artifact, and recent interaction history to decide what interaction to provide next. More formally, an *adaptive in-situ workflow* is a workflow that dynamically decides which hidden structure to expose and what form of in-situ control to provide.

6.1 Adaptive In-Situ Workflows

Consider an artist working on a complex graphical asset. They notice that a local region of the final output is wrong: in a procedural model, a support beam appears in the wrong place; in a character rig, a facial crease is too strong; in a generated material, a boundary does not follow the intended silhouette. In a traditional workflow, the artist must translate that visible problem into a search through graph interfaces, parameter lists, prompt histories, sliders, layers, or scripts. In an adaptive in-situ workflow, the first step does not have to be a search outside the scene. Instead, the problem can begin from the artifact the artist is already judging. The artist would communicate what they want to change, the system would infer possible controls or structures that could support that change, and the interface would respond by presenting an expressive interaction near the visible problem.

The three thesis contributions each occupy a role in this scenario. The visual-query work supports the question, "where did this visible result come from?" The Squidgets work supports the question, "how would I like this visible feature to change?" The face-rig UI work supports the question, "which known controls should be available here?" Together, these works point toward workflows where navigation, editing, and control layout are no longer separated from the artifact being judged but blended into the same visual context.

Of course, this thesis does not claim that the three techniques presented in Chapters 3–5, by themselves, produce a perfect adaptive in-situ workflow. Each technique is limited in scope, method, and application domain, and many other questions remain open. But imagine a workflow that could apply the same principles to a wider range of graphical systems, a more flexible set of in-situ interactions, and a more robust inference of user intent. The value of this workflow is not simply that more interface elements appear inside the viewport. The value is that the user can preserve attention on the artifact while minimizing movement between understanding, control discovery, editing, and refinement.

This does not mean that all interaction should move into the scene. As referenced in Chapter 1, the thesis does not claim that in-situ direct manipulation is the only way to interact with parameterized scenes. Spatial anchoring and locality principles from in-situ interaction, along with expressive effort and mapping complexity from direct manipulation, suggest that not all commands or tasks should be in-situ or direct. But when in-situ and direct manipulation do fit, they can provide a more immediate, contextual, and visible way to access hidden structure.

6.1.1 Context Disambiguation

To achieve the adaptive in-situ workflow described above, one problem that the thesis did not fully address is the problem of tool selection, or context activation; in all three works, the thesis assumed the user had already entered the context of the created tool to perform operations through writing a command or pressing a tool button. Context disambiguation in an adaptive in-situ workflow would determine which tool to enable to provide a seamless transition between tools and workflow stages. The problem is having the system determine which interaction context the user is trying to enter and then activating the right tool and expose the right in-situ controls for that context. Without context disambiguation, the interface (such as the mouse or controller) will be overloaded with multiple tools fighting for the same interface with no ability to select or differentiate.

Visual evidence alone cannot fully determine the user's intended context. Squidgets used visual similarity to infer which curves the user wanted to select, but could not determine whether to perform a translation, rotation, scale, or deformation; the user had to pre-select the tool to disambiguate the intended operation. Many operations for in-situ and direct manipulation can have similar visual effects from different intended operations caused by entering different contexts. Operations can be regularized to provide a controlled combination of operations, but artists often desire predictive interactions from clearly defined contexts, and the system should support that when possible. In fact, many buttons and menu operations in WIMP interfaces are UI elements to enter specific contexts, and the user must first select the context before performing an action. This tool selection is very clear and transparent to the user.

Context disambiguation also applies not only to tool selection but to parameter selection as well. Many entities can have parameters with shared visible consequences as mentioned earlier, but many parameters may also be non-visible. Context selection can also decide how to handle non-visible parameters such as the structural integrity of a chair, the stability of a building, the manufacturability of a design, or the semantic style of an object. The thesis focuses on visible evidence to infer the relevant hidden structures to alter, but some properties are not visible for in-situ anchoring or direct manipulation. Determining which parameters to manipulate (visible or not) can be done before handle creation to limit the possible space of interactions to provide, and this is typically done

through the detached interfaces.

This does not invalidate the thesis argument, but it qualifies it. In-situ direct manipulation is strongest when the relevant context is known and the intended change has visible, local, and manipulable evidence. When context is ambiguous or the target parameter is abstract, an adaptive workflow must first help the user enter the correct context, then present the appropriate in-situ handle. Traditional interfaces solve this problem through explicit detached controls: toolbars, menus, hotkeys, search, and selection handles. These mechanisms make the active interaction context visible, but they also reintroduce translation costs.

A future adaptive workflow could use a multi-modal approach: language, edit history, recent behavior, scene context, and available software operations, to infer the intended context more directly. Recent advances in AI have started to hint at this possible future.

6.2 In-Situ AI Workflows

6.2.1 Prompt-Based Workflows and Their Limits

Prompt-based AI tools have made graphical creation and editing more accessible. Users can generate and edit images, designs, prototypes, and graphical assets from text prompts, reference images, and other inputs [44, 52, 116]. Without first mastering every tool, parameter, or graph structure in a domain-specific application, users can describe what they want and let the system produce a result. A prompt such as “make the room feel colder” or “generate variations of this chair in a mid-century style” can be a useful way to explore a design direction. This interaction style has made prompts effective for ideation and exploration.

Prompting addresses the problem of navigating complex controls in a different way than in-situ direct manipulation. It lets users express intent without manually locating every control, passing the task to an AI agent that can explore, plan, and execute the right sequence of edits. Relieving users from researching software capabilities and translating intent into specific operations can make graphical creation more accessible. But this method hides editing structures behind a black box and proves difficult to steer after the first generation.

The hidden-control problem becomes clearest during refinement. AI systems can produce useful first drafts amazingly well - from a blank canvas, a well written prompt could generate all the figures in this thesis, but the first draft is rarely the final one. Artists often want to make small incremental changes to a generated result to arrive closer to the vision, but the prompt-only workflow can make this difficult; if an artist wants to make a small change to a generated result, they must again translate that change into language, wait for the system to interpret it, and judge the new result. With current models, this can turn into iterative prompting: an unpredictable cycle of describing, regenerating, judging, and redescribing. Change may struggle to stay localized: a small prompt change may alter unrelated parts of the image, scene, or design, leaving the artist unsure whether the issue was the wording, the missing context, or the model’s interpretation.

This issue is not a problem with AI models per se, but a problem with the text modality. Language loses precision when it becomes the only channel for specifying situated visual changes. The artist may know exactly which region, expression, timing, or material detail should change, but the prompt must compress that visual context into text. The result then depends on the model’s ability to

recover the missing context and infer the artist's intention. It must also preserve everything that was not explicitly described and choose the right degree of change. When the model misunderstands, iterative prompting gives the artist little fine-grained control over the correction. Prior work has also shown that prompt-based interactions can be brittle when users must translate intent into effective wording [165].

The issue is not that prompts are unsuitable for graphical work - clearly, they are useful for broad intent and exploration. The issue is that prompt-only refinement lacks the visual grounding, immediate feedback, and manipulable constraints needed for precise local edits. However, reaching into the software operations, editable parameters, or hidden scene structures behind an AI-generated result and exposing them as interactable controls may provide a more direct way to steer results.

6.2.2 From Prompting to In-Situ Direct Manipulation

Recent research has begun to pair AI with direct manipulation and spatial context. DirectGPT shows how language-model outputs can be made selectable and manipulable for localized direct prompting [100]. Curompt grounds prompts in a spatial 3D design canvas through selection and cursor context [47]. These systems illustrate how visual, situated, and manipulable interactions can support prompting in graphics creation.

For our context, the important shift is from using AI only to generate artifacts to using AI to generate or expose controls. These interactions could appear after initial prompting, where the agent not only performs an update but also exposes meaningful interface elements for further refinement. The output of an AI interaction could be an interface element, an inferred handle, or a temporary interaction that lets the user continue working directly on the visible artifact after the initial generation. The central opportunity is therefore not only to use visible evidence as the thesis suggests in prior chapters, but to additionally tie language context to visible evidence to guide the generation of these elements.

A language prompt can establish missing constraints that visual evidence alone cannot. Let's use the prompt "Make the leaves on this tree fatter and more moss green". To create an in-situ control for this prompt, we can use the subject tokens "leaves" and "tree" to establish the visual anchor for an in-situ interaction ("this" token can further disambiguate which tree to edit if multiple trees are present; the visual querying interaction can disambiguate which tree more precisely). The adjectives "fatter" and "moss green" along with inverse rendering can guide the system toward finding the leaf geometry and material parameters that the interaction should be mapped to. With the subject and adjectives, the system already has strong visual design constraints for anchoring the in-situ interaction and choosing which parameters to map.

One powerful aspect of language that the scene alone cannot provide is that it can estimate the desired post-prompt visual effect. Describing the desired change using the adjectives "fatter" and "moss green" not only identifies which parameters to drive, but also provides a prediction of the visual change the user wants to achieve. Analysis of the pre-edit and post-edit visible difference can inform a direct manipulation interaction that supports the intended effect. This analysis of visual differences was explored in Chapter 5 for rig parameters and Chapter 3 for construction-history states, and estimated in Chapter 4 from curve visual differences. Other motion-analysis methods, such as optical flow, can further suggest direct manipulation trajectories for the in-situ interaction to adopt [12].

This example works because the prompt requests a specific local change to a small set of parameters

of the visible entities. However, the weaknesses of in-situ and direct manipulation principles still apply. A prompt requesting large global changes such as "Make the whole scene look more cartoonish" or creating or removing entities such as "Add a tree to the left of the house" may not have either a clear visual anchor for an in-situ interaction or a clear gesture for direct manipulation. These larger, one-shot edits have benefited from gallery-based interfaces that expose alternatives for comparison, curation, and iteration [96]. Language can also lack precise cultural, social, and domain-specific context, which can lead to incorrect inferences about which entities to edit and how to edit them. Visible selection and interaction can provide additional context to disambiguate the user's intent.

6.2.3 Parameterized Scenes and Latent Representations

AI systems have shown strong ability to learn non-linear mappings in large latent spaces. Thus, AI can learn complex mappings between direct gesture and parameter change to create intended visual effects, but this raises a question of whether parameterized scenes need to be explicitly defined at all. Latent and neural representations have found success representing complex visual phenomena such as images and entire scenes [105, 124]. Learned representations can be more flexible and powerful than explicit ones because they encode complex relationships and constraints that would be difficult to handcraft into large parameter spaces.

Editing methods have shown that meaningful visual changes can sometimes be produced by moving along discovered directions in a model's latent space [58]. However, an in-situ interaction that follows direct manipulation principles requires a mapping between the space of all gesture trajectories and the space of all parameter changes, and this mapping must be consistent, predictable, and invertible. This requires the mapping between gesture and parameter change to be predictable and locally consistent, and the latent space must be structured enough to support that behavior. A gesture that moves a handle to the right should always produce the same change, and a change should always be reachable by some gesture; a gesture that then moves the handle back to the left should undo that change. Throughout this edit, the visual effect should also follow the direct gesture visually, so the mapping must also be designed to produce a predictable visual change for a given gesture.

The AI could learn this mapping from a large dataset of gesture-parameter pairs, but another way could be to reparameterize the latent space to be more linear and interpretable. A jungle scene represented using a diffusion latent representation could be reparameterized with AI to have explicit parameters for leaf size, leaf color, tree density, and so on, which would then be mapped to the latent space in a way that supports linear mappings between gesture and parameter change. Work on disentangled representation learning and interpretable latent controls suggests partial versions of this direction, but these controls are typically local or model-specific rather than guaranteed reusable parameters [58, 63]. Learning a simplified set of explicit parameters can help with language grounding as well, since the parameters can be directly tied to the adjectives and nouns in the prompt. This reparameterization would simplify the mapping between in-situ interaction and parameter change, making it more predictable and controllable for the user.

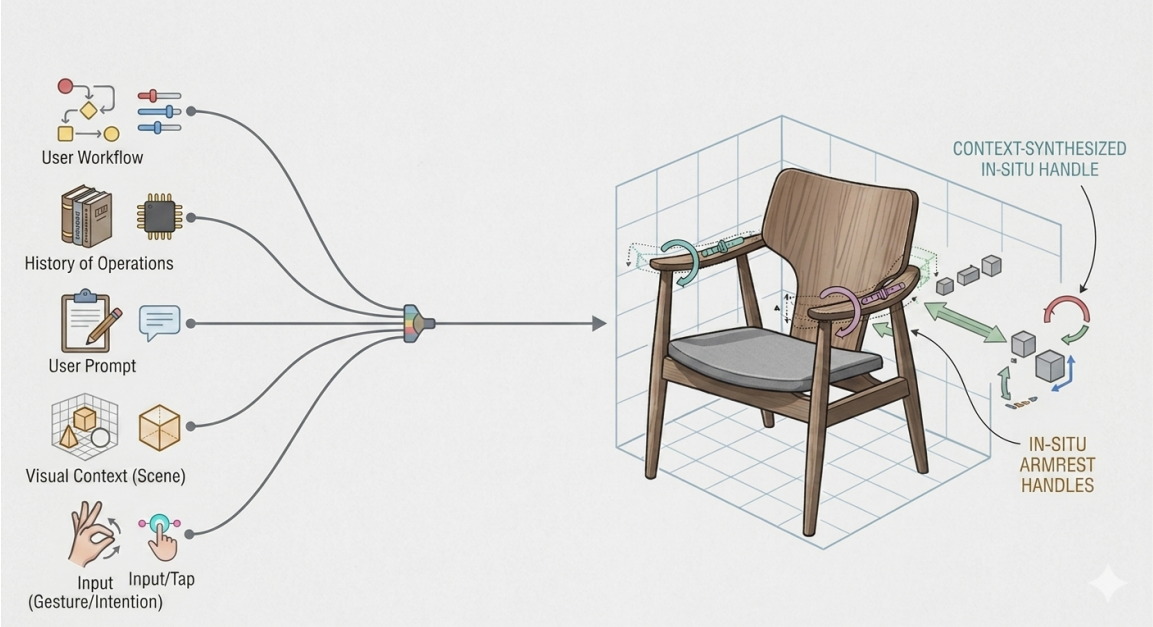


Figure 6.1: Multi-modal contexts from user behavior, operation history, language prompt, visual context, and gestures can drive the creation of in-situ interactions. This image was created with AI.

6.2.4 AI Support for Adaptive In-Situ Workflows

In an adaptive in-situ workflow, a system could consider multi-modal input from a language prompt, visual context, scene structure, edit history, and available software operations to infer in real time which hidden structure matters and what in-situ tool should expose it. By including more context from artist behavior, the system can make more informed decisions about which interactions to preload without explicit artist prompting. Recent work on next action prediction similarly frames proactive AI as predicting a user's future actions from multimodal interaction histories [134]. This can help resolve the context-selection problem of deciding which in-situ interaction to provide, which is a key question for adaptive in-situ workflows.

This framing connects directly to the three works presented in this thesis, which already demonstrate non-AI versions of this visible-evidence-to-control mapping. For example, consider an AI-generated jungle scene represented by a diffusion latent representation, where the artist has a visual question about how the cliffs were formed. The artist could highlight the cliff region and prompt the AI to generate an interpretable construction history of the cliff's formation. The AI could reparameterize the latent space to have explicit construction-history states and explicit parameters for each state that represent the important visual features of the cliff at that point. The AI would create in-situ visual handles that mark important visual steps in the cliff's formation, which the artist could navigate using scrubbing gestures. For each state of the cliff, in-situ handles for the parameters could appear locally wherever the artist hovers, allowing them to alter the cliff's visual features at that state through the defined parameters. For parameters that do not exist, the artist could draw Squidgets to create custom handles that map to the visual features they want to change and have the AI determine the mapping between those visual features and the latent parameters. As the artist works on the cliff's shape, the system may infer from the edit history that the artist is

focused on geometry deformation tasks and preload in-situ handles and Squidget curves that pertain to geometry deformation.

This raises a broader question about the role of specialized interaction methods in a future shaped by increasingly general AI models. Sutton's "Bitter Lesson" argues that methods built from human knowledge can be effective in the short term, but methods that scale with computation tend to dominate in the long term [148]. From this lens, the techniques in this thesis can be seen as specialized methods for particular graphical representations: visual querying construction histories, sketching perceived scene curves, and dragging face-rig parameters. A sufficiently capable model might eventually learn parts of these mappings directly, such as which construction states matter for a selected region, which scene feature a stroke refers to, or where rig controls should appear. The model may understand user workflow behaviors, scene context, gesture inputs, and inspect even more data. However, this does not remove the interaction problem. Even if the underlying mapping is learned by a general model rather than engineered by a specialized algorithm, the user still needs to understand the interaction rules. The long-term opportunity is therefore not only to replace specialized inference methods with larger models, but to use those models to produce usable in-situ interactions that guide user's abilities with the in-situ and direct manipulation tools.

This thesis has not fully developed such an AI-driven interface, but the proof-of-concept direction is clear from the preceding systems. A tool could treat an AI response as the beginning of an interaction rather than the end of one: after the system proposes an edit, it could expose the region, parameter group, construction step, or generated handle that best supports refining the edit. The artist would then continue through situated controls instead of repeatedly rewriting the prompt. In this framing, AI acts less like a one-shot generator and more like an assistant for proposing temporary, local, and correctable control structures.

Chapter 7

Conclusion

One of the clearest examples of the feeling I care about in this thesis is from a video game I played as a kid. It is the Face-Lift minigame from *Mario Party*. A character's face is shown in a strange expression, and the goal of the minigame is to pull directly on the character's face to match the example [143]. There is no separate panel for the cheeks, mouth, nose, or eyebrows, no need to search for the right parameter name, and no separate tool acting on the face from somewhere else. The player is working on the face.

What is fascinating about this minigame is that it is a game, not a creative tool. You look at the face, see what is wrong, and pull on the face itself. That interaction is fun and playful. Imagine if the minigame had been designed with a more traditional interface: clicking on buttons, sliders, or other options that change the face. I imagine it would be less fun, less playful, and more likely to interrupt flow.

That feeling is what stayed with me across this thesis: flow. Throughout the thesis, I have described the work in terms of in-situ interaction, direct manipulation, hidden parameters, and graphical representations. Those are still the right technical terms, but after building these systems, the experience I keep returning to is creative flow. I mean the state where the user's attention can stay with the thing they are making. The tool recedes a little, and the user can focus on the creative task instead of the software.

When an interface is indirect, part of the user's mind has to stay with the tool. Which control matters? Where is it? What is it called? How does this slider, node, prompt, or panel map back to the thing on screen? Those questions are sometimes necessary, but they pull attention away from the canvas. When the interaction begins in the scene, more of that attention can stay with the creation itself: this cheek, this curve, this region, this deformation, this visible mistake.

The three projects in this thesis each tried to create that kind of closeness in a different way. Visual querying lets a user ask a question from the rendered result instead of first searching through a construction history. Squidgets let sketches become handles in the scene instead of only marks drawn on top of it. The face-rig layout work puts controls where their effects are visible instead of leaving every control in a detached list. These systems are different, but they share the same goal: support creative flow by reducing the distance between seeing, deciding, and changing.

This is hard to do well. There are good reasons that many modern tools reuse familiar interface patterns: panels, sliders, text boxes, and graph interfaces are understandable, easy to explain, and

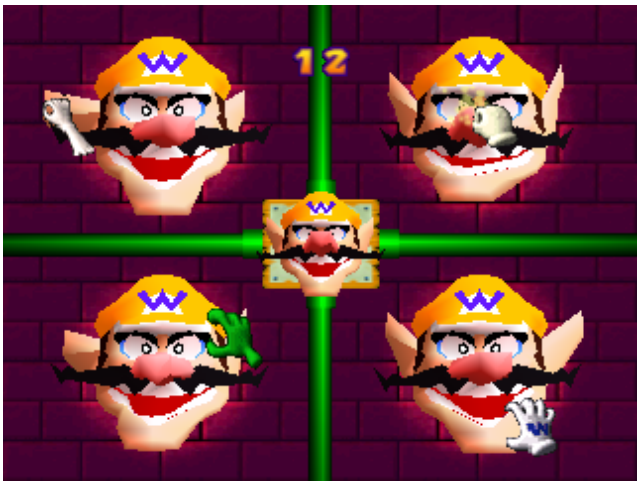


Figure 7.1: The Face-Lift minigame from *Mario Party 2*, where players pull and tug a character's face to match a target expression [144].

often the simplest way to give users access to a large number of parameters. In many cases, they are the right choice, but that does not mean they are the only choice. When we focus too much on familiarity, we can miss opportunities to imagine something new.

For visual creation, that matters. It is possible to build a tool around the representation that is easiest for the system rather than the representation that is closest to the user's creative judgment. This can remove the advantage of working within a visual medium in the first place. If we see something in the scene, but have to look somewhere else to change it, we are losing the opportunity to work with the thing we see. Sometimes that distance is acceptable or unavoidable. But it should not be the default.

The harder path is to ask what interaction could look like if it began from the artifact itself. That path is messier. It requires dealing with ambiguity, clutter, hidden structure, and cases where direct manipulation is not enough. It often means building interactions that are harder to explain at first because they do not look like the interfaces people already know. But it is also where some of the most meaningful interaction design work can happen. New interactions can change not only what a tool can do, but what creating with the tool feels like.

Visual creation deserves more of that effort. It should be playful, expressive, and exploratory, not a workflow of filling out forms on the side, searching through tabs of tools, or chatting with an AI assistant. When in-situ tools work, digital creation feels closer to the real thing. The user stays in flow. The interaction becomes more focused, more connected, and more fun. That kind of flow is not just a luxury. It is part of what makes creative tools worth caring about. The systems in this thesis are small steps toward that feeling, but they point to a larger direction for graphics interfaces: tools should invite people into the scene they are shaping instead of repeatedly pulling them away from it.

The scene you see should not only be where computation ends - it should be the place where control can begin.

Bibliography

[1] Rinat Abdrashitov, Fanny Chevalier, and Karan Singh. "Interactive Exploration and Refinement of Facial Expression Using Manifold Learning". In: *Proceedings of the 33rd Annual ACM Symposium on User Interface Software and Technology*. UIST '20. Virtual Event, USA: Association for Computing Machinery, 2020, pp. 778–790. ISBN: 9781450375146. DOI: 10.1145/3379337.3415877. URL: https://doi.org/10.1145/3379337.3415877.

[2] Radhakrishna Achanta et al. "SLIC Superpixels Compared to State-of-the-Art Superpixel Methods". In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 34.11 (2012), pp. 2274–2282. DOI: 10.1109/TPAMI.2012.120.

[3] Maneesh Agrawala, Wilmot Li, and Floraine Berthouzoz. "Design Principles for Visual Communication". In: *Commun. ACM* 54.4 (Apr. 2011), pp. 60–69. ISSN: 0001-0782. DOI: 10.1145/1924421.1924439. URL: https://doi.org/10.1145/1924421.1924439.

[4] Christopher Ahlberg, Christopher Williamson, and Ben Shneiderman. "Dynamic Queries for Information Exploration: An Implementation and Evaluation". In: *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*. CHI '92. ACM, 1992, pp. 619–626. DOI: 10.1145/142750.143082.

[5] Eric Allen and Kelly L. Murdock. *Body Language: Advanced 3D Character Rigging*. Pap/Cdr. USA: SYBEX Inc., 2008. ISBN: 0470173874.

[6] Georg Apitz and Françis Guimbretière. "CrossY: a crossing-based drawing application". In: *Proc. UIST '04*. Ed. by Steven Feiner and James A. Landay. ACM, 2004, pp. 3–12. DOI: 10.1145/1029632.1029635. URL: https://doi.org/10.1145/1029632.1029635.

[7] Andreas Aristidou et al. "Inverse Kinematics Techniques in Computer Graphics: A Survey". In: *Computer Graphics Forum* 37 (Sept. 2018), pp. 35–58. DOI: 10.1111/cgf.13310.

[8] Rahul Arora et al. "SketchSoup: Exploratory Ideation Using Design Sketches". In: *Computer Graphics Forum*. Vol. 36. 8. 2017, pp. 302–312.

[9] Norman I Badler, Brian A Barsky, and David Zeltzer. *Making Them Move: Mechanics, Control & Animation of Articulated Figures*. Routledge, 1990.

[10] S. H. Bae, Ravin Balakrishnan, and Karan Singh. "ILoveSketch: As-Natural-As-Possible Sketching System for Creating 3D Curve Models". In: *Proceedings of the 21st Annual ACM Symposium on User Interface Software and Technology*. UIST '08. ACM, 2008. ISBN: 9781595939753.

[11] Stephen W. Bailey et al. "Fast and Deep Facial Deformations". In: *ACM Trans. Graph.* 39.4 (July 2020). ISSN: 0730-0301. DOI: 10.1145/3386569.3392397. URL: https://doi.org/10.1145/3386569.3392397.

[12] John L. Barron, David J. Fleet, and Steven S. Beauchemin. "Performance of Optical Flow Techniques". In: *International Journal of Computer Vision* 12.1 (1994), pp. 43–77. DOI: 10.1007/BF01420984.

[13] Michel Beaudouin-Lafon. "Instrumental Interaction: An Interaction Model for Designing Post-WIMP User Interfaces". In: *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*. CHI '00. Association for Computing Machinery, 2000, pp. 446–453. DOI: 10.1145/332040.332473.

[14] Pierre Bénard and Aaron Hertzmann. "Line Drawings from 3D Models: A Tutorial". In: *Found. Trends. Comput. Graph. Vis.* 11.1–2 (Sept. 2019), pp. 1–159. ISSN: 1572-2740. DOI: 10.1561/0600000075. URL: https://doi.org/10.1561/0600000075.

[15] Lonni Besançon et al. "Mouse, Tactile, and Tangible Input for 3D Manipulation". In: *Proc. CHI '17*. CHI '17. Denver, Colorado, USA: Association for Computing Machinery, 2017, pp. 4727–4740. ISBN: 9781450346559. DOI: 10.1145/3025453.3025863. URL: https://doi.org/10.1145/3025453.3025863.

[16] Sukanya Bhattacharjee and Parag Chaudhuri. "A Survey on Sketch Based Content Creation: from the Desktop to Virtual and Augmented Reality". In: *Computer Graphics Forum* (2020).

[17] E. A. Bier. "Skitters and Jacks: Interactive 3D Positioning Tools". In: *Proc. I3D 86*. 1986, pp. 183–196.

[18] E. A. Bier. "Snap-Dragging in Three Dimensions". In: *Proc. I3D 1990*. 1990, pp. 193–204.

[19] Eric A. Bier et al. "Toolglass and Magic Lenses: The See-Through Interface". In: *Proceedings of the 20th Annual Conference on Computer Graphics and Interactive Techniques*. SIGGRAPH '93. ACM, 1993, pp. 73–80. DOI: 10.1145/166117.166126.

[20] Eric Allan Bier. "Skitters and jacks: interactive 3D positioning tools". In: *Proceedings of the 1986 workshop on Interactive 3D graphics*. ACM. 1987, pp. 183–196.

[21] P. Blair. *Advanced animation*. "How to draw" library. Foster, 1949. ISBN: 9785879740356. URL: https://books.google.ca/books?id=kvIQAwAAQBAJ.

[22] Mario Botsch et al. *Polygon mesh processing*. CRC press, 2010.

[23] Doug A. Bowman et al. *3D User Interfaces: Theory and Practice*. USA: Addison Wesley Longman Publishing Co., Inc., 2004. ISBN: 0201758679.

[24] Eric Brochu, Tyson Brochu, and Nando de Freitas. "A Bayesian interactive optimization approach to procedural animation design". In: *Proceedings of the 2010 ACM SIGGRAPH/Eurographics Symposium on Computer Animation*. Eurographics Association. 2010, pp. 103–112.

[25] Steven P. Callahan et al. "VisTrails: Visualization Meets Data Management". In: *Proceedings of the 2006 ACM SIGMOD International Conference on Management of Data*. SIGMOD '06. ACM, 2006, pp. 745–747. DOI: 10.1145/1142473.1142574.

[26] Chad Carson et al. "Blobworld: A System for Region-Based Image Indexing and Retrieval". In: *Visual Information and Information Systems*. Springer, 1999, pp. 509–516. DOI: 10.1007/3-540-48762-X_63.

[27] Ozan Cetinaslan and Verónica Orvalho. "Direct Manipulation of Blendshapes Using a Sketch-Based Interface". In: *Proceedings of the 23rd International ACM Conference on 3D Web Technology*. Web3D '18. Poznań, Poland: Association for Computing Machinery, 2018. ISBN: 9781450358002. DOI: 10.1145/3208806.3208811. URL: https://doi.org/10.1145/3208806.3208811.

[28] Paul Chandler and John Sweller. "The Split-Attention Effect as a Factor in the Design of Instruction". In: *British Journal of Educational Psychology* 62.2 (1992), pp. 233–246. DOI: 10.1111/j.2044-8279.1992.tb01017.x.

[29] Siddhartha Chaudhuri and Vladlen Koltun. "Data-Driven Suggestions for Creativity Support in 3D Modeling". In: *ACM Transactions on Graphics* 29.6 (2010). DOI: 10.1145/1882261.1866205.

[30] Byungkuk Choi et al. "SketchiMo: Sketch-Based Motion Editing for Articulated Characters". In: *ACM Trans. Graph.* 35.4 (July 2016). ISSN: 0730-0301. DOI: 10.1145/2897824.2925970. URL: https://doi.org/10.1145/2897824.2925970.

[31] Ravi Chugh et al. "Programmatic and Direct Manipulation, Together at Last". In: *ACM SIGPLAN Notices* 51.6 (2016), pp. 341–354. DOI: 10.1145/2980983.2908103.

[32] Andy Cockburn, Amy Karlson, and Benjamin B. Bederson. "A Review of Overview+Detail, Zooming, and Focus+Context Interfaces". In: *ACM Computing Surveys* 41.1 (2008), pp. 1–31. DOI: 10.1145/1456650.1456652.

[33] Robert L. Cook. "Shade Trees". In: *Proceedings of the 11th Annual Conference on Computer Graphics and Interactive Techniques*. SIGGRAPH '84. ACM, 1984, pp. 223–231. DOI: 10.1145/800031.808602.

[34] Navneet Dalal and Bill Triggs. "Histograms of Oriented Gradients for Human Detection". In: *Proceedings of the 2005 IEEE Computer Society Conference on Computer Vision and Pattern Recognition*. 2005, pp. 886–893. DOI: 10.1109/CVPR.2005.177.

[35] Jack B. Dennis and David P. Misunas. "A Preliminary Architecture for a Basic Data-Flow Processor". In: *Proceedings of the 2nd Annual Symposium on Computer Architecture*. ACM, 1974, pp. 126–132. DOI: 10.1145/800123.803913.

[36] Julie Dorsey et al. "The Mental Canvas: A Tool for Conceptual Architectural Design and Analysis". In: *Proc. of Pacific Conference on Computer Graphics and Applications*. PG '07. USA: IEEE Computer Society, 2007, pp. 201–210. ISBN: 0769530095.

[37] Pierre Dragicevic. "Fair statistical communication in HCI". In: *Modern statistical methods for HCI* (2016), pp. 291–330.

[38] Pierre Dragicevic et al. "Video browsing by direct manipulation". In: *Proc. CHI '08*. 2008, pp. 237–246.

[39] Pif Edwards et al. "JALI-Driven Expressive Facial Animation and Multilingual Speech in Cyberpunk 2077". In: *Special Interest Group on Computer Graphics and Interactive Techniques Conference Talks*. SIGGRAPH '20. Virtual Event, USA: Association for Computing Machinery, 2020. ISBN: 9781450379717. DOI: 10.1145/3388767.3407339. URL: https://doi.org/10.1145/3388767.3407339.

[40] Koos Eissen and Roselien Steur. *Sketching: Drawing Techniques for Product Designers*. Bis Publishers, 2008.

[41] Rosenberg Ekman. *What the face reveals: Basic and applied studies of spontaneous expression using the Facial Action Coding System (FACS)*. Oxford University Press, USA, 1997.

[42] Jennifer Fernquist, Tovi Grossman, and George Fitzmaurice. "Sketch-sketch revolution: an engaging tutorial system for guided sketching and application learning". In: *Proc. of the ACM Symposium on User Interface Software and Technology*. UIST '11. Santa Barbara, California, USA: Association for Computing Machinery, 2011, pp. 373–382. ISBN: 9781450307161. DOI: 10.1145/2047196.2047245. URL: https://doi.org/10.1145/2047196.2047245.

[43] Jeanne Ferrante, Karl J. Ottenstein, and Joe D. Warren. "The Program Dependence Graph and Its Use in Optimization". In: *ACM Transactions on Programming Languages and Systems* 9.3 (1987), pp. 319–349. DOI: 10.1145/24039.24041.

[44] Figma. *Figma AI: Your Creativity, Unblocked with Figma AI*. 2026. URL: https://www.figma.com/ai/ (visited on 05/05/2026).

[45] M. A. Fischler and H. C. Wolf. "Locating Perceptually Salient Points on Planar Curves". In: *IEEE Trans. Pattern Anal. Mach. Intell.* 16.2 (Feb. 1994), pp. 113–129. ISSN: 0162-8828. DOI: 10.1109/34.273737. URL: https://doi.org/10.1109/34.273737.

[46] Myron Flickner et al. "Query by Image and Video Content: The QBIC System". In: *Computer* 28.9 (1995), pp. 23–32. DOI: 10.1109/2.410146.

[47] Mitchell Foo et al. "Curompt: A Spatially Situated Interface for Generative AI in 3D Design Software". In: *Proceedings of the IEEE Symposium on Visual Languages and Human-Centric Computing*. 2025, pp. 439–440. DOI: 10.1109/VL-HCC65237.2025.00065.

[48] Krzysztof Z Gajos et al. "Exploring the design space for adaptive graphical user interfaces". In: *Proceedings of the working conference on Advanced visual interfaces*. 2006, pp. 201–208.

[49] Sarah Gibson et al. "Design galleries: A general approach to setting parameters for computer graphics and animation". In: (1997).

[50] Michael Gleicher. "Integrating constraints and direct manipulation". In: *Symposium on Interactive 3 D Graphics: Proceedings of the 1992 symposium on Interactive 3 D graphics*. Vol. 1992. 1992, pp. 171–174.

[51] Bruce Gooch and Amy Gooch. *Non-photorealistic rendering*. CRC Press, 2001.

[52] Google Developers Blog. *Introducing Gemini 2.5 Flash Image, Our State-of-the-Art Image Model*. 2025. URL: https://developers.googleblog.com/introducing-gemini-2-5-flash-image/ (visited on 05/05/2026).

[53] Thomas R. G. Green and Marian Petre. "Usability Analysis of Visual Programming Environments: A 'Cognitive Dimensions' Framework". In: *Journal of Visual Languages and Computing* 7.2 (1996), pp. 131–174. DOI: 10.1006/jvlc.1996.0009.

[54] Martin Guay, Marie-Paule Cani, and Rémi Ronfard. "The Line of Action: An Intuitive Interface for Expressive Character Posing". In: *ACM Trans. Graph.* 32.6 (Nov. 2013). ISSN: 0730-0301. DOI: 10.1145/2508363.2508397. URL: https://doi.org/10.1145/2508363.2508397.

[55] Martin Guay, Marie-Paule Cani, and Rémi Ronfard. “The line of action: an intuitive interface for expressive character posing”. In: *ACM Transactions on Graphics (TOG)* 32.6 (2013), pp. 1–8.

[56] Paul Haeberli. “ConMan: A Visual Programming Language for Interactive Graphics”. In: *Proceedings of the 15th Annual Conference on Computer Graphics and Interactive Techniques*. SIGGRAPH ’88. ACM, 1988, pp. 103–111. DOI: 10.1145/54852.378490.

[57] Mark Hamilton et al. “Unsupervised Semantic Segmentation by Distilling Feature Correspondences”. In: *International Conference on Learning Representations*. 2022. URL: https://openreview.net/forum?id=SaKO6z6Hl0c.

[58] Erik Härkönen et al. “GANSpace: Discovering Interpretable GAN Controls”. In: *Advances in Neural Information Processing Systems*. Vol. 33. 2020, pp. 9841–9850.

[59] Sandra G Hart and Lowell E Staveland. “Development of NASA-TLX (Task Load Index): Results of empirical and theoretical research”. In: *Human mental workload* 1.3 (1988), pp. 139–183.

[60] Sonia Hashim, Tobias Höllerer, and Jennifer Jacobs. “Drawing Transforms: A Unifying Interaction Primitive to Procedurally Manipulate Graphics across Style, Space, and Time”. In: *Proc. CHI ’23*. 2023, pp. 1–15.

[61] Ivan Herman, Guy Melançon, and M. Scott Marshall. “Graph Visualization and Navigation in Information Visualization: A Survey”. In: *IEEE Transactions on Visualization and Computer Graphics* 6.1 (2000), pp. 24–43. DOI: 10.1109/2945.841119.

[62] Aaron Hertzmann. “Introduction to 3d non-photorealistic rendering: Silhouettes and outlines”. In: *Non-Photorealistic Rendering. SIGGRAPH* 99.1 (1999).

[63] Irina Higgins et al. “beta-VAE: Learning Basic Visual Concepts with a Constrained Variational Framework”. In: *International Conference on Learning Representations*. 2017.

[64] Harry Hochheiser and Ben Shneiderman. “Dynamic Query Tools for Time Series Data Sets: Timebox Widgets for Interactive Exploration”. In: *Information Visualization* 3.1 (2004), pp. 1–18. DOI: 10.1057/palgrave.ivs.9500061.

[65] Edwin L. Hutchins, James D. Hollan, and Donald A. Norman. “Direct Manipulation Interfaces”. In: *Human-Computer Interaction* 1.4 (1985), pp. 311–338. DOI: 10.1207/s15327051hci0104_2.

[66] T. Igarashi, T. Moscovich, and J. F. Hughes. “Spatial Keyframing for Performance-Driven Animation”. In: *ACM SIGGRAPH 2007 Courses*. SIGGRAPH ’07. San Diego, California: Association for Computing Machinery, 2007, 25–es. ISBN: 9781450318235. DOI: 10.1145/1281500.1281536. URL: https://doi.org/10.1145/1281500.1281536.

[67] Takeo Igarashi, Satoshi Matsuoka, and Hidehiko Tanaka. “Teddy: A Sketching Interface for 3D Freeform Design”. In: *Proceedings of the 26th Annual Conference on Computer Graphics and Interactive Techniques*. SIGGRAPH ’99. ACM Press/Addison-Wesley Publishing Co., 1999, pp. 409–416. DOI: 10.1145/311535.311602.

[68] Takeo Igarashi, Satoshi Matsuoka, and Hidehiko Tanaka. "Teddy: A Sketching Interface for 3D Freeform Design". In: SIGGRAPH '99. New York, NY, USA: ACM Press/Addison-Wesley Publishing Co., 1999, pp. 409–416. ISBN: 0-201-48560-5. DOI: 10.1145/311535.311602. URL: http://dx.doi.org/10.1145/311535.311602.

[69] Takeo Igarashi, Tomer Moscovich, and John F. Hughes. "As-Rigid-As-Possible Shape Manipulation". In: *ACM Transactions on Graphics* 24.3 (2005), pp. 1134–1141. DOI: 10.1145/1073204.1073323.

[70] International Commission on Illumination. *Colorimetry, 3rd Edition*. CIE Publication 15:2004. 2004.

[71] Hiroshi Ishii and Brygg Ullmer. "Tangible Bits: Towards Seamless Interfaces between People, Bits and Atoms". In: *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*. CHI '97. Association for Computing Machinery, 1997, pp. 234–241. DOI: 10.1145/258549.258715.

[72] Robert J. K. Jacob et al. "Reality-Based Interaction: A Framework for Post-WIMP Interfaces". In: *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*. CHI '08. Association for Computing Machinery, 2008, pp. 201–210. DOI: 10.1145/1357054.1357089.

[73] Alec Jacobson et al. "Bounded biharmonic weights for real-time deformation". In: *ACM Trans. Graph.* 30.4 (July 2011). ISSN: 0730-0301. DOI: 10.1145/2010324.1964973. URL: https://doi.org/10.1145/2010324.1964973.

[74] Gabe Johnson et al. "Computational Support for Sketching in Design: A Review". In: *Foundations and Trends in Human-Computer Interaction* 2 (Jan. 2009), pp. 1–93. DOI: 10.1561/1100000013.

[75] Jeff A. Johnson et al. "The Xerox Star: A Retrospective". In: *Computer* 22.9 (1989), pp. 11–29. DOI: 10.1109/2.35211.

[76] Thorsten Karrer et al. "Dragon: a direct manipulation interface for frame-accurate in-scene video navigation". In: *Proc. CHI '08*. 2008, pp. 247–250.

[77] Rubaiat Habib Kazi et al. "Draco: bringing life to illustrations with kinetic textures". In: *Proc. of the SIGCHI Conference on Human Factors in Computing Systems*. CHI '14. Toronto, Ontario, Canada: Association for Computing Machinery, 2014, pp. 351–360. ISBN: 9781450324731. DOI: 10.1145/2556288.2556987. URL: https://doi.org/10.1145/2556288.2556987.

[78] Rubaiat Habib Kazi et al. "Kitty: sketching dynamic and interactive illustrations". In: *Proc. of the 27th Annual ACM Symposium on User Interface Software and Technology*. UIST '14. Honolulu, Hawaii, USA: Association for Computing Machinery, 2014, pp. 395–405. ISBN: 9781450330695. DOI: 10.1145/2642918.2647375. URL: https://doi.org/10.1145/2642918.2647375.

[79] Joonho Kim, Fanny Chevalier, and Karan Singh. "Squidgets: Sketch-based Widget Design for Scene Manipulation". In: *Proceedings of the 38th Annual ACM Symposium on User Interface Software and Technology*. UIST '25. Busan, Republic of Korea: Association for Computing Machinery, 2025. ISBN: 979-8-4007-2037-6. DOI: 10.1145/3746059.3747690. URL: https://doi.org/10.1145/3746059.3747690.

[80] Joonho Kim and Karan Singh. "Optimizing UI Layouts for Deformable Face-Rig Manipulation". In: *ACM Transactions on Graphics* (2021). DOI: 10.1145/3450626.3459842. URL: https://doi.org/10.1145/3450626.3459842.

[81] Joonho Kim and Karan Singh. "Optimizing UI Layouts for Deformable Face-Rig Manipulation". In: *ACM Transactions on Graphics (TOG)* 40.4 (July 2021). ISSN: 0730-0301. DOI: 10.1145/3450626.3459842. URL: https://doi.org/10.1145/3450626.3459842.

[82] Vladimir G. Kim et al. "Exploring Collections of 3D Models Using Fuzzy Correspondences". In: *ACM Transactions on Graphics* 31.4 (2012), 54:1–54:11. DOI: 10.1145/2185520.2185550.

[83] Alexander Kirillov et al. "Segment Anything". In: *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 2023, pp. 4015–4026. DOI: 10.1109/ICCV51070.2023.00371.

[84] Vladimir Kolmogorov and Ramin Zabih. "What Energy Functions Can Be Minimized via Graph Cuts?" In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 26.2 (2004), pp. 147–159. DOI: 10.1109/TPAMI.2004.1262177.

[85] Yuki Koyama, Daisuke Sakamoto, and Takeo Igarashi. "Crowd-powered parameter analysis for visual design exploration". In: *Proceedings of the 27th annual ACM symposium on User interface software and technology*. ACM. 2014, pp. 65–74.

[86] Paul G Kry, Doug L James, and Dinesh K Pai. "Eigenskin: real time large deformation character skinning in hardware". In: *Proceedings of the 2002 ACM SIGGRAPH/Eurographics symposium on Computer animation*. 2002, pp. 153–159.

[87] Bum Chul Kwon et al. "Axisketcher: Interactive nonlinear axis mapping of visualizations through user drawings". In: *IEEE transactions on visualization and computer graphics* 23.1 (2016), pp. 221–230.

[88] Manfred Lau et al. "Face poser: Interactive modeling of 3D facial expressions using facial priors". In: *ACM Transactions on Graphics (TOG)* 29.1 (2009), p. 3.

[89] Binh H. Le, Mingyang Zhu, and Zhigang Deng. "Marker Optimization for Facial Motion Acquisition and Deformation". In: *IEEE Transactions on Visualization and Computer Graphics* 19.11 (Nov. 2013), pp. 1859–1871. ISSN: 1077-2626. DOI: 10.1109/TVCG.2013.84. URL: https://doi.org/10.1109/TVCG.2013.84.

[90] Bongshin Lee et al. "Task Taxonomy for Graph Visualization". In: *Proceedings of the Working Conference on Advanced Visual Interfaces*. AVI '06. ACM, 2006, pp. 81–85. DOI: 10.1145/1133265.1133288.

[91] Juyong Lee et al. "ShadowDraw: Real-Time User Guidance for Freehand Drawing". In: *ACM Transactions on Graphics* 30.4 (2011). DOI: 10.1145/2010324.1964921.

[92] Yunjin Lee et al. "Line drawings via abstracted shading". In: *ACM Trans. Graph.* 26.3 (2007), p. 18. DOI: 10.1145/1276377.1276400. URL: https://doi.org/10.1145/1276377.1276400.

[93] John P Lewis and Ken-ichi Anjyo. "Direct manipulation blendshapes". In: *IEEE Computer Graphics and Applications* 30.4 (2010), pp. 42–50.

[94] Hao Li et al. "Realtime Facial Animation with On-the-Fly Correctives". In: *ACM Trans. Graph.* 32.4 (July 2013). ISSN: 0730-0301. DOI: 10.1145/2461912.2462019. URL: https://doi.org/10.1145/2461912.2462019.

[95] Markus Lipp, Peter Wonka, and Michael Wimmer. "Interactive Visual Editing of Grammars for Procedural Architecture". In: *ACM Transactions on Graphics* 27.3 (2008). DOI: 10.1145/1360612.1360701.

[96] Vivian Liu, Han Qiao, and Lydia B. Chilton. "Opal: Multimodal Image Generation for News Illustration". In: *Proceedings of the 35th Annual ACM Symposium on User Interface Software and Technology*. Association for Computing Machinery, 2022. DOI: 10.1145/3526113.3545621.

[97] Kok-Lim Low. "Linear least-squares optimization for point-to-plane icp surface registration". In: ().

[98] James MacQueen. "Some Methods for Classification and Analysis of Multivariate Observations". In: *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability* 1 (1967), pp. 281–297.

[99] Joe Marks et al. "Design Galleries: A General Approach to Setting Parameters for Computer Graphics and Animation". In: *Proceedings of the 24th Annual Conference on Computer Graphics and Interactive Techniques* (1997), pp. 389–400. DOI: 10.1145/258734.258887.

[100] Damien Masson et al. "DirectGPT: A Direct Manipulation Interface to Interact with Large Language Models". In: *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems*. Association for Computing Machinery, 2024. DOI: 10.1145/3613904.3642462.

[101] Richard E. Mayer and Roxana Moreno. "A Split-Attention Effect in Multimedia Learning: Evidence for Dual Processing Systems in Working Memory". In: *Journal of Educational Psychology* 90.2 (1998), pp. 312–320. DOI: 10.1037/0022-0663.90.2.312.

[102] D. Mendes et al. "A Survey on 3D Virtual Object Manipulation: From the Desktop to Immersive Virtual Environments". In: *Computer Graphics Forum* 38.1 (2019), pp. 21–45. DOI: https://doi.org/10.1111/cgf.13390. eprint: https://onlinelibrary.wiley.com/doi/pdf/10.1111/cgf.13390. URL: https://onlinelibrary.wiley.com/doi/abs/10.1111/cgf.13390.

[103] Elie Michel and Tamy Boubekeur. "DAG Amendment for Inverse Control of Parametric Shapes". In: *ACM Transactions on Graphics* 40.4 (2021), 173:1–173:14. DOI: 10.1145/3450626.3459823.

[104] Erick Mikiten. "The Layers of Architectural Design". In: *Concept App*. URL: https://concepts.app/en/stories/the-layers-of-architectural-design/.

[105] Ben Mildenhall et al. "NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis". In: *Proceedings of the European Conference on Computer Vision*. 2020, pp. 405–421. DOI: 10.1007/978-3-030-58452-8_24.

[106] José Carlos Miranda et al. "Sketch express: facial expressions made easy". In: *Proceedings of the Eighth Eurographics Symposium on Sketch-Based Interfaces and Modeling*. ACM. 2011, pp. 87–94.

[107] Niloy J. Mitra et al. "Illustrating how mechanical assemblies work". In: *Commun. ACM* 56.1 (2013), pp. 106–114. DOI: 10.1145/2398356.2398379. URL: http://doi.acm.org/10.1145/2398356.2398379.

[108] Niloy J. Mitra et al. "Symmetry in 3D Geometry: Extraction and Applications". In: *EUROGRAPHICS State-of-the-art Report*. 2012. DOI: 10.1111/cgf.12010. URL: http://dx.doi.org/10.1111/cgf.12010.

[109] Matthias Müller et al. “Meshless deformations based on shape matching”. In: *ACM transactions on graphics (TOG)* 24.3 (2005), pp. 471–478.

[110] Fionn Murtagh and Pedro Contreras. “Algorithms for hierarchical clustering: an overview”. In: *WIREs Data Mining and Knowledge Discovery* 2.1 (2012), pp. 86–97. DOI: https://doi.org/10.1002/widm.53. eprint: https://onlinelibrary.wiley.com/doi/pdf/10.1002/widm.53. URL: https://onlinelibrary.wiley.com/doi/abs/10.1002/widm.53.

[111] Brad A. Myers. “Demonstrational Interfaces: A Step Beyond Direct Manipulation”. In: *Computer* 25.8 (1992), pp. 61–73. DOI: 10.1109/2.153286.

[112] Brad A. Myers. “Taxonomies of Visual Programming and Program Visualization”. In: *Journal of Visual Languages & Computing* 1.1 (1990), pp. 97–123. DOI: 10.1016/S1045-926X(05)80036-9.

[113] Victor Navone. “Facial Animation for Feature Animated Films: Animating stylized facial expressions”. In: *https://www.thegnomonworkshop.com/tutorials/facial-animation-for-feature-animated-films* (2020).

[114] Andrew Nealen et al. “A Sketch-Based Interface for Detail-Preserving Mesh Editing”. In: *ACM Transactions on Graphics (Proceedings of ACM SIGGRAPH)* 24.3 (2005), pp. 1142–1147.

[115] L. Olsen et al. “A Taxonomy of Modeling Techniques using Sketch-Based Interfaces”. In: *Eurographics 2008 - State of the Art Reports*. Ed. by Theoharis Theoharis and Philip Dutre. The Eurographics Association, 2008. DOI: 10.2312/egst.20081044.

[116] OpenAI. *The New ChatGPT Images Is Here*. 2025. URL: https://openai.com/index/new-chatgpt-images-is-here/ (visited on 05/05/2026).

[117] Jason Osipa. *Stop Staring: Facial Modeling and Animation Done Right*. 3rd. USA: SYBEX Inc., 2010. ISBN: 0470609907.

[118] Maks Ovsjanikov et al. “Exploration of Continuous Variability in Collections of 3D Shapes”. In: *ACM Transactions on Graphics* 30.4 (2011). DOI: 10.1145/2010324.1964928.

[119] Patrick Paczkowski et al. “Insitu: Sketching Architectural Designs in Context”. In: *ACM Transactions on Graphics* 30.6 (2011). DOI: 10.1145/2070781.2024206.

[120] Karran Pandey, Fanny Chevalier, and Karan Singh. “Juxtaform: Interactive Visual Summarization for Exploratory Shape Design”. In: *ACM Transactions on Graphics* 42.4 (2023). DOI: 10.1145/3592436.

[121] Rick Parent. *Computer animation: algorithms and techniques*. Newnes, 2012.

[122] Yoav I. H. Parish and Pascal Müller. “Procedural Modeling of Cities”. In: *Proceedings of the 28th Annual Conference on Computer Graphics and Interactive Techniques*. SIGGRAPH ’01. ACM, 2001, pp. 301–308. DOI: 10.1145/383259.383292.

[123] Przemyslaw Prusinkiewicz and Aristid Lindenmayer. *The Algorithmic Beauty of Plants*. Springer-Verlag, 1990.

[124] Robin Rombach et al. “High-Resolution Image Synthesis with Latent Diffusion Models”. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2022, pp. 10684–10695. DOI: 10.1109/CVPR52688.2022.01042.

[125] Peter J. Rousseeuw. "Silhouettes: A Graphical Aid to the Interpretation and Validation of Cluster Analysis". In: *Journal of Computational and Applied Mathematics* 20 (1987), pp. 53–65. DOI: 10.1016/0377-0427(87)90125-7.

[126] Dean Rubine. "Specifying Gestures by Example". In: *Proc. SIGGRAPH '91*. SIGGRAPH '91. New York, NY, USA: Association for Computing Machinery, 1991, pp. 329–337. ISBN: 0897914368. DOI: 10.1145/122718.122753. URL: https://doi.org/10.1145/122718.122753.

[127] Stephanie Santosa et al. "Direct Space-Time Trajectory Control for Visual Media Editing". In: *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*. CHI '13. ACM, 2013, pp. 1149–1158.

[128] Ryan Schmidt, Karan Singh, and Ravin Balakrishnan. "Sketching and Composing Widgets for 3D Manipulation". In: *Computer Graphics Forum* 27.2 (2008), pp. 301–310. DOI: 10.1111/j.1467-8659.2008.01127.x.

[129] Ryan Schmidt, Karan Singh, and Ravin Balakrishnan. "Sketching and composing widgets for 3d manipulation". In: *Computer graphics forum*. Vol. 27. 2. Wiley Online Library. 2008, pp. 301–310.

[130] Thomas W Sederberg and Scott R Parry. "Free-form deformation of solid geometric models". In: *Proceedings of the 13th annual conference on Computer graphics and interactive techniques*. 1986, pp. 151–160.

[131] Mike Seymour. "Bebyface in Bebylon". In: (2019). URL: https://www.fxguide.com/fxfeatured/bebyface-in-bebylon/.

[132] Mike Seymour. "Making Thanos Face the Avengers". In: (2018). URL: https://www.fxguide.com/fxfeatured/making-thanos-face-the-avengers/.

[133] Mike Seymour. "Put your (digital) game face on". In: (2016). URL: https://www.fxguide.com/fxfeatured/put-your-digital-game-face-on/.

[134] Omar Shaikh et al. "Learning Next Action Predictors from Human-Computer Interaction". In: *arXiv preprint arXiv:2603.05923* (2026). URL: https://arxiv.org/abs/2603.05923.

[135] Ben Shneiderman. "Direct Manipulation: A Step Beyond Programming Languages". In: *Computer* 16.8 (1983), pp. 57–69. DOI: 10.1109/MC.1983.1654471.

[136] Tarique Siddiqui et al. "ShapeSearch: A Flexible and Efficient System for Shape-Based Exploration of Trendlines". In: *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*. SIGMOD '20. Association for Computing Machinery, 2020, pp. 51–65. DOI: 10.1145/3318464.3389722.

[137] Karan Singh and Eugene Fiume. "Wires: a geometric deformation technique". In: *Proceedings of the 25th annual conference on Computer graphics and interactive techniques*. 1998, pp. 405–414.

[138] David Canfield Smith et al. "Designing the Star User Interface". In: *BYTE* 7.4 (1982), pp. 242–282.

[139] John R. Smith and Shih-Fu Chang. "VisualSEEk: A Fully Automated Content-Based Image Query System". In: *Proceedings of the Fourth ACM International Conference on Multimedia*. MULTIMEDIA '96. Association for Computing Machinery, 1996, pp. 87–98. DOI: 10.1145/244130.244151.

[140] Marc P Stevens, Robert C Zeleznik, and John F Hughes. "An architecture for an extensible 3D interface toolkit". In: *Proceedings of the 7th annual ACM symposium on User interface software and technology*. 1994, pp. 59–67.

[141] Paul S. Strauss, Rikk Carey, et al. *An Object-Oriented 3D Graphics Toolkit*. Open Inventor. ACM SIGGRAPH Course Notes, 1992.

[142] Tanasai Sucontphunt et al. "Interactive 3D facial expression posing through 2D portrait manipulation". In: *Proceedings of graphics interface 2008*. Canadian Information Processing Society. 2008, pp. 177–184.

[143] Super Mario Wiki. *Face-Lift (minigame)*. 2026. URL: https://www.mariowiki.com/Face_Lift (visited on 05/12/2026).

[144] Super Mario Wiki. *File:FaceLift - Wario.png*. 2026. URL: https://www.mariowiki.com/File:FaceLift_-_Wario.png (visited on 05/12/2026).

[145] Ivan E. Sutherland. "Sketch Pad a Man-machine Graphical Communication System". In: *Proc. of the SHARE Design Automation Workshop*. DAC '64. New York, NY, USA: ACM, 1964, pp. 6.329–6.346. DOI: 10.1145/800265.810742. URL: http://doi.acm.org/10.1145/800265.810742.

[146] Ivan E. Sutherland. *Sketchpad: A Man-Machine Graphical Communication System*. PhD thesis. Massachusetts Institute of Technology, 1963.

[147] William Robert Sutherland. "The On-Line Graphical Specification of Computer Procedures". PhD thesis. Massachusetts Institute of Technology, 1966.

[148] Richard S. Sutton. *The Bitter Lesson*. 2019. URL: http://www.incompleteideas.net/IncIdeas/BitterLesson.html (visited on 06/10/2026).

[149] Ryo Suzuki et al. "RealitySketch: Embedding Responsive Graphics and Visualizations in AR through Dynamic Sketching". In: *Proc. UIST '20*. UIST '20. Virtual Event, USA: Association for Computing Machinery, 2020, pp. 166–181. ISBN: 9781450375146. DOI: 10.1145/3379337.3415892. URL: https://doi.org/10.1145/3379337.3415892.

[150] Daniel Sýkora, Jan Buriánek, and Jiří Žára. "Sketching Cartoons by Example". In: *Proc. of Eurographics Workshop on Sketch-Based Interfaces and Modeling*. 2005, pp. 27–34.

[151] Daniel Sýkora, John Dingliana, and Steven Collins. "LazyBrush: Flexible Painting Tool for Hand-drawn Cartoons". In: *Computer Graphics Forum* 28.2 (2009), pp. 599–608.

[152] Jerry O Talton et al. "Exploratory modeling with collaborative design spaces". In: *ACM Transactions on Graphics-TOG* 28.5 (2009), p. 167.

[153] J. Rafael Tena, Fernando De la Torre, and Iain Matthews. "Interactive Region-Based Linear 3D Face Models". In: *ACM Trans. Graph.* 30.4 (July 2011). ISSN: 0730-0301. DOI: 10.1145/2010324.1964971. URL: https://doi.org/10.1145/2010324.1964971.

[154] Ayush Tewari et al. “MoFA: Model-based Deep Convolutional Face Autoencoder for Unsupervised Monocular Reconstruction”. In: *The IEEE International Conference on Computer Vision (ICCV)*. 2017.

[155] James T. Todd. “The Visual Perception of 3D Shape”. In: *Trends in Cognitive Sciences* 8.3 (2004), pp. 115–121.

[156] Deepak Tolani, Ambarish Goswami, and Norman I. Badler. “Real-Time Inverse Kinematics Techniques for Anthropomorphic Limbs”. In: *Graphical Models* 62.5 (2000), pp. 353–388. DOI: 10.1006/gmod.2000.0528.

[157] Theophanis Tsandilas, Anastasia Bezerianos, and Thibaut Jacob. “Sketchsliders: Sketching widgets for visual exploration on wall displays”. In: *Proc. CHI ’15*. 2015, pp. 3255–3264.

[158] Joe H. Ward and Margaret E. Hook. “Hierarchical Grouping to Optimize an Objective Function”. In: *Journal of the American Statistical Association* 58.301 (1963), pp. 236–244.

[159] Colin Ware and Robert Bobrow. “Supporting Visual Queries on Medium-Sized Node-Link Diagrams”. In: *Information Visualization* 4.1 (2005), pp. 49–58. DOI: 10.1057/palgrave.ivs.9500090.

[160] Thibaut Weise et al. “Realtime performance-based facial animation”. In: *ACM transactions on graphics (TOG)*. Vol. 30. 4. ACM. 2011.

[161] Lance Williams. “Performance-driven facial animation”. In: *ACM SIGGRAPH Computer Graphics*. Vol. 24. 4. ACM. 1990, pp. 235–242.

[162] Jacob O. Wobbrock, Andrew D. Wilson, and Yang Li. “Gestures without Libraries, Toolkits or Training: A $1 Recognizer for User Interface Prototypes”. In: *Proc. UIST ’07*. UIST ’07. Newport, Rhode Island, USA: Association for Computing Machinery, 2007, pp. 159–168. ISBN: 9781595936790. DOI: 10.1145/1294211.1294238. URL: https://doi.org/10.1145/1294211.1294238.

[163] Haijun Xia et al. “Object-Oriented Drawing”. In: *Proc. CHI ’16*. CHI ’16. San Jose, California, USA: Association for Computing Machinery, 2016, pp. 4610–4621. ISBN: 9781450333627. DOI: 10.1145/2858036.2858075. URL: https://doi.org/10.1145/2858036.2858075.

[164] Baoxuan Xu et al. “True2Form: 3D curve networks from 2D sketches via selective regularization”. In: *ACM Transactions on Graphics* 33.4 (2014).

[165] J. D. Zamfirescu-Pereira et al. “Why Johnny Can’t Prompt: How Non-AI Experts Try (and Fail) to Design LLM Prompts”. In: *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*. Association for Computing Machinery, 2023. DOI: 10.1145/3544548.3581388.

[166] R. C. Zeleznik, K. P. Herndon, and J. F. Hughes. “Sketch: An Interface for Sketching 3D Scenes”. In: *Proc. SIGGRAPH ’96*. 1996, pp. 163–170.

[167] Eduard Zell et al. “Facial retargeting with automatic range of motion alignment”. In: *ACM Transactions on Graphics (TOG)* 36.4 (2017), p. 154.

[168] Jun-Yan Zhu, Yong Jae Lee, and Alexei A. Efros. “AverageExplorer: Interactive Exploration and Alignment of Visual Data Collections”. In: *ACM Transactions on Graphics* 33.4 (2014). DOI: 10.1145/2601097.2601145.

[169] J. Zimmerman, A. Nealen, and M. Alexa. "SilSketch: Automated Sketch-Based Editing of Surface Meshes". In: *Proceedings of the Eurographics Workshop on Sketch-Based Interfaces and Modeling*. Eurographics Association, 2007.